Skip to main content

Feedzai Platform

Feedzai is a platform for managing financial risk. It relies on a cutting-edge enterprise Data Science platform that integrates big data analytics and Machine Learning techniques with a native runtime production environment, and a Risk Operations platform focused on boosting the efficiency of financial crime prevention and increase detection accuracy.

Follow these guides to learn how to install and operate the Feedzai platform.

Learn more about Feedzai platform

Default Page

Markdown page example

You don't need React to write simple standalone pages.

Source: https://cam.feedzai.com/cs-hsbc-rma/0.2.0/blog/archive

On this page

Authentication

Feedzai allows you to authenticate and protect your data through JSON Web Tokens (JWTs). A JSON Web Token (JWT) is a piece of information that is transmitted as a JSON object from clients to Feedzai, and its purpose is to ensure that client requests (e.g. transactions for processing) have not been compromised.

These tokens are generated using the clientâ□□s private key, which can be verified with the corresponding public key. The public key must be shared with Feedzai beforehand to verify that the token sent with the request belongs to the client.

Overviewâ□□

JWTs protect against different attacks using the following mechanisms:

- **Encryption/signing** so that only authenticated users can access sensitive data.
- **Expiration dates** to protect against replay attacks.
- **JWT revocation** to block exposed JWTs as soon as possible.

You can see more details on integrating JWTs with your software in the sub-sections below, but in general you will:

1. Generate a private-public key pair to sign the JWT.
2. Provide the public key to Feedzai in JSON Web Key Set format.
3. Frequently generate new JWTs with a reduced validity period (e.g. 5 minutes). Each JWT is signed with your private key generated in step 1.
4. For each request to Feedzai, include a JWT in the `Authorization` header.

Given the recommended algorithm and parameters, you can choose the tool that best fits your system. A comprehensive list of tools is available at the libraries section of jwt.io.

danger

Third-party security vulnerabilities: When choosing a library, make sure it has no known security vulnerabilities.

JSON Web Keyâ□□

A key pair is necessary to cryptographically sign and verify the JWTs sent with your requests.

- You will use the private key to sign your JWTs.
- Feedzai will use the public key to verify the signature, and therefore the authenticity of the requests.

To create a JWK set, you need to choose the signing algorithm and the JWK parameters. Feedzai recommends the `RS512` algorithm (â□□RSASSA-PKCS1-v1_5 using SHA-512â□□) with a 4096-bit key. See the sections below to learn how to generate a JWK set with standard tools.

danger

Beware of private key parameters: Construct the JWK set so that you **do not** send any of the private key parameters.

Your JWK set should look similar to this:

```
{
    "keys": [
        {
            "e": "AQAB",
            "n":
```

```
"187EWmXQwXi08P8bqSOSeOiyOyJGsojjmeL6qwLLjTBfxxeNgTtivzG8YdCgrLnBQWy4j9FqQ5wbqyOwf6CBv6xj9ZnqyyS
cle7ubAKRt4RKtv4yrxVb1g5ZWcfD6yWDIH1jRy5oGQEwWRg9918Z73OS4SlYe8rMDiWkFPybt1vfGBdGqbvm374XNfLRb0s
F7cPCRcBa3Vdzmg5BgoN9kFUXIvFKjXdgZQGEEIonLL8ZSmqq993VBVLOPh02OiJ91DVvb9JMIFAqROHQNyzchbOWgEPUKoR
BRDXSCus9ZSpHuUl7iL7qKLaQdPyO1ffIG0o7kF6allFG6OBcnzqX8r",
          "kty": "RSA",
          "kid": "a28db6df-f2f4-4267-be46-371502768aa1sig",
          "use": "sig"
       }
    ]
}

{
    "kid":"mykey2",
    "kty":"RSA",
    "alg":"RS512",
    "use":"sig",
    "e":"AQAB",
    "n":"k51ZPyu9gocn508aXzob_hSKaIAnJApY4pY3h1oym1Z15-6DN3WxB3YHUyXPRd-iBzF7177XjQAywTzAaz-
PKLZ4bC52noanLx7Ihyf3nQAOwhEv16zaJLI-
HqMEMp6JKPjKJ8LKanVnvxWNI0zUBwcBOwt9cxph0evyVO94cXA0RZxUZxa7C3FuSYkO2zWorMH0ZkC3harkJx0C9Q7nBiGW
PQmB8ZkLx04iWlm8ldyxDJPKl-PC4-
gS3y2mgPfZVAaK_VGod88PUmF4vFax7a03v2VLtn9wm8UWs9SXATxJ5cExgIIvFkdsZAcS3hWnhUe3Of4zAEbKh2Ah5DtfY9
xvtsU8FGlak7wLCwVnubWWglI2hmFvFU7Jsyzb_L-
jD7XCYrg7QuFV8mX2jVcTbsSxnd8GLRNbvmztvsjfaQHXEHraTRJFoGO1Y3JXDiY8Kj40OQXxBNXaFxHda9lBfzWfDkgWvcy
eiTPeWpKc6cS1zcDaQNRzZr3aZR-
gI4U8YTAXcIKDdom2GCQSx9WfmFq3uAj95duL5CgyGfw7L-02c6ckRr6kBAKebURkSsZXOyTO8bdbS8GzPMD59CrRL2SnJJe
qHyrEav8JCu0x3UCXbyqlfqQgzVsXNpFUMIS-sUhe3nyVQOeJ_WoYq7oAbjLpx_Bw5elQJf4zJbJd2Pk"
}
```

Copy

In the aforementioned example, the following parameters are used:

Name	Extended name	Description
`e`	Exponent	RSA public exponent e.
`n`	Modulus	RSA public modulus n.
`kty`	Key Type	Should be `RSA` to symbolize you are using the RS512 algorithm.
`kid`	Key Id	Should specify a key identifier of your choice.
`use`	Public Key Use	Should be `sig` to note this key will be used to verify JWTs.

How to generate a key-pair and JWKâ□□

1. Unpack the tar.gz artifact.
2. Check generator options:

```
./bin/generate-asymmetric --help
```

Copy

info

Make sure you repeat steps 1 through 3 for a total of two private-public key pairs: one for `prod` and another one for `qa` .

3. Run the JWT Generator using the following command. Adjust the parameter values according to your needs:

```
./bin/generate-asymmetric \
  --mode JWK \
  --generateKeys \
  --privateKey "/path/to/privateKey" \
```

```
  --publicKey "/path/to/publicKey" \
  --kid "key_id"
```

Copy

The JWK Set JSON will be displayed on your standard output. Alternatively, you can use the `--jwk` parameter to specify a file where it should be stored instead.

4. Store the generated public and private key files securely as they will be required to generate JWTs. **Never share your private key.**
5. Manage JWKs in Pulse.

JSON Web Tokensâ

Once you have a private key, you can generate your JWTs. All JWTs are composed of a set of headers, a set of claims and a signature.

JWT Headersâ

Besides the algorithm (`alg`), the only mandatory header for JWTs is the `kid` (the key ID), which indicates which key was used to sign the token and **must** match a valid key in the JWK Set used for validation. The set of headers should be similar to:

```
{
  "kid": "keyID",
  "alg": "RS512"
}
```

Copy

JWT Claimsâ

Typically, the following claims should be present on the JWTs:

Claim name	Extended name	Description
`iss`	Issuer name	Identifies the issuer of the JWT. This value will be provided by Feedzai, and it should match the tenant id, if applicable.
`exp`	Expiration Time	Date after which the JWT must be rejected.
`nbf`	Not before Time	Date before which the JWT must be rejected.
`jti`	JWT id	Identifier used to revoke JWTs.
`sub`	Subject	Identifies the user of the JWT.

Your final set of claims should be similar to:

```
{
    "iss": "feedzai",
    "exp": 1523022355,
    "nbf": 1523022055,
    "jti": "a00cd6dd-4e55-4694-aeb5-4050e7d063b6",
    "sub": "700e55f9-15b0-4f95-8094-960138a0d886"
}
```

Copy

How to generate a JWTâ

1. Unpack the tar.gz artifact.
2. Check generator options:

```
./bin/generate-asymmetric --help
```

Copy

3. Run the JWT Generator using the following command. Adjust the parameter values according to your needs:

```
./bin/generate-asymmetric \
  --mode JWT \
  --privateKey "/path/to/privateKey" \
  --publicKey "/path/to/publicKey" \
  --iss "issuer" \
  --sub "subject" \
  --exp "$((EPOCHSECONDS + 300))" \
  --kid "key_id"
```

Copy

The JWT will be displayed on your standard output. Alternatively, you can use the `--jwt` parameter to specify a file where it should be stored instead.

How to send the signed JWTâ□□

danger

Note that you can only send the signed JWT after the corresponding JWK is uploaded to Pulse (see Manage JWKs in Pulse).

Send the signed JWT in the `Authorization` header of HTTP requests made to Feedzai's API, as follows:

```
Authorization: Bearer <your_jwt>
```

Copy

Manage JWKs in Pulseâ□□

Once you create the JWK set, the following JWK steps must be taken:

1. Log into Pulse and confirm the current list of Authorization keys. Before revoking any key, make sure you are no longer using it.
2. Add the new key to Pulse.
3. Start signing messages with JWTs based on the new key.
4. Revoke the old key.
5. Delete revoked Authorization keys.

List Authorization keysâ□□

To list the existing Authorization keys, access the **Administration** page and select the **Authorization Keys** tab on the sidebar.

Select a label name or expand to see the Authorization keys associated with that label:

Add Authorization keysâ□□

To add a new Authorization key, select a label name and upload the JSON file that contains the new key:

info

The new key must be associated with an existing label.

tip

Make sure you start signing the new messages with the new key.

Revoke Authorization keysâ□□

The following animation illustrates how to add a new key and revoke the current one.

danger

An Authorization key should only be revoked once a new Authorization key has been added (see previous step) and JWTs are already being signed with the new key.

Delete revoked Authorization keysâ□□

By deleting an Authorization key, you are permanently removing it from the list of keys. Only revoked keys can be deleted.

API Resources

This section provides you with the information you need to work with the API.

Postman Suite

Postman collection and Environment for testing purposes.

Postman

- Download Postman Collection
- Download Postman Local Environment

On this page

Responses and Errors

When a client makes a request to an HTTP server and the server successfully receives their request, the server notifies the client if the request was successfully processed or not. Read this page to understand what each response and error means.

Successful responseâ□□

Successful calls to Feedzai's API result in a response with HTTP 200 code. Please check each endpoint's documentation to see the structure of a successful response.

Warning responseâ□□

A warning response is a subtype of a successful response which is sent with the HTTP 200 code. The transaction is also processed. The response returns:

- `status` field with the response 'warning'.
- `warnings` field with the reasons for this outcome..

Error responseâ□□

When a call to Feedzai's API results in an error, the response object returned always has the same fields. In particular, the `status` field is shared with the successful response objects of all endpoints. The **Error Response** object is defined as follows:

Name	Field Type	Description
`code`	String	An indicative code for the error. See possible values below.
`errors`	Array of strings	A list of free text explanations for the error.
`status`	String	Always with value `error`.

Generic Error codes - Possible Valuesâ□□

Error code	Description	HTTP Code
`authenticationError`	The request could not be authenticated.	`401`
`authorizationError`	The authenticated user does not have permission to access the endpoint.	`403`
`parseError`	The request could not be parsed.	`400`
`invalidParameterError`	The URL parameter provided is invalid.	`400`
`invalidTimestampError`	The timestamp provided in the request is invalid.	`400`
`nonexistentEndpoint`	The endpoint being called does not exist.	`404`
`unsupportedMethod`	The HTTP method to call this endpoint is not supported.	`405`
`tooManyRequests`	The allowed request rate was exceeded.	`429`
`internalError`	Feedzai's system produced an internal error.	`500`
`timeoutError`	Feedzai's system failed to produce a timely response.	`504`

Please check each endpoint's documentation for specific error codes.

Responses for timestamps out of rangeâ□□

This subsection describes responses for requests with timestamps that are out of the valid range.

Endpoint	Timestamp	Response HTTP code	Error or Warning	Is transaction processed?
Payment	`event_occurred_at` in the past	400	Error	No
Payment	`event_occurred_at` in the future	400	Error	No
Info, Feedback, Settlement	`event_occurred_at` in the past	200	Warning	Yes
Info, Feedback, Settlement	`event_occurred_at` in the future	400	Error	No

Error Response Example:

```
{
    "status": "error",
    "code": "invalidTimestampError",
    "errors": [
        "Event has timestamp in the future: <EVENT_OCCURRED_AT> | Now is:
'2018-09-04T16:41:30.774Z'"
    ]
}
```

Copy

Warning response example:

```
{
    "lifecycle_id": <LIFECYCLE_ID>,
    "warnings": [
        "Event has timestamp in the past: <EVENT_OCCURRED_AT> | Now is:
'2018-09-04T16:40:40.522Z'"
    ],
    "status": "warning"
}
```

Copy

Overview

This page provides you with a set of Case Manager Guides.

On this page

Overview

Welcome to the Feedzai Concepts Page. In this document you will find useful information regarding conceptual elements.

Conceptsâ□□

On this page

Base Requirements

The purpose of this guide is to walk you through the requirements needed for setting up each Feedzai component.

Furthermore, this guide also includes the requirements for installing or configuring the following components:

- Pulse
- Case Manager
- Reference Data Service
- Cassandra
- RabbitMQ
- ZooKeeper

info

The requirements below are applicable to all servers where Feedzai services will be installed, as well as other third-party services that are part of the product stack.

Operating Systemâ□□

Feedzai deployments officially support the following operating systems:

- Red Hat Enterprise Linux 9.1 (64 bit)
- CentOS 9 (64 bit)

To ensure Feedzaiâ□□s services work properly, and the Feedzai team is able to investigate or remediate any issues, the following must be ensured:

- All servers need to have `ntp` daemon configured.
- Turn-off the `swap` file (disable all swap lines on the `/etc/fstab file`);
- All servers need to have an Operating System group and user named `feedzai`.
- All the servers should be accessible via SSH for the `feedzai` user.
- All Feedzai Pulse servers must have `ulimit` set to `65535`. Example:

```
sudo tee /etc/security/limits.d/99-feedzaiproc.conf << 'EOF'
feedzai soft nofile 65535
feedzai hard nofile 65535
feedzai soft nproc 65535
feedzai hard nproc 65535
EOF
```

Copy

- The following packages must be installed:
  - `which`
  - `tar`
  - `bzip2`
  - `less`
  - `nc`
  - `unzip`
  - `vim`
  - `telnet`
  - `wget`
  - `Python3.6`

- `Python2.7`
- `libselinux-python3`

info

It is recommended that both `Python2.7` and `Python3.6` are installed because there may be dependencies that require different Python versions.

tip

It is recommended that the servers are kept up to date with the latest Operating System and security patches. Our recommendaton is that servers have access to a CentOS mirror with at least daily updates. In case virtual machines are provisioned based on immutable images, this should be ensured for the base image.

Web Browserâ□□

Feedzai products that have a Web UI (Pulse and Case Manager) officially support the following browsers:

- Chrome (version 124 or later)
- Firefox (version 125 or later)
- Edge (version 124 or later)

On this page

Installation Guide

Welcome to the Feedzai Installation Guide. In this guide you will find useful information on how to install Feedzaiâ□□s products.

Artifacts

Name	Version	Note
Installer	0.2.0	
Cross-project Feedzai RMA Solution Inventories	0.2.0	
Project Pulse Configurations	0.2.0	
Project Case Manager Configurations	0.2.0	
Project Batch Processor Configurations	0.2.0	
Project Pulse Data Science Configurations	0.2.0	
Empty RMPSP Application for Pulse Data Science Environments	0.2.0	
Empty MM Application for Pulse Data Science Environments	0.2.0	
Pulse	25.1.9	
Spark Libs	25.1.9	
Spark Dist	3.5.7-hadoop-3.3.5	
Case Manager	13.31.0	
Batch Processor	3.5.6	
Reference Data Server	3.7.12	
Keycloak Feedzai Theme	2.3.0	
JupyterLab Docker Image	25.0.51-gcs-2.2.31	**Load Docker Image as** `docker.feedzai.com/feedzai/jupyterlab:25.0.51-gcs-2.2.31-gcp`

Installing Feedzai Servicesâ□□

- Pulse Requirements
- Case Manager Requirements
- Batch Processor Requirements
- Reference Data Service Requirements

Configuring Third-Party Servicesâ□□

- Cassandra Recommendations
- RabbitMQ Recommendations
- ZooKeeper Recommendations

Feedzai Ansible Toolkitâ□□

- Installing Feedzai with Ansible

Database Setupâ□□

- Retention Scripts

How-to Guides

Welcome to the Operations How-to Guides.

In this section you will find the following set of tutorials with information on how to authenticate via the Pulse API, changing log levels at runtime and how to import/export Pulse applications:

On this page

Operations Guide

Welcome to the Feedzai Operations Guide. In this document you will find useful information on how to operate Feedzaiâ□□s products.

Monitoringâ□□

- Building a Monitoring Solution
- Monitoring Feedzai Services
- Monitoring Third-Party Services
- Monitoring the Infrastructure

Permissions Guide

Â

Functionality Tool	Action	Permission	Description
Case Manager and Pulse **Common Permissions**	Login	cm:login pulseviews:login	Lets the user log into pulse and case r
Case Manager **Events**	View	cm:resource:event:read:*	Lets the user read an event.
	Update	cm:resource:event:update:*	Lets the user update events.
	Assign/Unassign Event	cm:resource:event-user:create:* - **Backend** cm:action:event-user:create:* - **Frontend**	Lets the user assign users to event.
		cm:resource:assign:*	Gives the user rights to assign an eve already assigned to another user to its Permission only relevant for Assign an
		cm:action:event-case:delete:*	Lets the user unlink events from cases
	Notes	cm:resource:event-note:create:* - **Backend** cm:action:event-note:create:* - **Frontend**	Lets the user create notes in events.
		(1) cm:action:event-note:read:* (2) cm:resource:event-note:read:*	(1) Lets the user see Event Notes widg events details page. (2) Lets the user see notes in events.
	Move to Queue	cm:resource:event-queue:create:* - **Backend** cm:action:event-queue:create:* - **Frontend**	Lets the user move events to queues.
	Move to Close	cm:action:manual-close-alert	Lets the user manually move events to status
	Resource Manager	cm:resource:manager:*	Gives the user rights to have all action permissions even if is not assigned the Permission only relevant for Assign an
	Export/download search results	cm:resource:event-export:read:*	Lets the user export the results.
		cm:action:event-export:create:*	Lets the user see the export button.

	Status	cm:action:event-status:create:*	Lets the user set event statuses
		cm:resource:status:read:*	Lets the user read event status option
	Files	(1) cm:resource:event-file:create:* (2) cm:resource:file:create:*	(1) Lets the user create N-to-N relation between events and files. (2) Lets the user create files.
		cm:resource:file:read:*	Lets the user read files.
		cm:resource:file:update:*	Lets the user update an existing file.
		cm:resource:file:delete:*	Lets the user delete an existing file
		(1) cm:resource:file:event-file:create:* (2) cm:resource:file:event-file:update:*	Lets the user upload an existing file in Details page (Attached Files widget). Lets the user update an existing file in Details page(widget: Attached Files).
		cm:resource:file:event-file:read:*	Lets the user list all the files associate event.
	Locale	cm:resource:locale:create:*	Lets the user create locales.
		cm:resource:locale:read:*	Lets the user read locales.
		cm:resource:locale:update:*	Lets the user update locales.
		cm:resource:locale:delete:*	Lets the user delete locales.
	Users	cm:resource:user:read:*	Lets the user read available users.
Case Manager Lists	View	cm:resource:list:search:*	Lets the user search for entities in lists
		cm:section:settings-lists:read:*	Lets the user see List Management op the menu Settings section.
		cm:action:view-entity-audit:read:*	Lets the user view the "view-entity-aud action.
	Update	cm:resource:list:update:*	Lets the user update lists content and
	Create	cm:action:add-entity:create:*	Lets the user add a EntityValue to a Li
		cm:action:event-list:create:*	Lets the user add events to lists.
	Delete	cm:action:remove-entity:delete:*	Lets the user remove an EntityValue fr List.
	Posneg	cm:action:event-posneg:create:*	Lets the user add events to posneg lis
		cm:resource:event-posneg:create:*	Lets the user create posneg lists for ev
		cm:resource:event-posneg:read:*	Lets the user read posneg lists for eve

Case Manager SLAs	View SLAs	cm:resource:sla:read:* - **Backend** cm:section:settings-slas:read:* - **Frontend**	Lets the user read SLAs.
	Create SLAs	cm:resource:sla:create:* - **Backend** cm:section:settings-slas:create:* - **Frontend**	Lets the user create SLAs.
	Edit SLAs	cm:resource:sla:update:* - **Backend** cm:section:settings-slas:update:* - **Frontend**	Lets the user update SLAs.
	Delete SLAs	cm:resource:sla:delete:* - **Backend** cm:section:settings-slas:delete:* - **Frontend**	Lets the user delete SLAs.
Case Manager Cases	View Cases	cm:resource:case:read:* - **Backend** cm:section:case:read:* - **Frontend**	Lets the user read cases.

	cm:section:case-details:read:*	Lets the user see the Case Details pa
	cm:section:case-details:*	Lets the user access all case-details p and functionalities.
Search Cases	cm:section:case-search:*	Lets the user search for cases
Create Cases	cm:resource:case:create:* - **Backend** cm:section:case:create:* - **Frontend**	Lets the user create cases.
Update case	cm:resource:case:update:* - **Backend** cm:section:case:update:* - **Frontend**	Lets the user update cases.
Delete case	cm:resource:case:delete:*	Lets the user delete cases.
Assign Case	cm:resource:case-user:create:*	Lets the user assign users to cases.
Link events/alerts to cases	cm:action:event-case:create:*	Lets the user link events to case.
	cm:resource:event-case:create:*	Lets the user add events to cases.
	cm:resource:event-case:delete:*	Lets the user delete events in cases.
	cm:resource:event-case:read:*	Lets the user read events in cases.
	cm:resource:event-case:update:*	Lets the user update events in cases.
Move case to queue	cm:resource:case-queue:create:*	Lets the user create queues for cases
Set case status	cm:action:case-state:create:*	Lets the user open or close case state cases.
Upload/Download Files	(1) cm:resource:case-file:create:* (2) cm:resource:file:case-file:read:*	(1) Lets the user upload file in Case D (2) Lets the user download file in Case
Files	cm:resource:case-details:create:*	Lets user list all the files associated wi current case.
	cm:resource:file:case-file:update:*	Lets the user update file in Case Deta
Channels	cm:action:case*-global:read:*	Lets the user see channels in cases.
	cm:action:case-global:create:*	Lets the user select channels in cases Note that, for Omnichannel Multichann channel field should be hidden and for Omnichannel Single channel (Default)
Export	(1) cm:action:case-export:create:* (2) cm:resource:case-export:read:*	(1) Lets the user see the export button (2) Lets the user export the results fro cases search page, or any other table only uses those cases fields.
	(1) cm:action:case-note:create:* - **Frontend**	

	Notes	cm:resource:case-note:create:* - **Backend** (2) cm:action:case-note:read:* - **Frontend** cm:resource:case-note:read - **Backend** (3) cm:resource:case-note:update (4) cm:resource:case-note:delete	(1) Lets the user add a note to a case Case Details page. (2) Lets the user see case notes on N widget. (3) Lets the user update notes in case (4) Lets the user delete notes in cases
	States	cm:resource:case-state:create:*	Lets the user create states for cases.
	Casetype	cm:resource:casetype:<CASE_TYPE_NAME>:read (e.g cm:resource:casetype:fraud:read)	Gives read-level permission over the c given by <CASE_TYPE_NAME>, whe case type name is given by the configu case type name without the type. prefi (example: for a configured case type c type.aml, the permission is cm:resource:casetype:aml:read)
		cm:resource:casetype:<CASE_TYPE_NAME>:write (e.g cm:resource:casetype:fraud:write)	Gives both read and write-level permis over the case type given by <CASE_TYPE_NAME>, where the ca name is given by the configured case name without the type. Prefix (exampl configured case type of type.fraud, the permission is cm:resource:casetype:fraud:write).
Case Manager Reasons	View Reasons	cm:resource:status-reason:read:* - **Backend** cm:section:settings-reason:read:* - **Frontend**	Enables the user to list the available re
	Create Reasons	cm:section:settings-reason:create:*	Enables the user to create new reason
	Delete Reasons	cm:action:reason:delete:*	Enables the user to remove reasons.
Case Manager Automation Rules	View Automation Rule	(1) cm:resource:automation:read:* (2) cm:resource:configuration-automation:read: (3) cm:section:settings-automation:read:*	(1) Lets the user read automation rule (2) Lets the user read the automation configuration. (3) Lets the user see the Automation R page
	Create Automation Rule	cm:section:settings-automation:create:* - **Frontend** cm:resource:automation:create:* - **Backend**	Lets the user create automation rules Automation Rules page
	Edit Automation Rule	cm:resource:automation:update:* - **Backend** cm:section:settings-automation:update:* - **Frontend**	Lets the user update automation rules Automation Rules page
	Delete Automation Rule	cm:resource:automation:delete:* - **Backend** cm:section:settings-automation:delete:* - **Frontend**	Lets the user delete automation rules Automation Rules page
		(1) cm:resource:queue:read:* (2) cm:resource:queue-user:read:*	(1) Lets the user read queues. (2) Lets the user read its own queues.

Case Manager Queue	View Queue	(3) cm:resource:queue-user:* (4) cm:section:settings-queue:read:*	(3) Lets the user perform CRUD opera over the current queues assigned to a (4) Lets the user see the queues page Settings section.
	Create Queue	cm:resource:queue:create:* - **Backend** cm:section:settings-queue:create:* - **Frontend**	Lets the users create queues in the Se section.
	Edit Queue	cm:resource:queue:update:* - **Backend** cm:section:settings-queue:update:* - **Frontend**	Lets the user update queues in the Se section.
	Delete Queue	cm:resource:queue:delete:* - **Backend** cm:section:settings-queue:delete:* - **Frontend**	Lets the user delete queues in the Set section.
	Assign Queues	cm:resource:queue-user:assign:*	Lets the user assign queues to users.
	Import and Export Queues	cm:resource:settings:import-export:* - **Backend** cm:action:settings:import-export:* - **Frontend**	Lets the user import and export queue
	Get Next	cm:resource:user-queue:*	Lets the user see the working queues, context of the review next feature.
Case Manager Self Service Rules	View SSR	cm:section:settings-rules:read:*	Lets the user see rules in the Self-Ser Rules page.
	Create SSR	cm:section:settings-rules:create:*	Lets the user create rules in the Self-S Rules page.
	Edit SSR	cm:section:settings-rules:update:*	Lets the user update rules in the Self-S Rules page.
	Delete SSR	cm:section:settings-rules:delete:*	Lets the user delete rules in the Self-S Rules page.
	Publish SSR	cm:section:settings-rules:publish:*	Lets the user publish rules in the Self- Rules page.
	Test SSR	(1) cm:resource:settings-rules-custom-test:create:* (2) cm:resource:settings-rules-test:create:* cm:section:settings-rules-test:create:*	(1) Lets the user test rules with custom properties, either the user is a landlord contains the permission. (2) Lets the user test rules, either the u landlord or contains the permission.
		cm:section:settings-rules-test:create:*	Lets the user test rules, either the use landlord or contains the permission.
	Rules Audit	cm:resource:self-service-rules:audit:*	Lets the user access rules audit entrie
Case Manager Reports	View reports	cm:resource:report:read:* - **Backend** cm:section:report:read:* - **Frontend**	Lets the user read reports.

	Create reports	cm:resource:report:create:* - **Backend** cm:section:report:create:* - **Frontend**	Lets the user create reports.
	Update reports	cm:resource:report:update:*	Lets the user update reports.
	Delete reports	cm:resource:report:delete:* - **Backend** cm:action:delete-generated-report:delete:* - **Frontend**	Lets the user delete reports.
	Download reports	cm:resource:file:report:read:*	Lets the user download reports.
Case Manager Report Templates	View	cm:resource:report-template:read:* - **Backend** cm:section:settings-report-template:read:*- **Frontend**	Lets the user read report templates.
	Create	cm:resource:report-template:create:* - **Backend** cm:section:settings-report-template:create:*- **Frontend**	Lets the user create report templates.
	Update	cm:resource:report-template:update:* - **Backend** cm:section:settings-report-template:update:*- **Frontend**	Lets the user update report templates.
	Delete	cm:resource:report-template:delete:* - **Backend** cm:section:settings-report-template:delete:*- **Frontend**	Lets the user delete report templates.
	Download	cm:resource:file:report-template:read:*	Lets the user download report templat
Case Manager Entity Notes	View	cm:action:entity-note:read:* - **Frontend** cm:resource:entity-note:read:* - **Backend**	Allows the user to see Notes widget (e customer notes widget) on an entity de page (e.g., Customer details page).
	Create	cm:action:entity-note:create:* - **Frontend** cm:resource:entity-note:create:* - **Backend**	Allows the user to assign notes to data entities.
Case Manager Entity Data	View	(1) cm:section:customer-details:read:* - **Frontend** (2) cm:entity-data:<entity_type>:read:* - **Backend** (entity_type available right now are accounts, cards and customers)	(1) Lets the user see the customer pag (2) Lets the user read data about an e a given type (e.g. customers, cards, et places such as the customer page
Case Manager Entity Files	View	cm:resource:file:entity-file:read:*	Lets the user read files associated with (e.g. customers, cards, etc.) in places the customer page.
Case Manager Cases Sources	View	cm:resource:case-source:read:* - **Backend** cm:section:settings-case-source:read:* - **Frontend**	Lets the user have access to the case page and list case sources
	Create	cm:resource:case-source:create:* - **Backend** cm:section:settings-case-source:create:* - **Frontend**	Lets the user create new case sources
	Update		

		cm:resource:case-source:update:* - **Backend** cm:section:settings-case-source:update:* - **Frontend**	Lets the user edit (enable/disable) exis case sources.
Case Manager **Custom Extensions**	onEventStatusUpdate	cm:action:extension:onEventStatusUpdate	Lets the user execute the custom exte given by onEventStatusUpdate
Case Manager **Channel Based Permissions**	Read and Write	cm:channel:<channel_id>:read-write	Lets the users see and edit <channel_ events and artifacts (cards, digital acti transfers, inbound-payments, non-mo
	Read Only	cm:channel:<channel_id:resource:group:readel_id>:read-only	Users can see <channel_id>'s events artifacts, but cannot edit them (except notes andfile-uploading)
	None	cm:channel:<channel_id:resource:group:readel_id>:none	Users do not have access to <channe
Case Manager **Access Groups**	Search	cm:resource:group:read:*	Lets the user search groups.
Case Manager **Multi-tenancy**	Landlord	cm:resource:landlord:*	Lets the user have landlord permissio
	Tenant	cm:resource:tenant:read:*	Lets the user read the tenant list.
Case Manager **Multiple State Machines**	Change Status	cm:state-machine:event-status:write:<STATEMACHINE>	Lets the user change the status for a g state machine (whose id is <STATEMACHINE>)
	Read	cm:state-machine:event:read:<state-machine-id>	Enables the user to read the state ma information.
	List Reasons	cm:state-machine:status-reason:read:<state-machine-id>	Enables the user to list the status reas a state machine with a given id.
	Create Reasons	cm:state-machine:status-reason:write:<state-machine-id>	Enables the user to create a status rea a state machine with a given id.
Case Manager **Tokenizer**	Reveal	(1) cm:action:reveal-<field_path> (2) cm:resource:reveal-all (3) cm:resource:reveal-<FIELD> (4) cm:resource:reveal:* (5) cm:resource:search-by:<FIELD>	(1) Lets the user see the reveal icon to a field. (2) Lets the user automatically reveal encrypted fields when entering an eve details page. (3)Lets the user reveal a field value us tokenizer. (The <field_path> has to be replaced with the field path in the obje (4) Give user all permissions regarding (5) Lets the user perform a search ope by a specified field in plain text using tokenizer. This permission combined w user permission cm:resource:search-b allows users to search for all tokenized elements.
Case Manager **Details Page**	Widgets	(1) cm:action:self-service-config:advanced-mode (2) cm:resource:self-service-config	(1) Lets the user have access to the A edit mode in the UI. (2) Lets the user list active preference

Case Manager **Login and Logout**	Login and Logout	(1) security:ownership:read (2) security:permissions:read	(1) Lets the user see the ownership de Pulse user groups (toggle "advanced" (2) Lets the user read the permissions
Case Manager **Manage Message**	Manage Message	(1) cm:section:settings-message:read:* (2) cm:resource:message:read:* (3) cm:action:settings-message:create:* (4) cm:resource:message:create:* (5) cm:action:settings-message:update:* (6) cm:resource:message:update:* (7) cm:action:settings-message:delete:* (8) cm:resource:message:delete:*	(1) Lets the user see the Messages pa the Settings section. (2) Lets the user read messages. (3) Lets the user create new message Settings section. (4) Lets the user create new message (5) Lets the user update messages in Settings section. (6) Lets the user update messages. (7) Lets the user delete messages in t Settings section. (8) Lets the user delete messages.
Case Manager General Permissions	Dashboards	cm:section:dashboard-analyst-performance:* cm:section:dashboard-business-overview:* cm:section:dashboard-psd2:* cm:section:rules-monitoring-rules-filter:* cm:resource:dashboards-user-sync:update:* cm:section:rules-monitoring-transactions-filter:*	Allows the users to see the dashboard (Requires enabling Insights)
	Get Next	cm:section:review-next:read:*	Lets the user see the review next func
	UI Configuration	cm:resource:configuration:read:* cm:resource:configuration:create:* cm:resource:configuration:update:* cm:resource:configuration:delete:* cm:action:gui-config:edit cm:resource:gui-config:edit cm:resource:gui-config:reset-to-default cm:resource:gui-config:list-active:*	Lets the user read and modify the UI configuration.
	Language Configuration	cm:resource:language:read:* cm:resource:language:create:* cm:resource:language:delete:* cm:resource:language:update:*	Lets the user read and modify the ava languages.
	Time Configuration	cm:resource:timezone:read:* cm:resource:timezone:create:*	Lets the user read and modify timezon

	cm:resource:timezone:delete:* cm:resource:timezone:update:*	
Translated Messages	cm:resource:message:read:*	Lets the user read the translated mess
Event Status	cm:resource:event-status:create:*	Lets the user create status for events.
Edit Resource	cm:action:edit-resource:update:*	Lets the user edit the resource.
Custom Link	cm:action:link-to	Lets the user navigate to a custom link
Pulse App	(1) cm:resource:pulse-app:read:* (2) cm:resource:pulse-rules:read:*	(1) Gets the Pulse application status a (2) Gets the Pulse application rules
Session	cm:resource:session:read:* cm:resource:session:create:* cm:resource:session:update:* cm:resource:session:delete:*	Lets the user read and modify session information in Case Manager.
Event Visibility	cm:resource:event-visibility cm:resource:event-visibility:create:*	Gives the user full event visibility.
Customer Page	cm:section:customer:read:*	Lets the user have access to Custome details
Event Details	cm:section:event-details:*	Lets the user have access to all Event operations
	cm:section:event-details:read:* cm:section:event:*	Lets the user have access to read Eve Details
	cm:resource:event-users:read:*	Retrieves a list of users that can be us assignment according to the tenant pr
	cm:resource:event-queues:read:*	Retrieves a list of queues that are ass with an event's tenant.
	cm:section:event:read:*	Lets the user have access to read Eve Details
	cm:section:event:create:*	Lets the user have access to create E Details
	cm:section:event:delete:*	Lets the user have access to delete E Details
	cm:section:event:update:*	Lets the user have access to update E Details
Fraud Alerts	cm:section:fraud-alerts:*	Lets the user have access to all Fraud operations
	cm:section:fraud-alerts:read:*	Lets the user have access to read Fra
Rules Block	cm:resource:configuration-rules-block:read:*	Lets the user read the configuration of blocks.
Search	cm:action:search-by-schema.card_pan	Lets the user search by Card Pan
Assign	cm:resource:assign-no-steal:*	

			Allows users to assign unassigned eve themselves and to others.
	Export Entity	cm:resource:entity-export:read:* cm:action:entity-export:create:*	Lets the user export the results from a entity's search page.
	Edit Mode	cm:action:self-service-config:normal-mode	Lets the user have access to the Norm mode in the UI.
Pulse App Management	Create	application:apps:create	Permission to create new applications
	Delete	application:apps:delete	Permission to delete applications.
	Manage	application:apps:manage	Permission to start/stop/publish/export applications.
	View	application:apps:read	Permission to read applications.
	Update	application:apps:update	Permission to update applications.
	Import	application:apps:partialImport	Permission to partially import applicati
	Export	application:apps:partialExport	Permission to partially export applicati
	All CRUD Permissions	application:*	Permission to perform all CRUD actior application models.
	Read Applications	application:*:read	Permission to read applications.
Pulse Sandbox Evaluations	View	application:sandbox_evaluations:read	Permission to read sandbox evaluatior
	Create	application:sandbox_evaluations:create	Permission to create sandbox evaluati
	Update	application:sandbox_evaluations:update	Permission to update sandbox evaluat
	Delete	application:sandbox_evaluations:delete	Permission to delete sandbox evaluati
Pulse Custom Services	View	application:dependencies:read	Permission to read custom service.
	Create	application:dependencies:create	Permission to create new custom serv
	Update	application:dependencies:update	Permission to update custom service.
	Delete	application:dependencies:delete	Permission to delete custom service.
Pulse Application History	View	application:history:global:read application:history:resource:read	Permission to read history log applicat wide. Permission to read history log of a sing resource.
Pulse Job Executions	Create	application:jobexecutions:create	Permission to create new job executio
	View	application:jobexecutions:read	Permission to read job execution.
	Update	application:jobexecutions:update	Permission to update job execution.
	Delete	application:jobexecutions:delete	Permission to delete job execution.
	Start	application:jobexecutions:start	Permission to start job executions
	Abort	application:jobexecutions:abort	Permission to abort job executions
Pulse Job Profiles	View	application:job_profiles:read	Permission to read job profiles.
	Create	application:job_profiles:create	Permission to create job profiles.
	Update	application:job_profiles:update	Permission to update job profiles.
	Delete	application:job_profiles:delete	Permission to delete job profiles.
	View	application:managedlists:read	Permission to read managed lists.

Pulse Managed Lists	Create	application:managedlists:create	Permission to create managed lists.
	Update	application:managedlists:update	Permission to update managed lists.
	Delete	application:managedlists:delete	Permission to delete managed lists.
Pulse Managed List Items	View	application:managedlistitems:read	Permission to read managed list items
	Create	application:managedlistitems:create	Permission to create managed list iten
	Update	application:managedlistitems:update	Permission to update managed list ite
	Delete	application:managedlistitems:delete	Permission to delete managed list iten
	Decrypt	application:managedlistitems:decrypt	Permission to download encrypted val managed list items in plain text format plain text download needs to be enabl the pulse.global.lists.allowListDownloadIn property.
	View Revision	application:managedlistitems:revision:read	Permission to read managed list revisi
	Update Revision	application:managedlistitems:revision:update	Permission to update managed list rev
	Create Revision	application:managedlistitems:revision:create	Permission to create managed list rev
Pulse Outcome Combinators	View	application:outcome_combinators:read	Permission to read outcome combinat
	Create	application:outcome_combinators:create	Permission to create a new outcome combinator.
	Update	application:outcome_combinators:update	Permission to update outcome combi
	Delete	application:outcome_combinators:delete	Permission to delete a outcome comb
Pulse Workflows	View	application:rte_workflows:read	Permission to read workflow.
	Create	application:rte_workflows:create	Permission to create new workflow.
	Update	application:rte_workflows:update	Permission to update workflow.
	Delete	application:rte_workflows:delete	Permission to delete workflow.
Pulse User Management	View	security:users:read	Permission to read user.
	Create	security:users:create	Permission to create new user.
	Update	security:users:update	Permission to update user.
	Delete	security:users:delete	Permission to delete user.
Pulse Admin Rights	Admin	admin:view	Permission to access the administratio screens.
	Read Logs	admin:logs:read	Permission that grants access to the l
	Manage Cluster	admin:cluster:manage	Permission that grants management a the cluster.
	Read Cluster	admin:cluster:read	Permission that grants read access to cluster.
	Delete Configuration	admin:configuration:delete	Permission to delete configurations on administration screens.
	Read Configuration	admin:configuration:read	Permission to read configurations on t administration screens.
	Update Configuration	admin:configuration:update	Permission to update configurations o administration screens.

	Deploy Services	admin:deploy	Permission to deploy services through REST API.
	Reload Security	admin:security:reload	Permission to reload security through API.
	Control Services	admin:service:control	Permission to start and stop services v Administration API.
Pulse Tabs	Management Tabs	pulseviews:management:tabs:*	Permission to allow the user to access pulseviews tabs.
Pulse SSR Audit	View	(1) application:ssr_audit:privilegedView (2) application:ssr_audit:read	(1) Permission to read Self-Service Ru Audit entries in privileged views, for ins retrieving the audit respective to all Ru present in a given application. (2) Permission to read self-service rule entries.
Pulse SSR Test	View	application:ssr_test:read	Permission to read self-service rule te results.
	Run	application:ssr_test:run	Permission to run a self-service rule te
	Update	application:ssr_test:update	Permission to update self-service rule results
Pulse Security	Security	application:streams:read:*	Permission to read streams (stream corresponds to a schema)
		security:tenants:read	Permission to read tenants.
		security:audit:*	Permission to read audit logs.
		useride:management	Permission to manage users
		security:roles:read	Permission to read role
		security:logging:update	Permission to update the logging level
		cm:resource:secure-metadata	Lets the user see encrypted metadata
		cm:element:tenant:read:*	Lets the user access UI elements acce to landlords.
Pulse Datascience Datasources	View	application:datascience_datasources:read	Permission to read data science data s
	Create	application:datascience_datasources:create	Permission to create data science data sources.
	Update	application:datascience_datasources:update	Permission to update data science dat sources.
	Delete	application:datascience_datasources:delete	Permission to delete data science data sources.
Pulse Datascience Evaluation Metrics	View	application:datascience_evaluationmetrics:read	Permission to read data science evalu metrics.
	Create	application:datascience_evaluationmetrics:create	Permission to create data science eva metrics.
	Update	application:datascience_evaluationmetrics:update	Permission to update data science eva metrics.
	Delete	application:datascience_evaluationmetrics:delete	Permission to delete data science eva metrics.
	View	application:datascience_modelprojects:read	

Pulse Datascience Model Projects			Permission to read data science mode projects.
	Create	application:datascience_modelprojects:create	Permission to create data science mo projects.
	Update	application:datascience_modelprojects:update	Permission to update data science mo projects.
	Delete	application:datascience_modelprojects:delete	Permission to delete data science mo projects.
Pulse Datascience Model Projects Evaluations	View	application:datascience_modelprojects_evaluations:read	Permission to read data science mode evaluations.
	Create	application:datascience_modelprojects_evaluations:create	Permission to create data science mo evaluations.
	Update	application:datascience_modelprojects_evaluations:update	Permission to update data science mo evaluations.
	Delete	application:datascience_modelprojects_evaluations:delete	Permission to delete data science mo evaluations.
Pulse Datascience Model Projects Feature Importance	View	application:datascience_modelprojects_featureimportance:read	Permission to read feature importance
	Create	application:datascience_modelprojects_featureimportance:create	Permission to create feature importan
	Update	application:datascience_modelprojects_featureimportance:update	Permission to update feature importan
	Delete	application:datascience_modelprojects_featureimportance:delete	Permission to delete feature importan
Pulse Datascience Model Projects Hyperparameter Tuning	View	application:datascience_modelprojects_hyperparam_tuning:read	Permission to read hyperparameter tu
	Create	application:datascience_modelprojects_hyperparam_tuning:create	Permission to create hyperparameter t
	Update	application:datascience_modelprojects_hyperparam_tuning:update	Permission to update hyperparameter
	Delete	application:datascience_modelprojects_hyperparam_tuning:delete	Permission to delete hyperparameter t
Pulse Datascience Model Projects Hyperparameter Tuning Eval	View	application:datascience_modelprojects_hyperparam_tuning_eval:read	Permission to read hyperparameter tu eval.
	Create	application:datascience_modelprojects_hyperparam_tuning_eval:create	Permission to create hyperparameter t eval.
	Update	application:datascience_modelprojects_hyperparam_tuning_eval:update	Permission to update hyperparameter eval.
	Delete	application:datascience_modelprojects_hyperparam_tuning_eval:delete	Permission to delete hyperparameter t eval.
Pulse Datascience Model Projects Models	View	application:datascience_modelprojects_models:read	Permission to read Data Science mod
	Create	application:datascience_modelprojects_models:create	Permission to create Data Science mo
	Update	application:datascience_modelprojects_models:update	Permission to update Data Science mo
	Delete	application:datascience_modelprojects_models:delete	Permission to delete Data Science mo
Pulse Datascience	View	application:datascience_modelprojects_targets:read	Permission to read Data Science targe

Model Projects Targets	Create	application:datascience_modelprojects_targets:create	Permission to create Data Science tar
	Update	application:datascience_modelprojects_targets:update	Permission to update Data Science ta
	Delete	application:datascience_modelprojects_targets:delete	Permission to delete Data Science tar
Pulse Datascience Plans	View	application:datascience_plans:read	Permission to read Data Science Plan
	Create	application:datascience_plans:create	Permission to create Data Science Pla
	Update	application:datascience_plans:update	Permission to update Data Science Pl
	Delete	application:datascience_plans:delete	Permission to delete Data Science Pla
	Run	application:datascience_plans:run	Permission to run Data Science Plans
Pulse Datascience Plans Exectutions	View	application:datascience_plans_executions:read	Permission to read Data Science Plan Execution.
	Create	application:datascience_plans_executions:create	Permission to create Data Science Pla Execution.
	Update	application:datascience_plans_executions:update	Permission to update Data Science Pl Execution.
	Delete	application:datascience_plans_executions:delete	Permission to delete Data Science Pla Execution.
Pulse Datascience Rules Projects	View	application:datascience_rulesprojects:read	Permission to read data science rules
	Create	application:datascience_rulesprojects:create	Permission to create data science rule projects.
	Update	application:datascience_rulesprojects:update	Permission to update data science rul project.
	Delete	application:datascience_rulesprojects:delete	Permission to delete data science rule projects.
	Self Service Publish	application:datascience_rulesprojects:self-service-publish	Permission to publish self-service rule
Pulse Datascience Rules Projects Evaluations	View	application:datascience_rulesprojects_evaluations:read	Permission to read data science rules evaluations.
	Create	application:datascience_rulesprojects_evaluations:create	Permission to create data science rule evaluations.
	Update	application:datascience_rulesprojects_evaluations:update	Permission to update data science rul project evaluations.
	Delete	application:datascience_rulesprojects_evaluations:delete	Permission to delete data science rule evaluations.
Pulse Datascience Rules Projects Rules	View	application:datascience_rulesprojects_rules:read	Permission to read data science rules.
	Create	application:datascience_rulesprojects_rules:create application:datascience_rulesprojects_rules:create:*	Permission to create Data Science Ru
	Update	application:datascience_rulesprojects_rules:update application:datascience_rulesprojects_rules:update:*	Permission to update data science rul
	Delete	application:datascience_rulesprojects_rules:delete	Permission to delete data science rule
	View	application:datascience_rulesprojects_snapshots:read	Permission to read data science rules snapshots.
Pulse			

Datascience Rules Projects Snapshots	Create	application:datascience_rulesprojects_snapshots:create	Permission to create data science rule snapshots.
	Update	application:datascience_rulesprojects_snapshots:update	Permission to update data science rul project snapshots.
	Delete	application:datascience_rulesprojects_snapshots:delete	Permission to delete data science rule snapshots.
Pulse Datascience Rules Projects Templates	View	application:datascience_rulesprojects_templates:read	Permission to read data science rules templates.
	Create	application:datascience_rulesprojects_templates:create	Permission to create data science rule templates.
	Update	application:datascience_rulesprojects_templates:update	Permission to update data science rul project templates.
	Delete	application:datascience_rulesprojects_templates:delete	Permission to delete data science rule templates.
Pulse Datascience Schemas	View	application:datascience_schemas:read	Permission to read Data Science Sche
	Create	application:datascience_schemas:create	Permission to create Data Science Sc
	Update	application:datascience_schemas:update	Permission to update Data Science Sc
	Delete	application:datascience_schemas:delete	Permission to delete Data Science Sc
Pulse Datascience Tag Groups	View	application:datascience_tag_groups:read	Permission to read Data Science Tag
	Create	application:datascience_tag_groups:create	Permission to create Data Science Tag Groups.
	Update	application:datascience_tag_groups:update	Permission to update Data Science Ta Groups.
	Delete	application:datascience_tag_groups:delete	Permission to delete Data Science Tag Groups.
Pulse Datascience Tag Types	View	application:datascience_tag_types:read	Permission to read Data Science Tag
	Create	application:datascience_tag_types:create	Permission to create Data Science Tag
	Update	application:datascience_tag_types:update	Permission to update Data Science Ta
	Delete	application:datascience_tag_types:delete	Permission to delete Data Science Tag
Pulse Datascience White Box Explanations	View	application:datascience_white_box_explanations:read	Permissions to read White Box Explan configuration.
	Create	application:datascience_white_box_explanations:create	Permissions to create White Box Expla configuration.
	Update	application:datascience_white_box_explanations:update	Permissions to update White Box Expl configuration.
	Delete	application:datascience_white_box_explanations:delete	Permissions to delete White Box Expla configuration.

Â

On this page

Custom UDFs

`Tmx` â

`Tmx.risk(String reasonCodes)`

Receives a String `reasonCodes`, which is a concatenation of 3 character reason codes, fetches the risk score of each and sums them all, returning a transaction risk score.

`FivMarkers` â

```
/**
 * Checks whether the FIV marker is present and not expired.
 *
 * @param fivMarkerCode  The FIV marker code being looked for.
 * @param fivMarkersJson Customer FIV markers list, encoded as JSON string.
 *                       In practice this should always be the value of the same
 *                       Pulse field - 'customer_fiv_markers'
 * @return true if the FIV marker is part of the Customer FIV markers list and has not expired,
 *         false otherwise.
 */
FivMarkers.getIfNotExpired(String fivMarkerCode, String fivMarkersJson)
```

Copy

Checks whether the specified `FivMarker` exists in the customer's FIV markers JSON data and, if present, validates its expiration date if present.

Exampleâ

In Pulse Rule variables:

```
pip_fiv_marker_present = FivMarkers.getIfNotExpired("PIP", customer_fiv_markers)
```

Copy

Where "PIP" is the FIV marker code we're looking for in `customer_fiv_markers`.

note

In practice we always expect this UDF to be called with `customer_fiv_markers` as the second argument.

Implementation Notesâ

- A FIV marker is considered expired as soon as the current date reaches expiration date, meaning if its expiration date is today, it is considered expired.
- We don't expect multiple entries with the same FIV marker code in `customer_fiv_markers`, but, if we do and in order to have consistent behavior, we only consider it not expired if none of the entries have expired.
- In entries in the `customer_fiv_markers` list are malformed according to the expected schema they will be disregarded. In that case, we'll still make a best effort to consider other valid entries in the array.

On this page

Release Notes

Please find below the complete list of releases.â□□

RMA API Resources

This section provides you with the specification of the RMA API.

On this page

Overview

Multi-Tenancyâ□□

Multi-Tenancy allows Feedzai clients to host and support multiple tenants in a single environment. This allows them to provide at-scale solutions to their own clients, who can use the product on their own.

For **HSBC**, tenants will correspond to **markets**. For example, within the APAC region, there might be a `singapore` and `australia` tenants, which can be used to segregate access to these specific regions, even if they're serviced by the same application/channel in Pulse and Case Manager (outbound payments, for instance).

You can refer this link for more on Rules and Lists.

Landlordâ□□

A **landlord** is a client who has a Fraud Prevention installation that they can offer to other clients, also known **tenants**. Landlords can carry out the following actions:

- Create, deploy and customize tenant instances, along with all the configuration/data integration and mapping required.
- Write simple and advanced rules.
- Create Lists.
- Edit lists: Add/Remove items to/from lists.
- Create applications.
- Create models, workflows, dashboards and engineer features that can be deployed to one or more tenants.
- Create case management queues, administer tenant users.

For example, **HSBC Hong Kong** will have the main installation so it can be considered as landlord.

Rules and Listsâ□□

Landlords can create rules and lists that can be applied to all tenants. In addition, landlords can create customized rules and lists for each tenant, and allow tenants to access and modify these resources.

From a multi-tenancy perspective, rules and lists can be of different natures:

- Rules can be **Global** (applying to all tenants and state shared by all tenants), **Common to all tenants** (applying to all tenants, and state specific for each tenant), or **Custom per tenant** (rules customized individually).
- Lists can also be **Global** (Used by all tenants, for instance, a blacklist shared by all tenants), **Common to all tenants** (a list with the same motivation but with different items for different tenants) and **Custom per tenant** (a list that is specifically created for a tenant).

There are also other resources (namely **Plans** and **Models**) that can be defined and/or customized for tenants, with a similar procedure to that of rules and lists.

info

Refer the below links for more on Multi-Tenancy.

- Multi-Tenancy Concepts
- Tenant Management

On this page

Rules and Lists

Multi-tenancy is the ability to support multiple tenants in the same environment offering a practical solution to apply rules to several tenants at the same time, as well as to customize rules for each tenant.

When creating a rules project, the user can select from the following options:

- **Global** is used for rules that are not multi-tenanted.
- With **Duplicated for Tenant**, the same rule applies to all tenants, but rule conditions can be computed using tenant-specific data.
- With **By Tenant**, custom rules apply to individual tenants.

info

A **landlord** is a client who has a fraud prevention installation that he can offer to other clients, also known as tenants, acting as a reseller. A **tenant** is a client that uses a fraud prevention application provided by a landlord. Tenants have the possibility to create, manage and deploy their own rules and lists depending on the permissions the landlord has given to them.

Duplicated for Tenantâ□□

When a rules project is set as **Duplicated for Tenant** it means that the set of rules applies to all the tenants in the same way. In terms of deployment, a rule snapshot is created and applied in runtime, the events are then grouped by each tenant and all rules defined in the rule snapshot are applied for each tenant individually, as illustrated in the following image.

By Tenantâ□□

When a rules project is set as **By Tenant**, it means that a set of rules applies to specific tenants. In terms of deployment, a rule snapshot is created and applied in runtime, the events are then grouped by tenant and the corresponding set of rules are applied for each tenant, as illustrated in the following image.

Advanced vs. simple rulesâ□□

Users can set the type to advanced or simple, the former being the default type.

This setting has the following meaning:

- **Advanced rules projects** contain rules that can be written using a template and which may have an environment and state.
- **Simple rules projects** contain rules that are not editable inside Pulse and may be used for self-service rules created in Case Manager.

When a rules project is set as Simple, the State input is hidden, the label for the Actions becomes Secondary Actions, and the multi-tenancy options are limited to Global and By Tenant.

Self-service rulesâ□□

With **self-service rules** (SSR), fraud analysts can write and deploy rules themselves very quickly.

The characteristics of self-service rules in a nutshell are as follows:

- They are deployed directly to the Production environment, in real time.
- They are created in Case Manager by analysts and/or their managers.
- Governance can audit and support self-service rules.

This lets you implement completely new processes and achieve higher operational efficiencies while dynamically following changes in fraud patterns.

Testing self-service rulesâ□□

Self-service rules can be tested in Case Manager, allowing production environments to execute Spark jobs processing the most recent events that have gone through a self-service rule block of a given workflow. These events will be tested against an updated version of that rule block and measure Hit Rate, Money Captured, etc.

In order for Pulse to process the data in the same way as when going through the live workflow, those events are stored in a custom data store used only for self-service rule testing.

This data store thus receives every event that goes through a self-service rule block in the workflow. Depending on the testing configuration in Case Manager, this data store keeps the data of the time interval that is necessary for rule testing.

Auditing self-service rulesâ□□

Pulse can be configured to keep audit logs of the creation, edition, deletion, ordering, testing, and publishing operations made to rules. The raw logs are accessible via a REST API through a query that specifies a rule, a workflow element or an application. Get in touch with your contact person at Feedzai to get more information on the audit logs and the Self-Service Rules REST API.

info

Refer below for more.

- Multi tenancy in rules
- Configuring Multi-Tenancy

On this page

Tenant Management

Create new Tenantsâ□□

You can create Tenants that you can later associate to your users. Do so in the **Multi-Tenancy** section in Pulse:

1. Click on the **Administration** button [] in Pulse
2. Click on **Multi-Tenancy** [] on the left hand pane
3. Click on **Add** [] a new Tenant
4. Populate its **Name** and **ID** as well as any comment you may want to add
5. **Save** and you will be able to see your tenants in the **Multi-Tenancy** page:

Create users associated with tenantsâ□□

1. Click on the **Security** Icon [ ] in Pulse
2. The **Users** Section should be visible as shown below
3. Click on **Add** [] which will show an user form
4. Fill in the required fields, select the **Tenant** to which the user should be associated and **Save**
5. The user should now be available in the previous page

Multi-Tenancy in Listsâ□□

Lists in Pulse also support the multi-tenancy concept:

- **Global** lists are aimed to be used on non multi-tenanted environments or if shared across all tenants
- In **By Tenant** and **By Tenant Group** lists each Tenant (or group of tenants) will be able to see and manage a subset of all the items without impacting other Tenants

The list configuration will should under the `Tenancy` column in the List Management page:

info

Refer below for more information

- Creating new Tenants
- Multi tenancy in lists

On this page

Installing Feedzai with Ansible

The Feedzai Ansible Toolkit refers to a series of Ansible Playbooks created by Feedzai to automate the services installation, and **manage configurations** across multiple environments.

In the HSBC RMA GCP deployment, the dedicated **control-node VM** serves as the primary Ansible control node. It contains all the necessary binaries and dependencies for installing each component. For application-level Ansible tasks, the **pulse-helper** and **case-manager-helper** VMs can also be utilised as execution nodes where appropriate. Ansible connects to each target VM via SSH over GCP's internal network. For more information about the connectivity requirements check the Ansible documentation.

warning

The Ansible Playbooks provided by Feedzai can be used solely for the purpose of **configuration management**, and are not a solution for deploying Feedzai's products.

Requirementsâ□□

- Hostname resolution is provided by **GCP internal DNS**
  - Use `ping` to confirm that each server is able to reach other servers by hostname.
- The **control-node VM** is the primary Ansible executor. For application-specific tasks, the **pulse-helper** and **case-manager-helper** VMs can also act as execution nodes.
- SSH key authentication between the control-node (or helper VMs) and all target VMs is configured.
- The deployment user is `control-node`, which has `sudo` permissions. `feedzai` is the local service user (`ansible_local_user`) on each target VM.

Services Installationâ□□

To install a `<service>` (i.e.: `pulse`, `case-manager`, `reference-data`) run the following command from the **control-node VM**:

```
python3 ansible-playbook \
  -i solution-default \
  -i project-default \
  -i gcp \
  -i project-hsbc \
  -i locations/rma/<env> \
  --limit=<service> \
  <playbook>.yml \
```

Copy

Where `<playbook>` depends on the component being installed:

Component	Playbook
Cassandra, RabbitMQ, Zookeeper	`thirdparty.yml`
Pulse, Case Manager, Reference Data	`<service>.yml`

note

Replace the `<env>` with the appropriate environment path for each environment. Example: for the `dev` environment, use `locations/rma/dev`

Services Configurationâ

To configure a `<service>` for a given environment, run the same command with the environment-specific inventory applied last. Later inventories take precedence over earlier ones, allowing environment-specific variables to override defaults.

```
python3 ansible-playbook \
  -i solution-default \
  -i project-default \
  -i gcp \
  -i project-hsbc \
  -i locations/rma/<env> \
  --limit=<service> \
  <playbook>.yml \
```

Copy

note

The inventory paths shown above (e.g. `locations/rma/dev`) are sample reference paths used to validate deployability. HSBC production deployments may use their own environment and location naming conventions.

Notesâ

The `ansible_user` must exist on the target machine prior to execution. In the HSBC RMA GCP deployment, `ansible_user` is `control-node` and `ansible_local_user` is `feedzai`. This applies to all playbook executions.

In the above command-line invocation of `ansible-playbook`, each corresponding inventory must exist and contain a `variables.yml` file. If the same variable is defined in multiple inventories, the later ones appearing in the command-line invocation take precedence over the earlier ones. In this way, by adding the most specific variables at the end (for example, environment-specific variables), they can overwrite any values inherited from earlier inventories such as the defaults (e.g. `solution-default`, `project-default`).

Inventory	Purpose	Directory
`solution-default`	Cross-project Feedzai defaults	`solution-default/group_vars/<service>/variables.yml`
`project-default`	Project-level defaults	`project-default/group_vars/<service>/variables.yml`
`project-ds`	Project-level Pulse Data Science environment specifics	`project-ds/group_vars/<service>/variables.yml`
`gcp`	GCP platform-specific overrides	`gcp/group_vars/<service>/variables.yml`
`locations/rma/<env>`	RMA environment-specific variables	`locations/rma/ansible/<env>/group_vars/<service>/variables.yml`

More inventories may be added as needed, in the same manner.

Feedzai Ansible Toolkit

Welcome to the Feedzai Ansible Toolkit Guide.

In this section you will find information about the Feedzai Ansible Toolkit:

- Installing Feedzai with Ansible

Batch Processor Installation Guide

Welcome to the Batch Processor Installation Guide.

In this section you will find a set of pages with information about the Batch Processor installation process:

On this page

Configuration Recommendations

Ansibleâ□□

info

Available ansible variables for running the ansible playbook.

Variable	Default Value
ansible_user	feedzai
ansible_local_user	{{ ansible_user }}
batch_processor_install_path	"/opt/feedzai"
batch_processor_home_path	"{{ batch_processor_install_path }}/batch-processor-{{ batch_processor_version }}"
batch_processor_port	5567
batch_processor_jvmopts	""
database_batch_processor_engine	"com.feedzai.commons.sql.abstraction.engine.impl.PostgreSqlEngine"
database_batch_processor_jdbc	"jdbc:postgresql://{{ database_batch_processor_hostname }}:{{ database_batch_processor_port }}/{{ database
database_batch_processor_jdbc_opts	[cancelSignalTimeout=2, connectTimeout=2, loadBalanceHosts=true, loginTimeout=10, socketTimeout=60, tc

Variable	Default Value
database_batch_processor_username	"batchprocessor"
database_batch_processor_password	"secret"
database_batch_processor_schema	"batchprocessor"
database_batch_processor_hostname	"postgres"
database_batch_processor_port	"5432"
database_batch_processor_database_name	"feedzai"
database_batch_processor_schemapolicy	none
batch_processor_merc_static_tenants	["feedzai"]
batch_processor_merc_retry_cron	"0 0 ? *"
batch_processor_merc_retry_max_tries	3

Variable	Default Value
batch_processor_merc_cron	"0 30 2 * ? "
batch_processor_rds_config_host	reference-data
batch_processor_rds_config_port	8080
batch_processor_rds_default_jwt	"Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvYW8gU2FudG9zIiv
batch_processor_rds_tenant_keys	[{ key: feedzai, value: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvYW8gU2FudG9zIiv
batch_processor_pulse_jwt_private_key	Default value hidden due to length
batch_processor_pulse_jwt_algorithm	"RS512"

Variable	Default Value
batch_processor_pulse_jwt_kid	"feedzai-internal"
batch_processor_pulse_jwt_issuer	"feedzai"
pulse_batchprocessor_username	"batchprocessor"
pulse_batchprocessor_password	"Secret1!"
pulse_scheme	"http"
pulse_port	8080
pulse_elb	"pulse"
pulse_host	"{{ pulse_elb }}"
pulse_url	"{{ pulse_scheme }}://{{ pulse_elb }}:{{ pulse_port }}"

note

(*) More details on the supported algorithms can be found at JWSAlgorithm. Only RS256, RS384 and RS512 are supported.

Quartz Cron Schedulingâ

tip

For more details on Quartz Cron Scheduling expressions, refer to the official Quartz CronTrigger Tutorial.

Field	Mandatory	Allowed Values	Description
Seconds	X	0-59	The cron expression for `second`
Minutes	X	0-59	The cron expression for `minute`
Hours	X	0-23	The cron expression for `hour`
Day of Month	X	1-31	The cron expression for `day of month` (**)
Month	X	1-12 or JAN-DEC	The cron expression for `month`
Day of Week	X	1-7 (1=SUN, 7=SAT) or SUN-SAT	The cron expression for `day of week` (**)
Year		blank, 1970-2099	The cron expression for `year`

note

(**) In Quartz, you must use the `?` character in either the Day of Month or Day of Week field to indicate "no specific value". You cannot specify both fields simultaneously, as they conflict!

On this page

Network and Connectivity

info

This page is based on the architecture of Feedzai standard solutions. Specific clients or custom solutions may have additional requirements, so this is not a definitive guide.

Firewall Rulesâ□□

Inboundâ□□

note

Inbound â□□other services connect to this service.

Service	Port	Description
HTTP	TCP 5567	Batch Processor Service HTTP
SSH	TCP 22	SSH access (if available)

Outboundâ□□

note

Outbound â□□this service connects to external services.

Service	Port	Description
RBDMS (PostgreSQL)	TCP 5432	Connection to the PostgreSQL database
Pulse	8080	Communication with Pulse (via HTTP)
Pulse	8443	Communication with Pulse (via HTTPS)
Reference Data Service	8080-8081	Communication with Reference Data Service

Load Balancersâ□□

For high available environments Load Balancers must be set as follow:

Service	Description	Algorithm	Healthcheck URL	Response Codes	Policy Example
Batch Processor	Entry to access the Batch Processor service.	Least connections (or round robin)	`HTTP:5567/healthchecks`	200 - Node is available	Checks should be performed every 30 seconds with a timeout of 5 seconds. Nodes should be removed after the second failure.

On this page

Requirements

Service Requirementsâ□□

This service requires the following software to run.

Java Environmentâ□□

OpenJDK SE Development Kit version 1.8, with the following vendors (with their latest hotfix versions):

- Azul Zulu rpm zip tar.gz
- Azul Platform Prime (formerly Zing)
- Amazon Corretto
- Oracle

info

Product tests are done with the **Azul Zulu JVM**.

The following versions are not supported, and are known to have problems:

- 1.8.0u242
- 1.8.0u121
- 1.8.0u60
- 1.8.0u45 and earlier

Non-listed releases are either not supported or haven't been put to extensive testing.

Setup Requirementsâ□□

RDBMSâ□□

Batch Processor uses PulseDB for database access, meaning that Batch Processor can be deployed on top of:

- Oracle 19.3 (or higher)
- PostgreSQL 14.20 (or higher) - driver version 42.7.10 should be used.
- SQL Server 2017 GA (or higher)
- H2 1.4.200 (or higher) - only for test/evaluation purposes.

info

Feedzai validates the product primarily against H2 and PostgresSQL.

caution

Before selecting a database engine, make sure you understand how different database systems are supported by our PulseDB.

Reference Data Serviceâ□□

Batch Processor retrieves data from the Reference Data Service (RDS), and therefore requires an accessible RDS server to execute jobs. Read more about how to install and configure RDS in the installation guide.

Pulseâ□

Batch Processor sends data to Pulse, and therefore requires an accessible Pulse cluster to start. Read more about how to install and configure Pulse in the installation guide.

Case Manager Installation Guide

Welcome to the Case Manager Installation Guide.

In this section you will find a set of pages with information about the Case Manager installation process:

On this page

Configuration Recommendations

tip

For a complete reference check the official Configuration Guide.

Ansibleâ□□

info

Available ansible variables for running the ansible playbook.

Variable	Default Value
ansible_user	feedzai
ansible_local_user	{{ ansible_user }}
cm_install_path	"/opt/feedzai"
cm_home_path	"{{ cm_install_path }}/product-cm-{{ cm_version }}"
cm_java_options	"-Xms4g -Xmx4g"
database_casemanager_engine	"com.feedzai.commons.sql.abstraction.engine.impl.PostgreSqlEngine"
database_casemanager_jdbc	"jdbc:postgresql://{{ database_casemanager_hostname }}:{{ database_casemanager_port }}/{{ databa
database_casemanager_jdbc_opts	[currentSchema={{ database_casemanager_schema }}, loginTimeout=10, connectTimeout=2, cancel loadBalanceHosts=true, reWriteBatchedInserts=true]
database_casemanager_username	casemanager

Variable	Default Value
database_casemanager_password	secret
database_casemanager_schema	casemanager
database_casemanager_hostname	"postgres"
database_casemanager_port	"5432"
database_casemanager_database_name	"feedzai"
database_casemanager_schemapolicy	"none"
database_casemanager_pool_size	"24"
database_casemanager_pool_lazy	false
database_casemanager_reconnect_on_lost	true

Variable	Default Value
database_casemanager_max_number_retries	"30"
database_casemanager_retry_interval	"5000"
database_casemanager_ro_jdbc	"jdbc:postgresql://{{ database_casemanager_ro_hostname }}:{{ database_casemanager_ro_port }}/{{
database_casemanager_jdbc_ro_opts	[currentSchema={{ database_casemanager_schema }}, loginTimeout=10, connectTimeout=2, cancel database_casemanager_jdbc_ro_target_server_type }}, loadBalanceHosts=true]
database_casemanager_jdbc_ro_target_server_type	master
database_casemanager_ro_username	{{ database_casemanager_username }}
database_casemanager_ro_password	{{ database_casemanager_password }}
database_casemanager_ro_schema	{{ database_casemanager_schema }}
database_casemanager_ro_hostname	{{ database_casemanager_hostname }}
database_casemanager_ro_port	{{ database_casemanager_port }}

Variable	Default Value
database_casemanager_ro_database_name	{{ database_casemanager_database_name }}
database_casemanager_ro_schemapolicy	{{ database_casemanager_schemapolicy }}
database_casemanager_ro_pool_size	{{ database_casemanager_pool_size }}
database_casemanager_ro_pool_lazy	{{ database_casemanager_pool_lazy }}
database_casemanager_ro_reconnect_on_lost	{{ database_casemanager_reconnect_on_lost }}
database_casemanager_ro_max_number_retries	{{ database_casemanager_max_number_retries }}
database_casemanager_ro_retry_interval	{{ database_casemanager_retry_interval }}

Variable	Default Value
cm_url	"http://localhost:8090"
reference_data_servers	{{ groups['reference-data']
reference_data_port	"{{ reference_data_app_port }}"
reference_data_url	"http://{{reference_data_servers}}:{{reference_data_port}}""
reference_data_casemanager_jwt_token	"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvYW8gU2F
cm_jetty_http_port	8080
pulse_casemanager_username	casemanager
pulse_casemanager_password	Secret1!
rabbitmq_casemanager_brokers	{{ rabbitmq_brokers }}
rabbitmq_casemanager_username	cm
rabbitmq_casemanager_password	secret
rabbitmq_casemanager_vhost	cm
rabbitmq_casemanager_use_quorum_queues	false

Variable	Default Value
jks_store_password	
cm_truststore_path	
pulse_internal_url	{{ pulse_url }}
pulse_url	"{{ pulse_scheme }}://{{ pulse_elb }}:{{ pulse_port }}"
pulse_scheme	"http"
pulse_elb	"pulse"
pulse_port	8080
cm_secret_value	ed178d2d1b6e5d036b0aab4402212704
cm_properties_to_encrypt	[database_casemanager_password, database_casemanager_ro_password, pulse_casemanager_pa tokenizer_casemanager_jwt_token]
casemanager_api_url	{{ cm_url }}

Variable	Default Value
casemanager_api_username	{{ pulse_casemanager_username}}
casemanager_api_password	{{ pulse_casemanager_password }}
cm_liquibase_rollback	false
cm_multi_tenancy_enabled	true
cm_loggers	{}
cm_probes_log_level	"OFF"

Variable	Default Value
cm_probes_output_file_name	"cm-probes.log"

Performance configurationâ□□

With the inclusion of new use cases it is expected that Case Manager sizing to be updated. Aside from bare metal sizing updates, Case Manager also provides performance configurations that allow fine tuning of the various nodes.

warning

Changing any of these configurations can have a performance impact in Case Manager. They should only be changed in agreement with the project team.

Please find below a description of each of the available configurations.

Variable	Default Value	Description
cm_performance_event_worker_threads	8	The number of workers that should be used to asynchronously store events into event storage.
cm_performance_queue_prefetch_size	200	The number of messages Case Manager fetches from RabbitMQ.
cm_performance_batch_storage_size	300	The size of the batch, i.e. the number of entries the batch can hold after which a flush will occur.
cm_performance_batch_storage_timeout	1000	The batch timeout, i.e. the time after which a batch flush will occur, regardless of the amount of entries present in it.
cm_performance_batch_flush_workers	4	The number of workers that should be used to asynchronously execute batch flush callback operations.
cm_performance_batch_flush_queue_size	5000	The size of the buffer that holds events to be added to the batch.
cm_performance_event_storage_queue_size	200	The size of the buffer that holds events to be added to the batch.
cm_performance_event_storage_batch_size	500	

Variable	Default Value	Description
		The size of the batch, i.e. the number of entries the batch can hold after which a flush will occur.
cm_performance_event_storage_batch_timeout	1000	The batch timeout, i.e. the time after which a batch flush will occur, regardless of the amount of entries present in it.
cm_performance_event_storage_batch_threads	1	The batch threads, i.e. the number of threads getting entries of the queue and writing to the batch.
cm_performance_related_max_last_flush_secs	10	Maximum period (in seconds) that notifier should wait before sending the RTS notification.
cm_performance_max_queued_explanations	5000	The Number of queued explanations for persistence by the explanations batch.
cm_performance_max_related_transactions	500	The maximum number of related transactions to an event to fetch.
cm_performance_explanations_queue_size	5000	The size of the buffer that holds events to be added to the batch.
cm_performance_explanations_batch_size	300	The size of the batch, i.e. the number of entries the batch can hold after which a flush will occur.
cm_performance_explanations_batch_timeout	1000	The batch timeout, i.e. the time after which a batch flush will occur, regardless of the amount of entries present in it.
cm_performance_explanations_batch_threads	1	The batch threads, i.e. the number of threads getting entries of the queue and writing to the batch.
cm_performance_automation_worker_threads	4	The batch threads, i.e. the number of threads getting entries of the queue and writing to the batch.
cm_performance_automation_batches_queue_size	200	The size of the buffer that holds events to be added to the batch.
cm_performance_automation_batches_batch_size	500	The size of the batch, i.e. the number of entries the batch can hold after which a flush will occur.
cm_performance_automation_batches_batch_timeout	1000	The batch timeout, i.e. the time after which a batch flush will occur, regardless of the amount of entries present in it.
cm_performance_automation_batches_batch_threads	1	The batch threads, i.e. the number of threads getting entries of the queue and writing to the batch.
cm_performance_automation_rule_cache	0	Automation cache value until automation rules are refreshed in memory.
cm_performance_max_queued_automations	5000	Maximum number of queued automations for execution by the automation worker threads
cm_performance_audit_batches_queue_size	200	The size of the buffer that holds events to be added to the batch.
cm_performance_audit_batches_batch_size	300	The size of the batch, i.e. the number of entries the batch can hold after which a flush will occur.
cm_performance_audit_batches_batch_timeout	1000	The batch timeout, i.e. the time after which a batch flush will occur, regardless of the amount of entries present in it.
cm_performance_audit_batches_batch_threads	1	The batch threads, i.e. the number of threads getting entries of the queue and writing to the batch.
cm_performance_ignore_event_updates	false	Whether event updates should be ignored, and thus, the event ingestion process optimized.

Variable	Default Value	Description
cm_performance_disable_auditing_without_users	false	Whether the audit for actions without users should be skipped in the database.
cm_performance_enable_db_heuristic_for_event_queue_search	false	Whether an explicit `queueId neq null` should be added to improve queue searches.

Quartz Cron Schedulingâ□□

tip

For more details on Quartz Cron Scheduling expressions, refer to the official Quartz CronTrigger Tutorial.

Field	Mandatory	Allowed Values	Description
Seconds	X	0-59	The cron expression for `second`
Minutes	X	0-59	The cron expression for `minute`
Hours	X	0-23	The cron expression for `hour`
Day of Month	X	1-31	The cron expression for `day of month` (**)
Month	X	1-12 or JAN-DEC	The cron expression for `month`
Day of Week	X	1-7 (1=SUN, 7=SAT) or SUN-SAT	The cron expression for `day of week` (**)
Year		blank, 1970-2099	The cron expression for `year`

note

(**) In Quartz, you must use the `?` character in either the Day of Month or Day of Week field to indicate "no specific value". You cannot specify both fields simultaneously, as they conflict!

On this page

Multi-tenancy Configuration

This page describes the configuration required in order to enable Multi-tenancy in Case Manager.

Enabling Multi-tenancyâ

Multi-tenancy is enabled by default in RMA, and can be toggled using the `cm_multi_tenancy_enabled` variable. Tenancy exists for the following features:

- Event segregation
- List management
- Queues management

Tenant fieldâ

The tenant field to which Case Manager looks to define the tenant will always be the same `schema.tenant` in Case Manager since its value is based on a field configured in Pulse.

Check Pulse Configuration Reference for more details.

Permissionsâ

For users to be able to use multi-tenancy features in Case Manager, their role should have the `cm:resource:landlord:*` permission.

This permission reduces tenancy configuration, thus not requiring system administrators to manually add each tenant to a landlord user.

info

Refer below for more

- Multi tenancy configuration

On this page

Network and Connectivity

info

This page is based on the architecture of Feedzai standard solutions. Specific clients or custom solutions may have additional requirements, so this is not a definitive guide.

Firewall Rulesâ□□

tip

For a complete list of the the Firewall Configurations required for Case Manager, please refer to the product documentation.

Inboundâ□□

note

Inbound â□□other services connect to this service.

Service	Port	Description
HTTP	TCP 8080	Case Manager UI HTTP
HTTPS	TCP 8443	Case Manager UI HTTPS
SSH	TCP 22	SSH access (if available)

Outboundâ□□

note

Outbound â□□this service connects to external services.

Service	Port	Description
RBDMS (PostgreSQL)	TCP 5432	
RabbitMQ	TCP 5761-5762	
Elasticsearch	TCP 9200	
Pulse	8080	Communication with Pulse (via HTTP)
Pulse	8443	Communication with Pulse (via HTTPS)
Reference Data Service	8080-8081	Communication with Reference Data Service

Load Balancersâ□□

For high available environments Load Balancers must be set as follow:

Service	Description	Algorithm	Healthcheck URL	Response Codes	Policy Example
Case Manager	Entry to access the web UI.	Least connections (or round robin)	`HTTPS:8443/api/configuration/latest` `HTTP:8080/api/configuration/latest`	200 - Node is available	Checks should be performed every 30 seconds with a timeout of 5 seconds. Nodes should be removed after the second failure.

On this page

Requirements

tip

For a complete list of the Case Manager requirements, please refer to the official product documentation.

Service Requirementsâ□□

This service requires the following software to run.

Java Environmentâ□□

OpenJDK SE Development Kit version 1.8, with the following vendors (with their latest hotfix versions):

- Azul Zulu rpm zip tar.gz
- Azul Platform Prime (formerly Zing)
- Amazon Corretto
- Oracle

info

Product tests are done with the **Azul Zulu JVM**.

The following versions are not supported, and are known to have problems:

- 1.8.0u242
- 1.8.0u121
- 1.8.0u60
- 1.8.0u45 and earlier

Non-listed releases are either not supported or haven't been put to extensive testing.

Setup Requirementsâ□□

RDBMSâ□□

Case Manager uses PulseDB for database access, meaning that Case Manager can be deployed on top of:

- Oracle 19.3 (or higher)
- PostgreSQL 14.20 (or higher) - driver version 42.7.10 should be used.
- SQL Server 2017 GA (or higher)
- H2 1.4.200 (or higher) - only for test/evaluation purposes.

info

Feedzai validates the product primarily against H2 and PostgresSQL.

caution

Before selecting a database engine, make sure you understand how different database systems are supported by our PulseDB.

RabbitMQâ

RabbitMQ is the messaging platform Case Manager uses to ingest events from Pulse, and also to propagate events to other downstream services.

The following RabbitMQ and Erlang versions are supported:

- RabbitMQ - 3.11.13 with Erlang - 25.3.2.21+
- RabbitMQ - 3.9.8 with Erlang - 24.0+

Before starting Pulse you must configure the messaging cluster:

1. Provision a RabbitMQ cluster and ensure that Pulse is able to establish a connection to that any node of the cluster.
2. Create a new vhost for Case Manager.

```
rabbitmqctl rabbitmqctl add_vhost casemanager
```

Copy

1. Create a new user for Case Manager.

```
rabbitmqctl add_user casemanager <password>
```

Copy

1. Update the vhost policies to grant all permissions to the Case Manager user.

```
# sets write and read permissions on every resource
rabbitmqctl set_permissions casemanager -p casemanager ".*" ".*" ".*"
```

Copy

Pulseâ

Case Manager retrieves data from Pulse, and therefore requires an accessible Pulse cluster to start. Read more about how to install and configure Pulse in the installation guide.

On this page

Rolling Upgrades

The recommended approach for zero downtime deployments in Case Manager is a technique known as rolling upgrades.

Rolling upgrades leverage Case Manager's maintenance mode in the following way:

- The existing cluster is put into maintenance mode which will stop all RabbitMQ message consumption, e.g. new events.
- The existing Case Manager cluster (or a new one) upgrade is performed
- When the upgrade is finished maintenance mode is stopped allowing the cluster to start processing messages again

Please note that this technique should only be used when an existing Case Manager cluster exists.

Requirementsâ□□

- Maintenance mode is enabled via Case Manager API. This requires a user with the following permissions:
  - `*` : includes the permission to login to Case Manager
  - `cm:resource:maintenance-start-stop:admin` : permission to enable/disable maintenance mode
- A dedicated machine to run database patches with access to Case Manager API. This can be either direct access to a Case Manager instance or access to a Load Balancer

---

Rolling upgradesâ□□

warning

This approach is currently under discussion and may not represent the final solution

To prevent database locking issues when running in high available environments the orchestrator (control-node) should be responsible for running the database changelogs.

The pipeline is defined as follows:

1. Maintenance mode is enabled in all Case Manager instances
2. A machine is created to apply patches
3. Case Manager images are created with the latest Feedzai release
4. Case Manager instances are configured but do not apply patches
5. After validating the new cluster, load balancers are updated to redirect traffic to the new cluster
    1. If validation fails a rollback command can be issued to bring the database back to its previous state
6. Maintenance mode is disabled allowing the new cluster to start processing events

This solution follows a similar approach to Blue-Green deployments where SSH to Case Manager machines is not available.

To support maintenance mode we have created two new playbooks, `case-manager-enable-maintenance-mode.yml` and `case-manager-disable-maintenance-mode.yml`. The required configuration parameters can be found in Case Manager Configuration with the description tag `Maintenance mode only`.

These playbooks use Case Manager Ansible variables and hosts inventory and all playbook tasks run only once even if the hosts file contains more than one Case Manager instance. This is due to the fact that we only require a request to a single Case Manager instance to enable/disable maintenance since the maintenance status will be synchronized across all instances.

Step 1 consists in running the following Ansible task that will enable Case Manager maintenance mode.

```
python3.6 ansible-playbook \
    -i project-default \
    -i project-hsbc \
    -i gcp \
    -i project-<env> \
    case-manager-enable-maintenance-mode.yml
```

Copy

In step 2 only tags that are responsible for extracting and setting up the required components to run Liquibase patches should be included. This step can be run in the same machines as step 1 in which case only tags `case-manager.750` and `case-manager.755` are required.

```
python3.6 ansible-playbook \
    -i project-default \
    -i project-hsbc \
    -i gcp \
    -i project-<env> \
    case-manager.yml \
    --tags case-manager.050,case-manager.060,case-manager.110,case-manager.120,case-manager.215,case-manager.220,case-manager.410,case-manager.415,case-manager.425,case-manager.440,case-manager.710,case-manager.730,case-manager.750,case-manager.755
```

Copy

In step 3 all tags should be included except tags that are responsible applying patches which were executed in the previous step.

```
python3.6 ansible-playbook \
    -i project-default \
    -i project-hsbc \
    -i gcp \
    -i project-<env> \
    case-manager.yml \
    --tags case-manager.configure \
    --skip-tags case-manager.750,case-manager.755
```

Copy

In case any issues are detected during validation, the database can be reverted to its previous state by rollbacking the database changes.

For this cases, there is a need to override the variable `cm_liquibase_rollback` to `true` and then run the proper Ansible task

```
python3.6 ansible-playbook \
    -i project-default \
    -i project-hsbc \
    -i gcp \
    -i project-<env> \
    case-manager.yml \
    --tags case-manager.rollback
```

Copy

After validation of the new cluster/installation is done, maintenance mode can be disabled (step 6) by running the corresponding tag.

```
python3.6 ansible-playbook \
    -i project-default \
    -i project-hsbc \
    -i gcp \
```

```
    -i project-<env> \
    case-manager-disable-maintenance-mode.yml
```

Copy

On this page

Data Science Environment validation tool Configuration

- This Tool allows to automatically validate a Data Science environment. It creates the most common data science resources and interact with them to verify their expected behaviours (e.g. execution of Spark Jobs, gathering of enrichment data, db's connection). Regardless of the final state of the validation task, this tool must be able to clean up the environment, to leave it in a similar state to that before its execution

Validation Tasksâ□□

Order	Task	Description
1	data-set generation	Creates random data-sets in a temporary directory, according to the number of file-systems supported by the environment
2	data-set import	Copies the data-sets created in the previous task to the different file-systems supported by the environment
3	data source creation	Creates data sources on Pulse with the data-sets copied in the previous task. Then it executes data source analysis and validates their results
4	plan creation	Creates plans on Pulse with the data sources created in the previous task. Then it executes plan executions and validates their results
5	individual model creation	Creates two individual models, Feedzai (Random Forest) and H2O (Generalized Linear Modeling) using cross validation. Then it executes another model evaluation with feature importance and validates their results
6	hyperparameter tuning creation	Creates a Hyperparameter Tuning with a Feedzai (Random Forest) and H2O (XGBoost) and validates that they execute successfully in the cluster.
7	rule creation	Creates two/three rules and publishes the application. Then it creates a snapshot evaluation and validates its result
8	sandbox creation	Creates a sandbox evaluation with the models and snapshot created in the previous tasks and validates its result. By default it used the default outcome combinator but it is possible to configure it.
9	automl creation	Creates an AutoML and validates that it finds a best model successfully.
10	jupyterlab integration	Checks whether JupyterLab is configured. If so, then tries to start it via Pulse.
11	scheduled metrics job creation	Creates a Scheduled Metrics Job (SMJ) on Pulse using the resources created in the previous tasks. Then it executes a job execution and validates their results

Ansible variablesâ□□

Variable	Type	Default Value	Description	task
tool_datascience_plan_enabled	boolean	true	Whether this validation should be skipped or not.	plan creation
tool_datascience_plan_deployable	boolean	true	Whether this validation should be skipped or not.	plan creation
tool_datascience_model_enabled	boolean	true	Whether this validation should be skipped or not.	individual model creation
tool_datascience_hyperparameter_enabled	boolean	true	Whether this validation should be skipped or not.	hyperparameter tuning creation

Variable	Type	Default Value	Description	task
tool_datascience_rule_enabled	boolean	true	Whether this validation should be skipped or not.	rule creation
tool_datascience_sandbox_enabled	boolean	true	Whether this validation should be skipped or not.	sandbox creation
tool_datascience_jupyterlab_enabled	boolean	false	Whether this validation should be skipped or not.	jupyterlab integration
tool_datascience_automl_enabled	boolean	false	Whether this validation should be skipped or not.	automl creation
tool_rte_scheduledmetricsjob_enabled	boolean	true	Whether this validation should be skipped or not.	scheduled metrics job creation
tool_cleanup_resources_enable	boolean	true	Whether this validation should be skipped or not.	
tool_failure_cleanup_enable	boolean	true	Whether this validation should be skipped or not.	

As an example, Data Science Environment validation tool tool default configuration includes the following execution task:

```
- name: "[900] Run Data science environment validation tool"
  become: yes
  become_user: "{{ pulse_validation_tool_unix_user }}"
  shell: "{{ run_command }}"
  when: enable_run_tool
```

Copy

Database Setup

Welcome to the Database Setup Guide.

In this section you will find information about the Retention scripts:

- Retention scripts

On this page

Retention Scripts

To enhance scalability, minimize contention, and optimize performance, it is advisable to partition the Pulse and Case Manager databases. Additionally, in accordance with Feedzai system requirements, data should not be retained for more than 6 months. All scripts enforcing the data retention period are developed and included in the installation bundle.

List of Retention Scriptsâ□□

- create_drop_old_partition_function.sql
- create_drop_old_partitions_function.sql
- drop_7_month_old_partitions.sql

How to execute the deletion scriptâ□□

The deletion script `drop_old_partitions` only deletes one partition per execution. This means that in order to delete multiple partitions the script has to be run multiple times with a different value set in the parameter it takes.

The script should be run as follows:

```
SELECT drop_old_partitions(MONTHS);
```

Copy

Where `MONTHS` indicates how far back the script will calculate the partition to delete, based on the formula `current month - MONTHS`.

As an example and assuming the current month is June, to delete the previous month's partition (May) `MONTHS` would be set to `1`:

```
SELECT drop_old_partitions(1);
```

Copy

Configuration Recommendations

On this page

Network and Connectivity

info

This page is based on the architecture of Feedzai standard solutions. Specific clients or custom solutions may have additional requirements, so this is not a definitive guide.

Firewall Rulesâ□□

Inboundâ□□

note

Inbound â□□other services connect to this service.

Service	Port	Description
HTTP	TCP 8080	Case Manager UI HTTP
Keycloak	TCP 7600	Intra-cluster communication
SSH	TCP 22	SSH access (if available)

Outboundâ□□

note

Outbound â□□this service connects to external services.

Service	Port	Description
RBDMS (PostgreSQL)	TCP 5432	
Keycloak	UDP 1812	Keycloak plugin for Radius server (OIDC)
Pulse	8080	Communication with Pulse via HTTP
Elasticsearch	9300	Communication with Elasticsearch via HTTP
Case Manager	8080	Communication with Case Manager via HTTP

Requirements

tip

For a complete list of Keycloak requirements, please refer to the official product documentation.

On this page

Azul Platform Prime

Azul Platform Prime is a highly optimized JVM and elastic runtime that breaks traditional Java scale barriers and provides applications with improvement in Java responsiveness, scale, and throughput.

Why Azul Platform Prime?â□□

Pulse (real-time) is a memory intensive application due the way the real-time metrics are stored / calculated. Metrics are kept in memory and for each transaction the metrics for the entities involved (eg. Customer) are updated (according to the windows definitions transactions need to be expired, windows updated and metrics calculated) - and this can be a memory and GC intensive task depending on the number of transactions, metrics, size of the windows, etc).

Azul prime optimizes these kinds of workloads where low-latency / high-throughput with high memory usage is needed mainly due to its pauseless garbage collector.

info

Feedzai recommends the usage of Azul Prime (in Pulse Real Time machines) for the most demanding use cases (in terms of volumes). In those cases Azul Prime is required to accomplish the latencies and the throughput required.

Azul Platform Prime Installationâ□□

info

Azul Platform Prime is only required for Pulse RT nodes.

Azul Platform Prime can be downloaded through the Azul website

Pulse supports the last stable build compatible with the JDK8 (at the time of this document was written 22.08.301.0).

Alternatively Feedzai is also providing the packages in nexus:

- zing rpm zip tar.gz
- zing-zst rpm zip tar.gz

note

As the links to download the previous binaries from the Azul website are behind an account required link, Feedzai is providing the installation packages in nexus - since it was previously approved and configured as the standard way to provide Feedzai software to HSBC.

Due security guardrails, HSBC was having problems downloading the `RPM` packages, it was requested if it is possible to provide the packages in `zip` and `tar.gz` formats. This is the reason why there are multiple links for the Azul installation files.

To install the software please follow the steps on this page.

Validate the installationâ□□

Execute `java -version` or `/opt/zing/zing-<jdk-version>/bin/java -version` (if you don't have the java command on path):

The output should be similar to the one below:

```
java version "1.8.0-zing_19.07.0.0"
Zing Runtime Environment for Java Applications (build 1.8.0-zing_19.07.0.0-b3)
Zing 64-Bit Tiered VM (build 1.8.0-zing_19.07.0.0-b4-product-linux-X86_64, mixed mode)
```

Copy

License installationâ□□

The license key is not necessary to run Azul Platform Prime. However, to comply with your internal documents or processes regarding contract compliance, it may be required to install an Azul license key.

To install the licence key on a machine, copy the license file to your target hostâ□□s `/etc/zing/` directory. To use any other file location, you must add `-XX:ZingLicenseFile=<filename>` to your Azul Zulu Prime JVM instance launch command. Licenses allow Azul Zulu Prime JVM instance launches up to the end of the day they are due to expire within the local timezone specified in the license file.

To force Azul Prime (version 20.10 and newer) to verify the license, add the following arguments: `-XX:+UseCommercialFeatures -XX:+ValidateLicenseKey`.

Some applications depend on the *JAVA_HOME* environment variable being set. So it is recommended to define the *JAVA_HOME* environment variable to point to the Azul Platform Prime installation directory, e.g. `$ export JAVA_HOME=/opt/zing/zing-<jdk-version>`.

Configurationsâ□□

The Azul Zulu Prime JVM recognizes the standard and non-standard command line options for the HotSpot VM that is a core component of Java Platform, Standard Edition (Java SE). If the Azul Zulu Prime JVM detects a HotSpot VM native command line option that it does not support, its response can be set with the following option:

```
-Xnativevmflags:[ignore|error|warn]
[source, shell,opts="user,check"]
```

Copy

The default option is ignore.

This is important because it allows us to test the Azul Prime installation without changing the already defined JVM parameters.

tip

One initial test to check if it is everything ok can be simply change the JVM and run everything else as usual.

Azul Platform Prime Memory allocationâ□□

Azul Platform Prime uses the Azul Zulu Prime JVM to run Java applications, similar to a typical JVM running a Java application. The significant difference is how memory is handled. Azul Zulu Prime JVM uses the Azul Zulu Prime System Tools (ZST) for managing the memory of three components: Contingency memory, Pause Prevention memory, and Zing Memory, the memory used for the Azul Zulu Prime JVMâ□□s Garbage Collector.

Using Azul Zulu Prime JVM with the Garbage Collector and Contingency memory means that your Java application is no longer limited to the maximum heap size specified on the Java command line when you started your Java application. As an additional resource, Azul Zulu Prime JVMâ□□s Garbage Collector might temporarily use Pause Prevention memory to prevent pauses that are caused by lack of available clean pages.

Azul Platform Prime provides two different methods to configure/instruct ZVM to use the system memory:

- **Reserve-at-config**: memory for use by the JVMs running on the machine can be reserved at configuration
- **Reserve-at-launch**: the memory reservation for each JVM can be deferred until the launch of that JVM process

For more details about ZVM memory allocation please refer to Azul zing documentation.

Feedzai recommends reserved-at-config. On a typical production installation, around 80 % of the total system memory is reserved for zing, leaving 20 % for the OS/Kernel and/or off-heap allocation. Out of 80%, 75 % is set for Reservable Memory where all the applications JVMs will run, and the remaining ~4% are divided equally between Pause Prevention Memory and Contingency Memory.

For more information about how to set up the memory please check this guide.

Monitoringâ□□

Pulse already reports information about the JVM status by showing for instance the heap usage, garbage collector periodicity and execution time, and number of threads via the exposed monitoring endpoints.

The same metrics can be used to monitor the system running on top of Azul Platform Prime.

For more information please check the Pulse section in the monitoring guide.

Additionally Azul also provides an official runtime monitoring and diagnostics tool, for more information please check this page.

On this page

Blue-Green deployments

The recommended approach for zero downtime deployments is a technique known as blue-green deployment.

Blue-green deployment uses two almost identical clusters in the following way:

- At the start, one of the clusters is in use. Let's call it the blue environment.
- The other one is on standby, created from a new release. Let's call this one the green environment.
- When the green cluster is deployed, traffic is redirect to this cluster and deployment becomes the stand-by environment. It is available in case of a necessary rollback. We recommended this step to be manual so that every validation is done prior to changing traffic to the new cluster.

warning

Prior to a Blue-Green deployment RabbitMQ should have colored vhosts set up (`pulseblue` and `pulsegreen`)

Using a control-node as Blue-Green deployment orchestratorâ

When SSH connection is available between the control-node and service machines (Pulse, RabbitMQ, etc) it can orchestrate the entire deployment, excluding only validation and switching load balanceres tasks.

The first step is to federate the necessary queues, which can be done as follows:

```
python3 ansible-playbook \
  -i solution-default \
  -i project-default \
  -i gcp \
  -i project-hsbc \
  -i locations/rma/<env> \
  rabbitmq-setup-blue-green-federation.yml \
```

Copy

Note: the above playbook should only run on a single RabbitMQ node.

Pulse machines are installed using the current production opposite color and by copying its metadata. This can be achieve by using Ansible variables `deployment_color=<opposite_color>` and `keep_metadata_between_deployments=true` :

```
python3 ansible-playbook \
  -i solution-default \
  -i project-default \
  -i gcp \
  -i project-hsbc \
  -i locations/rma/<env> \
  pulse.yml \
  -e deployment_color=<opposite_color> \
  -e keep_metadata_between_deployments=true \
```

Copy

After Pulse installation ends and the new cluster is validated, load balancers can be changed to point to the new cluster.

Federate queues can also be disabled by running the following playbook:

```
python3 ansible-playbook \
  -i solution-default \
  -i project-default \
  -i gcp \
  -i project-hsbc \
  -i locations/rma/<env> \
  rabbitmq-disable-blue-green-federation.yml \
```

Copy

Note: the above playbook should only run on a single RabbitMQ node.

Temporary solution for joint sessionâ□□

warning

This approach is currently under discussion and may not represent the final solution

When SSH connection between the orchestrator (control-node) and Pulse images is not available, the deployment can be achieved by offloading the steps that are prone to race conditions to a different machine(s).

The pipeline is defined as follows:

1. Deployment color is set to the current production environment opposite color
2. Pulse images are created with the latest Feedzai release
3. RabbitMQ queues replication (federation) is set up to allow new cluster to receive updates from the production cluster. Refer to RabbitMQ federation for details
4. A machine is created to clean up the existing metadata for colored cluster and to copy the metadata from the production cluster
5. Pulse machines are configured but ignore metadata and patches set up
6. A machine is create (can be the same as step 4) to apply patches and publish the application
7. After validating the new cluster, load balancers are updated to redirect traffic to the new cluster
8. RabbitMQ replication queues are removed
9. Old cluster may be kept for a period of time to be later removed. This ensures production cluster can be swapped to the previous stable cluster in case anything is wrong

Comparing to Using a control-node as a Blue-Green deployment orchestrator, steps 4 to 6 split up existing Ansible roles while running from different machines.

The machine used in *Steps 3 and 8* to create and remove replication queues should have `rabbitmqctl` installed. This can be done by running the following playbook:

```
python3 ansible-playbook \
  -i solution-default \
  -i project-default \
  -i gcp \
  -i project-hsbc \
  -i locations/rma/<env> \
  thirdparty.yml \
  --tags=rabbitmq.install,rabbitmq.440 \
```

Copy

Note: this machine should have TCP connection to RabbitMQ instances on ports `4369` and `25672`.

After the machine is provisioned replication queues can be created/destroyed by running `rabbitmq-setup-blue-green-federation.yml` and `rabbitmq-disable-blue-green-federation.yml` respectively.

```
python3 ansible-playbook \
  -i solution-default \
  -i project-default \
  -i gcp \
  -i project-hsbc \
```

```
  -i locations/rma/<env> \
  rabbitmq-setup-blue-green-federation.yml \ # or rabbitmq-disable-blue-green-federation.yml to
destroy replication queues in Step 8
```

Copy

In *Step 4* only tags responsible for extracting and setting up the required components to run metadata clean up and copy current application metadata should be included. It is also required to set `keep_metadata_between_deployments=true` to enable metadata copy from existing cluster to the new cluster.

```
python3 ansible-playbook \
  -i solution-default \
  -i project-default \
  -i gcp \
  -i project-hsbc \
  -i locations/rma/<env> \
  pulse.yml \
  -e deployment_color=<opposite_color> \
  -e keep_metadata_between_deployments=true \
  --tags pulse.050,pulse.060,pulse.110,pulse.120,pulse.210,pulse.220,pulse.415,pulse.425,pulse.
440,pulse.710,pulse.730,pulse.731,pulse.750 \
```

Copy

In *Step 5* all tags should be included except tags that are responsible for metadata clean up, application patches and publishing. The first were already executed in the previous step and the latter will be executed afterwards.

```
python3 ansible-playbook \
  -i solution-default \
  -i project-default \
  -i gcp \
  -i project-hsbc \
  -i locations/rma/<env> \
  pulse.yml \
  -e deployment_color=<opposite_color> \
  --tags=pulse.configure \
  --skip-tags pulse.730,pulse.731,pulse.750,pulse.932,pulse.940,pulse.941 \
```

Copy

In *Step 6* included tags depend whether it is running in the same machine as *Step 4* or in a new machine. In case it is running in the same machine the playbook can run including only tags `pulse.940` and `pulse.941`.

```
python3 ansible-playbook \
  -i solution-default \
  -i project-default \
  -i gcp \
  -i project-hsbc \
  -i locations/rma/<env> \
  pulse.yml \
  -e deployment_color=<opposite_color> \
  --tags pulse.940,pulse.941 \
```

Copy

When running in a new machine tags that set up the machine to run patches also need to be included. These are the same as in *Step 4* excluding tags `pulse.730`, `pulse.731` and `pulse.750`.

```
python3 ansible-playbook \
  -i solution-default \
  -i project-default \
  -i gcp \
  -i project-hsbc \
  -i locations/rma/<env> \
  pulse.yml \
```

```
  -e deployment_color=<opposite_color> \
  --tags pulse.050,pulse.060,pulse.110,pulse.120,pulse.210,pulse.220,pulse.415,pulse.425,pulse.
440,pulse.710,pulse.940,pulse.941 \
```

Copy

warning

Pulse Data Science deployments **must** include the `project-ds` inventory after passing the `solution-default` and `p roject-default` inventories.

Additionally, Pulse Data Science deployments **must** include `pulse.225` (to setup the JupyterLab directory) and exclude `pulse.940` and `pulse.941` tasks (as patches and publishing are not required).

On this page

Configuration Recommendations

tip

For the full Pulse configuration refer to the official Configuration Reference and the recommended Production Setup.

Setting up the JVM DNS cache TTLâ

A couple of security properties should be set up. This can be achieved by:

1. Making sure that the `security.overridePropertiesFile` property is set to true in the `$JAVA_HOME/jre/lib/security/java.security` file.
2. Creating a new file (e.g. `/opt/feedzai/pulse/conf/java.security.properties) with the following content`:

```
# conf/java.security.properties
networkaddress.cache.ttl=1
networkaddress.cache.negative.ttl=3
```

Copy

1. Using the previously created file to set up the security properties on Pulse by adding `-Djava.security.properties=conf/java.security.properties` to the `JVM_OPTS` variable on `/opt/feedzai/pulse/bin/startup.sh`:

```
# /opt/feedzai/pulse/bin/startup.sh snippet
JVM_OPTS="(...) -Djava.security.properties=conf/java.security.properties"
```

Copy

1. Using the previously created file to set up the security properties on App engines by adding `-Djava.security.properties=conf/java.security.properties` to the `pulse.global.appengine.jvmopts` property on Pulse properties.

```
# /opt/feedzai/pulse/conf/pulse.properties snippet
pulse.global.appengine.jvmopts=(...) -Djava.security.properties=conf/java.security.properties
```

Copy

warning

Setting up these properties affects the cache behavior of all network address name resolutions for both Pulse server and App engine processes.

From the `java.security` file, next to the `networkaddress.cache.ttl` property on, for example, Zulu JDK:

> "NOTE: setting this to anything other than the default value can have serious security implications. Do not set it unless you are sure you are not exposed to DNS spoofing attack."

Ansibleâ

info

Available ansible variables for running the ansible playbook.

Variable	Default Value	Desc
ansible_user	feedzai	User execu
ansible_local_user	{{ ansible_user }}	User owne unpa asser the s `le_u`
pulse_install_path	"/opt/feedzai"	Pulse
pulse_home_path	"{{ pulse_install_path }}/product-pulse-{{ pulse_version }}"	Pulse
pulse_license_src_path	"{{ basesoftware_path }}/pulse-licenses/dev"	Sourc licens
pulse_spark_dist_archive_src_path	"{{ basesoftware_path }}/{{ spark_dist_archive }}"	Sourc Sparl
pulse_sso_idp_cert		Pulse certifi
pulse_sso_private_key		Pulse key
pulse_sso_public_key		Pulse
pulse_sso_force_https	false	Force HTTP behin requi is ena
pulse_api_url	"{{ jwk_service_host }}"	URL API, i `sche` `<hos` `>` (e mand multi availa point Balan
cm_sso_private_key		Case privat
cm_sso_public_key		Case publi
reference_data_servers	reference-data	Refer serve
reference_data_port	"{{ reference_data_app_port }}"	Refer serve corre port e using `data`
zookeeper_nodes	zookeeper	Zook
cassandra_nodes	"{{ groups['cassandra']	join(',
messaging_brokers	"rabbitmq:5672"	Mess
keycloak_url	"http://{{ keycloak_hostname }}:{{ keycloak_port }}"	Keyc
keycloak_hostname	"{{ groups['keycloak'][0] }}"	Keyc

Variable	Default Value	Desc
keycloak_port	8080	Keyc
cassandra_pulse_keyspace_replication	"{ 'class'\: 'NetworkTopologyStrategy', 'dc1'\: 1 }"	Cass strate
cassandra_cluster_name	"{{ inventory }} cluster"	Cass name
cassandra_pulse_keyspace	"profiles"	Cass store
cassandra_pulse_username	"pulse"	Pulse Cass
cassandra_pulse_password	"secret"	Pulse in Ca
database_pulse_eventstorage_hostname	"postgres"	Defau Pulse host
database_pulse_eventstorage_port	"5432"	Defau Pulse port
database_pulse_eventstorage_database_name	"feedzai"	Defau Pulse Datal
database_pulse_metadata_hostname	"postgres"	Postg Meta
database_pulse_metadata_port	"5432"	Postg Meta
database_pulse_metadata_database_name	"feedzai"	Postg Meta name
database_pulse_eventstorage_read_only_hostname	"{{ database_pulse_eventstorage_hostname }}"	Defau Pulse Read
database_pulse_eventstorage_read_only_port	"{{ database_pulse_eventstorage_port }}"	Postg Stora port
database_pulse_eventstorage_read_only_database_name	"{{ database_pulse_eventstorage_port }}"	Postg Stora Datal
database_pulse_metadata_jdbc	"jdbc:postgresql://{{ database_pulse_metadata_hostname }}: {{ database_pulse_metadata_port }}/ {{ database_pulse_metadata_database_name }}? {{ database_pulse_metadata_jdbc_opts	join('
database_pulse_eventstorage_jdbc	"jdbc:postgresql://{{ database_pulse_eventstorage_hostname }}: {{ database_pulse_eventstorage_port }}/ {{database_pulse_eventstorage_database_name }}? {{ database_pulse_eventstorage_jdbc_opts	join('
database_pulse_eventstorage_jdbc_read_only	"jdbc:postgresql://{{ database_pulse_eventstorage_read_only_hostname }}: {{ database_pulse_eventstorage_read_only_port }}/ {{database_pulse_eventstorage_read_only_database_name }}? {{ database_pulse_eventstorage_jdbc_opts_read_only	join('
database_pulse_eventstorage_jdbc_opts	[currentSchema={{ database_pulse_eventstorage_schema }}, cancelSignalTimeout=2, connectTimeout=2, loadBalanceHosts=true,	Postg Stora

Variable	Default Value	Desc
	loginTimeout=10, socketTimeout=60, tcpKeepAlive=true, targetServerType=master, reWriteBatchedInserts=true]	Datal optio
database_pulse_eventstorage_jdbc_opts_read_only	[currentSchema={{ database_pulse_eventstorage_schema }}, cancelSignalTimeout=2, connectTimeout=2, loadBalanceHosts=true, loginTimeout=10, socketTimeout=60, tcpKeepAlive=true, targetServerType={{ database_pulse_eventstorage_target_server_type_read_only }}]	Postg Stora Datal optio
database_pulse_eventstorage_target_server_type_read_only	master	Postg Stora Datal targe
jwk_service_db_jdbc	jdbc:postgresql://postgres:5432/feedzai?currentSchema=jwkservice	JDBC servi
jwk_service_db_schema	jwkservice	Sche servi
jwk_service_db_username	jwkservice	User servi
jwk_service_db_password	secret	Pass servi
jwk_service_host	"http://localhost:8080"	JWK Wher more this v point node balan
rabbitmq_brokers	"{{ messaging_brokers }}"	Rabb broke
rabbitmq_queue_type	"QUORUM"	Sets queu value and C
rabbitmq_pulse_username	"pulse"	Rabb usern
rabbitmq_pulse_password	"secret"	Rabb passw
rabbitmq_pulse_vhost	"pulse{{ deployment_color }}"	Rabb vhost
rabbitmq_pulse_queue_type	"QUORUM"	Rabb Queu varia rabbi varia
rabbitmq_feedback_storage_service_vhost	"feedback-storage"	Rabb Stora
rabbitmq_casemanager_brokers	"{{ rabbitmq_brokers }}"	Rabb broke used Stora send Mana
rabbitmq_casemanager_username	"cm"	Rabb usern cluste

Variable	Default Value	Desc
		Alert to se Case
rabbitmq_casemanager_password	"secret"	Rabb passw cluste Alert to se Case
rabbitmq_casemanager_vhost	"cm"	Rabb vhost used Stora send Mana
database_pulse_metadata_username	"pulse"	Pulse usern
database_pulse_metadata_password	"secret"	Pulse passw
database_pulse_metadata_schema	"pulse"	Pulse schen
database_pulse_eventstorage_username	"eventstorage"	Pulse usern
database_pulse_eventstorage_password	"secret"	Pulse passw
database_pulse_eventstorage_schema	"eventstorage"	Pulse schen
pulse_casemanager_username	casemanager	Case usern
pulse_casemanager_password	Secret1!	Case passw
pulse_batchprocessor_username	batchprocessor	Batch usern
pulse_batchprocessor_password	Secret1!	Batch passw
pull_hadoop_files_from_bucket	true	Pull c confi GCS
hadoop_files_download_dir	"/tmp"	Temp desti for th file
gcp_bucket_name	"pulse-dataproc-config"	The r bucke
gcp_hadoop_conf_file	"hadoop-files.tar.gz"	Hado archi
model_training_directory	"hdfs:///user/pulse/models"	The c where shoul This a loca (HDF

Variable	Default Value	Desc
model_storage_directory	"hdfs:///user/pulse/models"	The d where mode Colla be sa direct local (HDF
model_hpt_directory	"hdfs:///user/pulse/hpt-models"	The d where hyper tunin be sa mode evalu direct local (HDF
number_threads_workflow_recovery	1	Numb used recov
pulse_spark_eventlog_enabled	false	Enab loggi appli
pulse_spark_eventlog_dir	"hdfs:///var/log/spark/apps"	Spec direct Spark be st
pulse_secret_value	6edf6aaf214a3b94b4536d1f5388d7ca	Value sensi

pulse_properties_to_encrypt	[rabbitmq_pulse_password, rabbitmq_casemanager_password, database_pulse_metadata_password, database_pulse_eventstorage_password, jwk_service_db_password, cassandra_pulse_password, pulse_sso_private_key, cm_sso_private_key, pulse_user_password, dummy_token, jwk_service_client_password, reference_data_pulse_jwt_token, tokenizer_pulse_jwt_token]	List o
		encry
		prope
		used
		suffix
		to the
		prope
		`PROP`
		`encr`
		Does
		dictio

pulse_loggers	{}	Contr logge in Pu dictio logge value for th
spark_datascience_jdk_home		Sets `ME` f jobs Datap datas If om Java used.
spark_distributed_jdk_home		

Variable	Default Value	Desc
		Sets `ME` f execu Datap sched Jobs. defau will b

(*) The configuration files should be stored on a bucket in tar.gz format. The list of files inside the tar.gz should be:

- core-site.xml
- hdfs-site.xml
- mapred-site.xml
- yarn-site.xml

The service account used to fetch the configuration files from the bucket should have read permissions for that bucket (`storage.buckets.get` and `storage.buckets.list`).

Quartz Cron Schedulingâ□□

tip

For more details on Quartz Cron Scheduling expressions, refer to the official Quartz CronTrigger Tutorial.

Field	Mandatory	Allowed Values	Description
Seconds	X	0-59	The cron expression for `second`
Minutes	X	0-59	The cron expression for `minute`
Hours	X	0-23	The cron expression for `hour`
Day of Month	X	1-31	The cron expression for `day of month` (**)
Month	X	1-12 or JAN-DEC	The cron expression for `month`
Day of Week	X	1-7 (1=SUN, 7=SAT) or SUN-SAT	The cron expression for `day of week` (**)
Year		blank, 1970-2099	The cron expression for `year`

note

(**) In Quartz, you must use the `?` character in either the Day of Month or Day of Week field to indicate "no specific value". You cannot specify both fields simultaneously, as they conflict!

On this page

Configuration Recommendations for Data Science Environment

tip

For the full Pulse configuration refer to the official Configuration Reference, the recommended Production Setup and Data Science Setup.

Ansibleâ□□

info

Available ansible variables for running the Data Science ansible playbook. These variables are an addition to the variables described in Pulse Configuration Recommendations.

Variable	Default Value	Description
jupyterlab_docker_image	docker.feedzai.com/feedzai/jupyterlab:{{ jupyterlab_version }}	Feedzai JupyterLab Docker Image
hdfs_feedzai_user	pulse	User that will be the owner of the folders created on Hadoop master nodes
hadoop_master_node	"hadoop_master"	Hadoop master node hostname
hadoop_conf_path	"/etc/hadoop/conf"	Hadoop configuration folder in the master nodes
spark_libs_install_path	"{{ pulse_install_path }}/product-pulse-{{ pulse_version }}"	Source path to Spark libs on the Pulse node
pulse_jupyterlab_setup_docker	true	Flag to enable setup of JupyterLab
model_training_directory	"hdfs:///user/pulse/models"	The directory location where trained models should be saved. This directory can be a local or distributed (HDFS) filesystem.
model_storage_directory	"hdfs:///user/pulse/models"	The directory location where imported models through App Collaboration should be saved. This directory can be a local or distributed (HDFS) filesystem
model_hpt_directory	"hdfs:///user/pulse/hpt-models"	The directory location where the hyperparameter tuning models should be saved between model training and evaluation. This directory can be a local or distributed (HDFS) filesystem.
pulse_jupyterlab_container_max_memory	8589934592 (8GB)	The maximum memory (in KB) allowed for JupyterLab containers. Other units can be used instead, e.g. `512m` would set maximum memory to 512 MB.
pulse_loggers	{}	Controls extra loggers to be enabled in Pulse. Keys in this dictionary are the loggers and the values the log level for that logger.

On this page

Google Dataproc 2.2

With Google Dataproc 2.0 reaching end of life it becomes crucial to support the latest Dataproc releases. The limitation with Dataproc version (2.2) is that it comes with Java 11 by default and jobs triggered in Pulse require Java 8.

Dataproc uses the Adoptium Temurin JDK which offers Java 8 packages for both types of OS distributions (Rocky/Debian), so in this guide Java 8 and Java 11 refer to packages offered by Adoptium.

In this page you can find a description on how to install Java 8 in Dataproc 2.2 and also on how to configure Pulse to use the correct version.

note

Java 8 should be available in all cluster instances (master(s) and workers).

Set upâ□□

As stated above Dataproc 2.2 comes with Java 11 by default. However Dataproc clusters allow other packages to be installed during the cluster creation by using two different approaches: **custom images** or **initialization scripts**.

Below you can find details on how to install such packages using both approaches.

Custom Imagesâ□□

By using this approach we can generate a custom image (either fresh or based on another one) that includes all the packages we require as follows:

- Create a customization script that installs the necessary packages:

```
#! /usr/bin/bash
# Install other packages if needed

# Install Java 8
yum install temurin-8-jdk
```

Copy

- Save the file locally
- Generate a new image, for example, using the following Python script provided by Google, passing the local path to the script in `--customization-script`:

```
python3 generate_custom_image.py \
    --image-name=CUSTOM_IMAGE_NAME \
    [--family=CUSTOM_IMAGE_FAMILY_NAME] \
    --dataproc-version=IMAGE_VERSION \
    --customization-script=LOCAL_PATH \
    --zone=ZONE \
    --gcs-bucket=gs://BUCKET_NAME
```

Copy

warning

The installation script should only install the Java 8 JDK and not set `JAVA_HOME`, which should be set to the default Java version shipped with Dataproc 2.2.

Please find a completed description of the above steps in Google's Create a Dataproc custom image guide.

Initialization scriptsâ

This approach leverages the initialization action that is available in Dataproc clusters to run a script when the cluster instances are created as follows:

- Create a script that installs the necessary JDK version (or reuse the one in Custom Images)
- Upload the script to a GCS Bucket accessible by the cluster
- Set the script path while creating the cluster. For example, if the cluster is created with the `gcloud` client:

```
gcloud dataproc clusters create CLUSTER_NAME \
    --region=${REGION} \
    --initialization-actions=gs://my-bucket/cloud-sql-proxy/v1.0/cloud-sql-proxy.sh \
    ...other flags...
```

Copy

For more information please refer to Initialization Actions.

Pulse configurationâ

After a Dataproc cluster is created with the necessary Java dependencies, a new Pulse cluster will also need to be created. It should use the new Dataproc cluster and include two new Ansible variables that set the `JAVA_HOME` when creating jobs in the cluster:

- `spark_datascience_jdk_home`: sets the `JAVA_HOME` for Data Science jobs, e.g. datasource analysis
- `spark_distributed_jdk_home`: sets the `JAVA_HOME` for other jobs, e.g. scheduled Metrics Jobs

Each of these variables should point to the Java 8 path in the Dataproc cluster which is by default installed in `/usr/lib/jvm/temurin-8-jdk`.

After the Pulse cluster is created you will notice two new properties in the Pulse UI:

```
pulse.global.services.modules.distributed.jobs.spark.forward.spark.executorEnv.JAVA_HOME
pulse.global.services.modules.datascience.spark.forward.spark.executorEnv.JAVA_HOME
```

Copy

If correctly configured, when a new job is created the Spark configuration for that job will include the `spark.executorEnv.JAVA_HOME` in its execution context and force the job to run with the Java 8 JDK.

note

In Pulse Data Science deployments these properties might need to be set manually via the UI. No restart is needed since both properties are hot reloadable.

On this page

GCP Resources Creation

GCP Resourcesâ□□

INFO

This section is reserved for future documentation on GCP resources that may need to be created manually as part of the Pulse installation process.

Currently, **no GCP resources require manual creation**. All necessary infrastructure shold be provisioned automatically by deployment scripts.

This page will be updated when manual steps are introduced.

How to persist JupyterLab notebooks

warning

This page is only applicable to Pulse Data Science installations

This is Feedzai's proposed solution to persist JupyterLab notebooks when VM boot disks are deleted upon VM recreation.

A new JupyterLab docker container spawns for each user that accesses it, allowing management of notebooks per-user. Those Docker containers have a volume bound to the host - Pulse DS.

When Pulse DS bootable disk is persistent, user notebooks are kept on VM recreation. In cases where the host bootable disk is not persistent the notebooks will be lost on machine recreation. To mitigate this problem Pulse DS instances can be configured to have two different disks:

- A non-persistent bootable disk. Pulse can be installed / configured in this disk since those steps should be achievable by the original image.
- A persistent disk that is attached to Pulse VMs during VM creation

The persistent disk mount target should be set to the directory where Pulse DS stores user notebooks (`/opt/feedzai/pulse/jupyterlab`). This ensures that, when a docker container is spawn, Jupyter Lab working directory for that specific user will be mapped to the persistent disk and will load its notebooks, even if Pulse DS VM was recreated.

To create and mount non-boot disks on Linux VMs follow the steps in Add a persistent disk to your VM. After mounting the persistent disk, permissions on mount target should be set to the following:

- owner should be user `feedzai`
- group should be group `1450` which is the group id mapping to group `feedzai-jupyterlab` in the docker containers

  ð□□□**Note:** Google recommends the non-boot to be formatted as `ext4` . Doing so will create a `lost+found` file which breaks JupyterLab when Pulse tries to create the docker containers.

  To prevent this from happening permissions for this file should be change by running the following command:

  ```
  sudo chmod 6776 /opt/feedzai/pulse/jupyterLab/lost+found/
  ```

After everything is set up, validate installation by opening JupyterLab from Pulse DS UI and create/edit notebooks.

> ð□□□**Note:** If `Permission denied` pop up appears to the user when creating or saving a notebook, the error is likely due to permission changes during the persistent disk mount.
>
> If SSH to the machine is enabled one can verify this by using `ls -la /opt/feedzai/pulse/jupyterLab` . The results should have every file and directory (aside from the top directory) with owner `feedzai` and group `feedzai-jupyterlab` .
>
> To set proper permissions run the following command:
>
> ```
> sudo chown feedzai:1450 -R /opt/feedzai/pulse/jupyterLab
> ```

On this page

Network and Connectivity

info

This page is based on the architecture of Feedzai standard solutions. Specific clients or custom solutions may have additional requirements, so this is not a definitive guide.

Firewall Rulesâ□□

tip

For a complete list of the the Firewall Configurations required for Pulse, please refer to the product documentation.

Inboundâ□□

note

Inbound â□□other services connect to this service.

Service	Port	Description
HTTP	TCP 8080	Pulse Server UI (HTTP)
HTTP	TCP 8443	Pulse Server UI (HTTPS)
HTTP	TCP 8081-8091	API HTTP Server (Feedzai Connectors)
YARN (Spark, HDFS)	TCP *	Job management
Pulse	TCP *	Communication between Pulse nodes (RMI)
SSH	TCP 22	SSH access (if available)

Outboundâ□□

note

Outbound â□□this service connects to external services.

Service	Port	Description
RDBMS (PostgreSQL)	TCP 5432	Access to database
RabbitMQ	TCP 5671-5672	RabbitMQ AMQP (5671 for TLS)
YARN (Spark, HDFS)	TCP *	Job submission and management
Pulse	TCP *	Communication between Pulse nodes (RMI)
Reference Data Service	TCP 8080-8081	Communication with Reference Data Service
Profile Store (Cassandra)	9042	Fetch data from Profile Store
ZooKeeper	2181	Fetch data from Profile Store
Feedzai Enrichment Server (Public Network)	443	Connection to Feedzai Enrichment Service for specific Feedzai IP

Load Balancersâ

For high available environments Load Balancers must be set as follow:

Service	Description	Algorithm	Healthcheck URL	Response Codes	Policy Example
Pulse	Entry point used by end users to access the web UI and by Case Manager Processor to interact remotely with Pulse.	Follow the leader	`HTTPS:8443/api/client/cluster/pulse/leader/global` `HTTP:8080/api/client/cluster/pulse/leader/global`	200 - Node is global leader. 503 - Node is not global leader	Checks should be performed every 30 seconds with a timeout of 5 seconds. Nodes should be removed after the second failure.
Pulse Application API	Entry point used to send events for Pulse to score them.	Round robin	`HTTP:8081/api/health`	200 - Pulse received event and AppEngine workflow processed it successfully 500 - Some error occurred processing the event in the AppEngine workflow 503 - Input adapter is not available to receive the event	Checks should be performed every 6 seconds with a timeout of 5 seconds. Nodes should be removed after the second failure.

Pulse Installation Guide

Welcome to the Pulse Installation Guide.

In this section you will find a set of pages with information about the Pulse installation process:

On this page

Requirements

info

This guide is just a quick reference of the requirements to install Pulse. For a comprehensive guide, please refer to the official documentation.

Service Requirementsâ□□

This service requires the following software to run.

Java Environmentâ□□

OpenJDK SE Development Kit version 1.8, with the following vendors (with their latest hotfix versions):

- Azul Zulu rpm zip tar.gz
- Azul Platform Prime (formerly Zing)
- Amazon Corretto
- Oracle

info

Product tests are done with the **Azul Zulu JVM**.

The following versions are not supported, and are known to have problems:

- 1.8.0u242
- 1.8.0u121
- 1.8.0u60
- 1.8.0u45 and earlier

Non-listed releases are either not supported or haven't been put to extensive testing.

Setup Requirementsâ□□

ZooKeeperâ□□

Pulse uses ZooKeeper for coordinating all of its components (Pulse Servers and Application Engines) in a datacenter. With ZooKeeper Pulse knows what components available in a datacenter, be informed when one of them goes down or rejoins the datacenter, create distributed locks, elect a leader, as well as maintain information about each component.

To setup Pulse with ZooKeeper, you should follow these steps:

1. Install a ZooKeeper ensemble and make sure it is working properly.
2. Configure `pulse.manager.cluster.dc.{dc}.zookeeper.nodes` with the nodes, or a load balancer (recommended). The `{dc}` parameter refers to the name of the datacenter the nodes belong to. It is defined in the `node.properties` file.
3. Configure the `node.properties` with the ZooKeeper namespace wish to use. If you are preparing a coloured deployment, use the colour of the cluster of the nodes being configured. For example, you can use `pulseblue`.
4. Start Pulse. If the service is able to connect to ZooKeeper, a new namespace will be created automatically with the name you have specified in the previous step.

info

The current installation of Pulse has been tested with ZooKeeper 3.9.4.

Profile Storeâ□□

note

All configurations referenced in this section are prefixed by `pulse.app.{appSlug}.services.rte.workflow.plan.scheduledMetrics.profileStore` - this prefix is denoted in the examples below by `{PREFIX}`.

Pulse relies on a read-optimized NoSQL database system to store entity profile data (e.g. Customer-level aggregations), and be able to fetch in real-time without compromising latencies. The following systems are supported:

- Apache Cassandra 4.1 or higher - version 4.1.10 is the recommended version, and DSE 6.8 is also supported

You can decide which store to use with the configuration `{PREFIX}.storage`. You must also specify the name of the table that will store the profile data. That is set by `{PREFIX}.table`.

Cassandraâ□□

When using Cassandra, Pulse will also automatically create the profiles table, considering the following:

1. A Cassandra cluster is available, and working properly. To connect to Cassandra, you must define the contact points in `{PREFIX}.cassandra.contactPoints` (for example, `cassandra-1,cassandra-2`).
2. A user has been created for Pulse (see how to create users in this guide). Configuring Pulse with the credentials to access the Cassandra cluster is done by setting `{PREFIX}.cassandra.user|password`. Pulse requires a user that has permissions to:
    - Create a keyspace
    - Create tables
    - Modify (Insert, Delete, Update, Truncate)
    - Select

warning

Cassandra user credentials are stored in the `system_auth` keyspace, which is not replicated by default. This may make users unable to login if certain nodes are down. Read more about this in the DataStax documentation.

tip

To know more about the recommended Cassandra setup, refer to the Pulse official documentation. To see which configurations can be set to connect to Cassandra see the official configuration reference.

RDBMSâ□□

Pulse uses PulseDB for database access, which allows it to run on top of multiple database systems:

- Oracle 19.3 (or higher)
- PostgreSQL 14.20 (or higher) - driver version 42.7.10 should be used.
- SQL Server 2017 GA (or higher)
- H2 1.4.200 (or higher) - only recommended for testing purposes.

info

Feedzai validates the product primarily against **H2** and **PostgresSQL**.

caution

Before selecting a database engine, make sure you understand how different database systems are supported by our PulseDB.

Pulse relies on a relational database not only to store the events ingested and scored (also known as the "Event Storage"), but also to store metadata about the cluster and the applications that it serves. For that reason, before starting Pulse you must:

1. Provision a database with the engine of your choice.
2. Create a new user for Pulse.
3. Create a new schema for Pulse. If you are preparing a coloured deployment (i.e. to support Blue-Green deployments), you must create two schemas. For example, `pulseblue` and `pulsegreen`.
4. Update the Pulse configurations with the JDBC connection string, the schema, and the credentials for the user you just created.

The database settings must be configured for the following prefixes:

- Pulse Server Metadata - `pulse.global.database`
- Pulse AppEngine Metadata - `pulse.app.{appSlug}.database`
- AppEngine Event Storage - `pulse.app.{appSlug}.services.modules.rte.workflowmanager.eventStorage.database`

To connect to the database you need to provide the `jdbc`, `schema`, `username` and `password` settings.

tip

To see which configurations can be set to connect to a database see the official configuration reference.

RabbitMQâ□□

RabbitMQ is the messaging platform Pulse uses to communicate between machines and processes within the same datacenter.

The following RabbitMQ and Erlang versions are supported:

- RabbitMQ - 3.11.13 with Erlang - 25.3.2.21+
- RabbitMQ - 3.9.8 with Erlang - 24.0+

Before starting Pulse you must configure the messaging cluster:

1. Provision a RabbitMQ cluster and ensure there Pulse is able to establish a connection.
2. Create a new vhost for Pulse. If you are preparing a coloured deployment (i.e. to support Blue-Green deployments), you must create two vhosts. For example, `pulseblue` and `pulsegreen`.

```
rabbitmqctl rabbitmqctl add_vhost pulseblue
rabbitmqctl rabbitmqctl add_vhost pulsegreen
```

Copy

1. Create a new user for Pulse.

```
rabbitmqctl add_user pulse <password>
```

Copy

1. Update the vhost policies to grant all permissions to the Pulse user.

```
# set write and read permissions on every resource
rabbitmqctl set_permissions pulse -p pulseblue ".*" ".*" ".*"
rabbitmqctl set_permissions pulse -p pulsegreen ".*" ".*" ".*"
```

Copy

note

On single-node and testing environments, you can opt by using ActiveMQ as a local, embedded message broker. Nonetheless, ActiveMQ is not recommended even for standalone Production setups (e.g., using a standalone Pulse for model training activities).

No-Downtime Deploymentsâ

Pulse uses a **Blue-Green** deployment strategy to implement no-downtime deployments. In order to do that, the architecture of the Feedzai platform needs to be prepared for the deployment switching.

Blue-green deployment uses two almost identical clusters in the following way:

1. At the start, one of the clusters is in use. Let's call it the blue environment.
2. The other one is on standby, in the final testing phase of a development project. Let's call this one the green environment.
3. When the new software version has been validated, the system gets redirected to the green deployment, and the blue deployment becomes the stand-by environment. It is available in case of a necessary rollback.
4. When the new version is verified in production, the blue deployment can be upgraded, making it identical to the green one. While still on stand-by, the blue environment is ready to receive a new development project.

To support the segregation between the blue and the green clusters, it is necessary to do the following:

- Create coloured vhosts in RabbitMQ (the messages from each cluster are isolated).
- Create coloured namespaces in ZooKeeper (each cluster then has its own cluster). Pulse will actually create it automatically if it doesn't exist, but it still needs to be configured differently for the two clusters.

When starting a new deployment, the cluster being deployed must be configured with the opposite colour.

/opt/feedzai/pulse/confg/node.properties

```
node.cluster=pulsegreen
```

Copy

/opt/feedzai/pulse/confg/config.properties

```
configuration.clusterTablesPrefix=green
```

Copy

/opt/feedzai/pulse/confg/pulseinit.properties

```
pulse.manager.cluster.dc.dc1.messaging.vhost=pulsegreen
```

Copy

While the database configuration remains the same, the metadata tables are prefixed automatically with the value set by the `clusterTablesPrefix` property.

tip

The colour of the deployment should be a template in the configuration files, so that it can be updated dynamically when a new service instance is provisioned. This is particularly relevant in setups where immutable VM images are created and the same one is reused across the multiple environments (usually the case on Cloud-based setups).

RabbitMQ Federationâ

During a deployment, the active cluster continues to process events and synchronising state between the cluster nodes. However, if that state is not shared with new cluster, scoring will be impaired after switching the traffic to it. For that reason, it is important to **federate the event replicas between between the two clusters**. This is done with the RabbitMQ Federation plugin.

To connect the green cluster to receive updates from the blue one, you must create a federation policy, and then apply it to the the queues

```
rabbitmqctl set_parameter [-p vhost] federation-upstream blue-green '{"uri": ["amqp://
<user>:<pass>@<node 1>/pulseblue", "amqp://<user>:<pass>@<node 2>/pulseblue", ...]}'

rabbitmqctl set_policy [-p vhost] federate-blue-green 'global-fdz-metadata-sync|global-fdz-apps-
```

```
service|global-fdz-tenant-sync|global-fdz-tenant-group-sync|global-fdz-sync-topic-.*'
'{"federation-upstream": "blue-green"}' --apply-to exchanges
```

Copy

caution

When doing a Blue-Green deployment, only the topics above should be federated, otherwise there could be unwanted control messages being propagated between the two clusters (e.g. stopping the application).

Data Science Studioâ□□

Even though Pulse is commonly referred to as an Risk Engine and used for monitoring fraud in real-time, it is in fact a Data Science platform where the data analytics tooling can be integrated with a runtime environment.

The following are the requirements for setting up Pulse for a Data Science environment:

- A database for metadata storage, ideally a Relational one, such as PostgreSQL
- 1 machine for Pulse to execute on (a single replica)
- Coordination processes, which can be dockerized and/or executing in the same machine as Pulse:
  - 1 RabbitMQ process
  - 1 ZooKeeper process
- A Spark cluster for distributed job execution. The recommended approach is to run Spark on YARN (Cloudera, AWS EMR, Azure HDInsight, Google Dataproc), which requires a compliant Hadoop File System to exist in the cluster.

warning

Feedzai **does not recommend** the use of embedded coordination processes (i.e., using **embedded ZooKeeper** and **ActiveMQ**), as well as databases that are not production-ready, like **H2**.

For more information about the Data Science setup, refer to the official documentation.

Integration with JupyterLabâ□□

Pulse supports integration with JupyterLab.

- To enable Setup of JupyterLab:

  1. Docker must be installed on the Pulse machine.
  2. Feedzai Jupyterlab Docker Image must exist on Pulse machine.

- If the Docker software is not available on the Pulse machine, so the flag ***"pulse_jupyterlab_setup_docker"*** must be set to false, so that Pulse doesn't configure Jupyterlab erroneously.

On this page

Topology-Aware Resource Scheduling

Topology-Aware Resource Scheduling, or TARS for short, is the ability of distributing applications on a per node pool basis where a node pool is a set of Pulse instances.

This approach improves resiliency and resource management by:

- Isolating application processes, meaning that an application will keep running even if different node pools are degraded
- Giving the possibility of scaling machines according to the workload of each application

In this section you can find how TARS can be configured.

Overviewâ

As explained above TARS gives the possibility of distributing an application/set of applications in different node pools. The Pulse Server (PS) cluster will have to context of all deployed applications and acts as a single point of configuration for all.

In the following illustration you can find that:

- There are two node pools, `outbound` and `inbound`
- There are three Pulse instances, where two belong to `outbound` and one to `inbound`
- The Outbound application (Transfers and Digital Activity) is configured to be deployed in the `outbound` node pool
- The Inbound application is configured to be deployed in the `inbound` node pool
- The UI Load Balancer will connect to the Pulse leader node where the whole runtime cluster can be managed including the applications that are not deployed in the leader node
- The API Load Balancer on the other hand will only redirect traffic to the nodes that serve a specific application

This however does not mean one node pool can only serve one application. If, for instance, there are two applications that have a small workload they can be deployed in the same node pool.

The example below illustrates this case:

- The `Outbound` is still deployed in two (unshared) nodes
- The `Cards` application was configured to be deployed in the `inbound` node pool, hence sharing the node with the `Inbound` application
- The API Load Balancer for `Inbound` will now also redirect `Cards` requests for the `Cards` application running in the `inbound` node pool

Configurationâ

TARS configuration is controlled by the following Ansible variables:

- `apps_enabled` : should have all the applications to deploy
- `tars_enabled` : `true` to enable TARS, `false` otherwise
- `node_app_pool` : pool where the node belongs to, read from an environment variable
- `pulse_app_topology` : application topology in the structure defined below

```
pulse_app_topology:
  <app_1_id_as_defined_in_apps_enabled>:
    # should be the amount of instances across all pools (for apps without partitioned workflows)
    replication_factor: <app_1_replication_factor>
    app_pools:
```

```
      # dictionary of pools with the number of instances per pool
      <app_1_pool_name>:
        number_of_instances: <app_1_number_of_instances_in_pool>
  <app_2_id_as_defined_in_apps_enabled>:
    # should be the amount of instances across all pools (for apps without partitioned
workflows)
    replication_factor: <app_2_replication_factor>
    app_pools:
      # dictionary of pools with the number of instances per pool
      <app_2_pool_1_name>:
        number_of_instances: <app_2_number_of_instances_in_pool_1>
      <app_2_pool_2_name>:
        number_of_instances: <app_2_number_of_instances_in_pool_2>
```

Copy

The first thing to do regarding configuration is to define the topology you want to achieve. For the sake of this guide we will assume the topology defined in the second example given above, where `Inbound` and `Cards` will share a node pool and `Outbound` will have its own pool in two different nodes.

This means that when creating the instance groups they will need to have an environment variable indicating the pool they belong to. For this example the instance group for `Outbound` should have an environment variable `NODE_APP_POOL=outbound` and the `Inbound` / `Cards` should have its own set to `NODE_APP_POOL=inbound`.

warning

Since `node_app_pool` reads the node environment variable during deployment, the environment variable of each node needs to be set before running the deployment otherwise the pool will be configured as an empty string.

At this point with all the instance groups created we can continue configuration:

- Define the `apps_enabled` variable to include the applications to deploy

```
apps_enabled:
  - uk_outbound_payments
  - uk_reference_data
  - uk_inbound_payments
  - uk_cards
```

Copy

- Enable TARS by setting `tars_enabled: true`
- Define `pulse_app_topology` as follows:

```
pulse_app_topology:
  uk_outbound_payments:
    replication_factor: 2
    app_pools:
      outbound:
        number_of_instances: 2
  uk_reference_data:
    replication_factor: 1
    app_pools:
      outbound:
        number_of_instances: 1
  uk_inbound_payments:
    replication_factor: 1
    app_pools:
      inbound:
        number_of_instances: 1
  uk_cards:
    replication_factor: 1
    app_pools:
      inbound:
        number_of_instances: 1
```

Copy

Some notes regarding the `pulse_app_topology`:

- `uk_outbound_payments` has a `replication_factor` and `number_of_instances` of `2` because we want to have two instances of the `Outbound` application running in two different `outbound` nodes
- `uk_reference_data` will only be deployed in one of the `outbound` nodes
- `uk_inbound_payments` and `uk_cards` will be deployed in one `inbound` node

warning

Please note this example assumes the default `pulse_cluster_datacenters` variable was not overridden. In case it was the override needs to be extended as follows:

```
pulse_cluster_datacenters:
  - name:                              "dc1"
    numberofnodes:                     "{{ pulse_cluster_number_of_nodes }}"
    replicationfactor:                 "{{ pulse_cluster_replication_factor }}"
    (...)
    app_topology:                      "{{ pulse_app_topology if tars_enabled else [] }}"
```

Copy

After deployment you should be able to see how the applications were distributed in the Pulse UI by accessing the datacenter page in the administration tab.

Additional configurationâ□□

As stated before using TARS enables application distribution across different Pulse nodes while keeping the application configuration accessible via a singe Pulse UI.

This means that after a TARS deployment, the UI Load Balancer will need to be updated to include all the backends (Pulse instances) while choosing to where redirect traffic based on the leader node.

As for the API Load Balancer, given the example on this page, it will need to be configured as follows:

- Redirect `Transfers` and `Digital Activity` traffic to the `outbound` nodes
- Redirect `Inbound` and `Cards` traffic to the `inbound` nodes

info

Find a detailed description of all available configurations in Pulse Configuration Recommendations

On this page

Configuration Recommendations

Ansibleâ□□

info

Available ansible variables for running the ansible playbook.

Variable	Default Value	Description
reference_data_install_path	"/opt/feedzai"	Reference Data Service (RDS) install path
reference_data_home_path	"{{ reference_data_install_path }}/reference-data-{{ reference_data_version }}"	RDS home path
reference_data_app_port	8080	RDS connection port
reference_data_admin_port	8081	RDS administration connection port
cassandra_rds_username	"rds"	RDS username in Cassandra
cassandra_rds_password	"secret"	RDS user password in Cassandra
cassandra_cluster_name	"project-hsbc cluster"	Profile store cluster name in Cassandra
cassandra_rds_consistency	"LOCAL_ONE"	Profile store consistency level in Cassandra
cassandra_rds_keyspace_replication	"'class':'NetworkTopologyStrategy', 'dc1': {{ groups['cassandra']	length }}"
reference_data_cassandra.tableName	feedzai_reference_data	Cassandra profile store table name
reference_data_cassandra.keyspaceName	hsbc	Cassandra profile store keyspace name
reference_data_cassandra.username	{{ cassandra_rds_username }}	Cassandra RDS username
reference_data_cassandra.password	{{ cassandra_rds_password_encrypted }}	Cassandra RDS user password
reference_data_cassandra.cluster	{{ cassandra_cluster_name }}	Cassandra cluster name
reference_data_cassandra.contactPoints	{{ groups['cassandra'] }}	Cassandra contact points
reference_data_cassandra.consistencyLevel	{{ cassandra_rds_consistency }}	Cassandra consistency level
reference_data_cassandra.replicationStrategy	{{ cassandra_rds_keyspace_replication }}	Cassandra replication strategy
reference_data_jwt_secret	"0jd40327td208347t02k3487t23k40t87320t"	JWT secret, needs to be set
reference_data_use_https	false	When `true` RDS will use https for connection

Variable	Default Value	Description
reference_data_app_keystore_path	"{{ reference_data_home_path }}/conf/keystore.jks"	Keystore path, needs to be set when reference_data_use_https is `true`
reference_data_app_keystore_password	N/A	Keystore password, needs to be set when reference_data_use_https is `true`
reference_data_app_keystore_type	"JKS"	Key store type, needs to be set when reference_data_use_https is `true`
reference_data_app_keymanager_password	N/A	Key manager password, needs to be set when reference_data_use_https is `true`
reference_data_admin_keystore_path	"{{ reference_data_home_path }}/conf/keystore.jks"	Keystore path, needs to be set when reference_data_use_https is `true`
reference_data_admin_keystore_password	N/A	Keystore password, needs to be set when reference_data_use_https is `true`
reference_data_admin_keystore_type	"JKS"	Key store type, needs to be set when reference_data_use_https is `true`
reference_data_admin_keymanager_password	N/A	Key manager password, needs to be set when reference_data_use_https is `true`
reference_data_validate_certs	true	Set to validate certificates, needs to be set when reference_data_use_https is `true`
reference_data_validate_peers	true	Set to validate peers, needs to be set when reference_data_use_https is `true`
reference_data_allow_renegotiation	true	Set to allow renegotiation, needs to be set when reference_data_use_https is `true`
supportedProtocols	[TLSv1.2]	Set supported protocols, needs to be set when reference_data_use_https is `true`
supportedCipherSuites	[TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256]	Set supported ciphers, needs to be set when reference_data_use_https is `true`
reference_data_secret_value	"830ebea1212f74040dbab3506cad4ec3"	Value used to encrypt sensitive properties

Variable	Default Value	Description
reference_data_properties_to_encrypt	[cassandra_rds_password]	List of properties to encrypt. Encrypted properties can be used by adding the suffix `_encrypted` to the original property name, i.e `<PROPERTY_NAME>_encrypted` . Doesn't support dictionary variables
reference_data_log_level	INFO	Default log level for all components in RDS. Please note that changing this value may impact the amount of produced logs.
reference_data_loggers	{}	Controls extra loggers in RDS, allowing to override the default value per logger. Keys in this dictionary are the loggers and the values the log level for that logger.

RDS Service configurationâ□□

In addition to the above Ansible variables, some properties can be directly changed on the Reference Data Service's configuration file, configuration.yml.

Please find below a description of each of the available configurations.

Property	Default Value	Description
server.applicationConnectors.supportedProtocols	[TLSv1.2]	Set supported protocols, needs to be set when reference_data_use_https is `true`
server.applicationConnectors.supportedCipherSuites	[TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256]	Set supported ciphers, needs to be set when reference_data_use_https is `true`
server.adminConnectors.supportedProtocols	[TLSv1.2]	Set supported protocols, needs to be set when reference_data_use_https is `true`
server.adminConnectors.supportedCipherSuites	[TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256]	Set supported ciphers, needs to be set when reference_data_use_https is `true`

info

There are no configuration recommendations for this service besides the ones already described in the guide.

On this page

Network and Connectivity

info

This page is based on the architecture of Feedzai standard solutions. Specific clients or custom solutions may have additional requirements, so this is not a definitive guide.

Firewall Rulesâ□□

tip

For a complete list of the the Firewall Configurations required for Reference Data Service, please refer to the product documentation.

Inboundâ□□

note

Inbound â□□other services connect to this service.

Service	Port	Description
HTTP	TCP 8080-8081	HTTP Requests from Reference Data Service client.
SSH	TCP 22	SSH access (if available)

Outboundâ□□

note

Outbound â□□this service connects to external services.

Service	Port	Description
Database (Cassandra)	TCP 9042	Connection to the database

Load Balancersâ□□

For high available environments Load Balancers must be set as follow:

Service	Description	Algorithm	Healthcheck URL	Response Codes	Policy Example
Reference Data Service	Used to read/write reference data.	Round robin	`HTTPS:8080/health` `HTTP:8080/health`	200 - Node is available	Checks should be performed every 10 seconds with a timeout of 2 seconds. Nodes should be removed after the second failure.

RDS Installation Guide

Welcome to the RDS Installation Guide.

In this section you will find a set of pages with information about the RDS Installation process:

On this page

Requirements

Service Requirementsâ□□

This service requires the following software to run.

Java Environmentâ□□

OpenJDK SE Development Kit version 1.8, with the following vendors (with their latest hotfix versions):

- Azul Zulu rpm zip tar.gz
- Azul Platform Prime (formerly Zing)
- Amazon Corretto
- Oracle

tip

Product tests are done with the **Azul Zulu JVM**.

The following versions are not supported, and are known to have problems:

- 1.8.0u242
- 1.8.0u121
- 1.8.0u60
- 1.8.0u45 and earlier

Non-listed releases are either not supported or haven't been put to extensive testing.

Setup Requirementsâ□□

Databaseâ□□

The Reference Data Service is a service responsible for storing reference data that belongs to individual entities (e.g. Merchant, Customer, etc). The type of storage that betters suits this use-case are *key-value* stores.

The following systems are supported:

- Apache Cassandra 4.1 or higher - version 4.1.10 is the recommended version, and DSE 6.8 is also supported

Cassandraâ□□

When using Cassandra, RDS will also automatically create the entities tables, considering the following:

1. A Cassandra cluster is available, and working properly. To connect to Cassandra, you must define the contact points in `cassandra.contactPoints` (for example, `cassandra-1,cassandra-2`).
2. A user has been created for RDS with the following permissions:

- Create a Keyspace
- Create tables
- Modify (Insert, Delete, Update, Truncate)
- Select

The credentials of the user must be configured in `cassandra.user|password` .

On this page

Firewall Rules for Cassandra

info

This page is based on the architecture of Feedzai standard solutions. Specific clients or custom solutions may have additional requirements, so this is not a definitive guide.

Inboundâ□□

note

Inbound â□□other services connect to this service.

Service	Port	Description
Cassandra	TCP 7000-7001	Cluster communication
Pulse	TCP 9042	Requests from Cassandra Driver
Reference Data Service	TCP 9042	Requests from Cassandra Driver
SSH	TCP 22	SSH access (if available)

Outboundâ□□

note

Outbound â□□this service connects to external services.

Service	Port	Description
Cassandra	TCP 7000-7001	Connection to the database

On this page

Firewall Rules for RabbitMQ

info

This page is based on the architecture of Feedzai standard solutions. Specific clients or custom solutions may have additional requirements, so this is not a definitive guide.

Inboundâ□□

note

Inbound â□□other services connect to this service.

Service	Port	Description
Erlang	TCP 25672	Erlang inter-node and CLI tool communication
Erlang	TCP 4369	Peer discovery service (Erlang)
Pulse	TCP 5671-5672	RabbitMQ AMQP (5671 for TLS)
Case Manager	TCP 5671-5672	RabbitMQ AMQP (5671 for TLS)
Logstash	TCP 5671-5672	RabbitMQ AMQP (5671 for TLS)
HTTP	TCP 15672	RabbitMQ Management UI
SSH	TCP 22	SSH access (if available)

Outboundâ□□

note

Outbound â□□this service connects to external services.

Service	Port	Description
Erlang	TCP 25672	Erlang inter-node and CLI tool communication
Erlang	TCP 4369	Peer discovery service (Erlang)

Network Requirements

Welcome to the Network Requirements Guide.

In this section you will find a set of pages with information about Network Requirements:

On this page

Firewall Rules for ZooKeeper

info

This page is based on the architecture of Feedzai standard solutions. Specific clients or custom solutions may have additional requirements, so this is not a definitive guide.

Inboundâ□□

note

Inbound â□□other services connect to this service.

Service	Port	Description
ZooKeeper	TCP 2888	Peer communication
ZooKeeper	TCP 3888	Peer communication
Pulse	TCP 2181	ZooKeeper Client communication
SSH	TCP 22	SSH access (if available)

Outboundâ□□

note

Outbound â□□this service connects to external services.

Service	Port	Description
ZooKeeper	TCP 2888	Peer communication
ZooKeeper	TCP 3888	Peer communication

On this page

Recommendations for Cassandra

info

The configurations listed below result from the experience Feedzai has running this system. Nonetheless, it is important to review the configurations in light of the workload requirements, and adjusted accordingly.

Cassandra Configurationâ

The `cassandra.yaml` file is the main configuration file for Cassandra.

Property	Recommended Value	Description
cluster_name	`Feedzai Cassandra Cluster`	This setting prevents nodes in one logical cluster from joining another. All nodes in a cluster must have the same value.
num_tokens	256	Set this property for virtual node token architecture. Determines the number of token ranges to assign to this [vnode].
endpoint_snitch	`GossipingPropertyFileSnitch` (see note below)	The implementation used informs Cassandra about the network topology to route requests efficiently, and allows Cassandra to spread replicas around your cluster to avoid correlated failures.
concurrent_reads	`8 * #cores`	
concurrent_writes	`8 * #cores`	
trickle_fsync	true	Causes fsync to force the operating system to flush the dirty buffers at the set interval `trickle_fsync_interval_in_kb`, to prevent sudden flushes from impacting read latencies.
trickle_fsync_interval_in_kb	4096	The size of the fsync in kilobytes.
concurrent_compactors	`#cores`	The number of concurrent compaction processes allowed to run simultaneously on a node.
compaction_throughput_mb_per_sec	0	Throttles compaction to the specified Mb/second across the instance. Setting the value to 0 disables compaction throttling.
disk_failure_policy	stop	On disk failures, shuts down gossip and client transports, leaving the node effectively dead, but it still can be inspected with JMX.
commit_failure_policy	stop	On commit disk failures, shuts down gossip and client transports, leaving the node effectively dead, but it still can be inspected with JMX.
commitlog_sync	periodic	Ack writes immediately and sync the CommitLog every `commitlog_sync_period_in_ms`
commitlog_sync_period_in_ms	10000	Frequency of the CommitLog sync when in `periodic` mode.
auto_bootstrap	true	It causes new (non-seed) nodes migrate the right data to themselves automatically.

info

Endpoint Snitch

With the `GossipingPropertyFileSnitch` the rack and datacenter for the local node must be defined in `cassandra-rackdc.properties`.

If you are installing Cassandra on AWS, the recommendation is to use `Ec2Snitch`. This will rely on the instances and the availability zones they are done to define a better and more reliable topology.

If you are installing Cassandra on GCP, there is a `GoogleCloudSnitch` available.

Make sure `ntp` is installed, and all Cassandra servers are in-sync.

Javaâ□□

Java Native Access (JNA)â□□

Using JNA is one way to improve Cassandra memory usage, and, hence, performance. If JNA is not installed and enabled, Cassandra will not boot unless `cassandra.boot_without_jna` is set to true.

JVM Optsâ□□

It is advisable that the `-Xms` and `-Xmx` flags are set to the same value. Optionally, `-Xmn` could be set as well, according to the resources and workload.

Make sure you define the most appropriate GC algorithm according to the workload and the heap size.

On this page

Recommendations for RabbitMQ

info

The configurations listed below result from the experience Feedzai has running this system. Nonetheless, it is important to review the configurations in light of the workload requirements, and adjusted accordingly.

RabbitMQ Configurationâ

Highly Available Queuesâ

RabbitMQ clustering capabilities do not offer replicated message queues by default, so a message queue only resides in the node which created them. As such, Feedzai recommends configuring Highly Available Queues.

In contrast to exchanges and bindings, which can always be considered to be on all nodes, queues can optionally run mirrors (additional replicas) on other cluster nodes.

Each mirrored queue consists of one **leader replica** and one or more **mirrors** (replicas). The leader is hosted on one node commonly referred to as the leader node for that queue. Each queue has its own leader node. All operations for a given queue are first applied on the queue's leader node and then propagated to mirrors.

warning

Consumers are connected to the leader regardless of which node they connect to, with mirrors dropping messages that have been acknowledged at the leader. Queue mirroring therefore enhances availability, but does not distribute load across nodes (all participating nodes each do all the work).

To create policy that defines the mirroring conditions, use the command below.

```
rabbitmqctl set_policy replicate-queues "^(fdz-sq-)" '{"ha-mode":"all", "expires":7200000, "ha-sync-batch-size":1000, "ha-promote-on-shutdown":"always"}' --apply-to queues
```

Copy

tip

All queues created by Feedzai services have the prefix `fdz-sq` , and are suffixed with a UUID to ensure that new processes will always get new queues - for example, `fdz-sq-fdz-workflow-publish-responsec6e7cef5-3239-4a9f-963b-98ef108eb4c5` .

Queue TTLâ

Defining a TTL for Pulse queues ensures that queues **without subscribers or without consuming the messages** will be deleted after a certain time. Feedzai recommends it to be at least two times the configured client session timeout in ZooKeeper. The TTL must also be confortably higher than the Workflow Recovery time, as during recovery Pulse nodes already subscribed to replicas, but haven't started to consume them. This also means that, in case of an incident, there is a limited period to recover the system without losing any messages.

The queue TTL is set using the `expires` configuration, as shown in the example above.

info

To ensure data is not lost in case of an incident with Case Manager, Feedzai usually recommends against setting the TTL for the Case Manager vhost.

Recommendations for ZooKeeper

info

There are no configuration recommendations for this service besides the ones already described in the guide.

On this page

Case Manager Alert Queue Cleanup

In this page you can find the Case Manager alert cleanup process definition, how to configure and run it.

Alert queue cleanupâ

The Case Manager queue cleanup is a process that can be used to close all Case Manager alerted events that match a specific criteria with the intended reason. This is particularly useful before a go live so that Case Manager alert pages do not include any alerts that may have been already acted upon on a different platform.

This process is essentially a Python script that loads a set of events based on a given criteria, removes the queues from those events and closes them with a given reason. It also supports a mode in which it simply closes open alerts that are not assigned to any queue.

Requirementsâ

The script execution has the following requirements:

- Connection between the machine where the script is being run and the Case Manager load balancer
- A service account credentials
- Python version 3.8+ installed
- `requests` Python library installed

warning

This script was tested against Python version 3.8. If this specific version is not available, the script can be tested against the available one but take into account it may be unsupported.

Resourcesâ

The cleanup script can be downloaded from cleanup-alerts-queues.txt. The extension must be changed from `.txt` to `.py.` after downloading. The rest of the documentation below assumes the script has been renamed to `cleanup-alerts-queues.py`.

The corresponding dependency requirements file for the script can be downloaded from requirements.txt.

Configurationâ

In this section you can find the configuration reference for the script.

Variable	Mandatory	Default Value	Description
`-h, --help`			Show an help message.
`-c, --url`	X		The Case Manager load balancer URL.
`-e, --exec-mode`			Flag indicating if the script should make any change. If not specific it will do a test run.
`-sk, --skip-ssl`		`False`	Flag indicating if SSL certificate validation should be skipped when making HTTPS requests
`-u, --username`		`pulse`	Username of the service account.

Variable	Mandatory	Default Value	Description
`-p, --password`	X		Password of the service account (will prompt if not specified).
`-s, --status-reason`	X		Status reason name to add when closing an event that will be used to fetch the reason id as shown in the UI.
`-ch, --channel`		`transfers`	Channel to search for events.
`-mints, --min_timestamp`		`1688169600000`	The minimum timestamp to fetch events in milliseconds.
`-mints, --max_timestamp`		`1688169600000`	The maximum timestamp to fetch events in milliseconds.
`-wq, --without_queue`		`1688169600000`	Flag indicating if the script should close only alerts not assigned to any queue.
`-t, --threshold`		`100`	Threshold to apply to the search filter.
`-r, --retries`		`3`	Number of retries when making HTTP requests.
`-b, --backoff`		`1`	Backoff factor for the HTTP requests.
`-tk, --tenant-keys`			The tenant keys (one or more) used to filter events. Should be used without `-ats, --all-tenants`.
`-ats, --all-tenants`			Include all tenants when filtering events. Should be used without `-tk, --tenant-keys`.

Running the script with the help flag (`-h`) will output the script information and available configuration:

```
$ python3 cleanup-alerts-queues.py -h
usage: cleanup-alerts-queues.py [-h] -c CM_URL [-e] [-sk] [-u USERNAME] [-p PASSWORD] -s
STATUS_REASON
                                [-ch CHANNEL] [-mints MIN_TIMESTAMP] [-maxts MAX_TIMESTAMP] [-
wq] [-t THRESHOLD]
                                [-r NUMBER_RETRIES] [-b BACKOFF_FACTOR] (-tk TENANT_KEYS [TENANT_
KEYS ...] | -ats)

This script cleanups Case Manager queued alerts by closing them with the given status reason.

optional arguments:
  -h, --help            show this help message and exit
  -tk, --tenant-keys TENANT_KEYS [TENANT_KEYS ...]
                        The tenant keys (one or more) used to filter events. Should be used
without `-ats, --all-tenants`.
  -ats, --all-tenants   Include all tenants when filtering events. Should be used without `-tk,
--tenant-keys`.

  -c CM_URL, --url CM_URL
                        Case Manager URL
  -e, --exec-mode       Run in change mode. If not specific will do a dry run.
  -sk, --skip-ssl       Skip SSL validation when making HTTPS requests.
  -u USERNAME, --username USERNAME
                        Username of the service account (default: pulse)
  -p PASSWORD, --password PASSWORD
                        Password of the service account (will prompt if not specified)
  -s STATUS_REASON, --status-reason STATUS_REASON
                        Status reason name to add when closing an event that will be used to
fetch the reason id.
  -ch CHANNEL, --channel CHANNEL
                        Channel to search for events (default: transfers)
  -mints MIN_TIMESTAMP, --min_timestamp MIN_TIMESTAMP
                        The minimum timestamp to fetch events in milliseconds (default: 168816960
0000)
  -maxts MAX_TIMESTAMP, --max_timestamp MAX_TIMESTAMP
                        The maximum timestamp to fetch events in milliseconds (default: 168816960
```

```
0000)
  -wq, --without_queue  Close only alerts not assigned to any queue. If not specified, will close
alerts not assigned
                        to a queue, clearing all queues.
  -t THRESHOLD, --threshold THRESHOLD
                        Threshold to apply to the search filter (default: 100)
  -r NUMBER_RETRIES, --retries NUMBER_RETRIES
                        Number of retries when making HTTP requests (default: 3)
  -b BACKOFF_FACTOR, --backoff BACKOFF_FACTOR
                        Backoff factor for the HTTP requests (default: 1)
```

Copy

Executionâ□□

The bare minimum execution only requires the fields that are define as being mandatory. All other fields will use their default values but take note that they might need to be adjusted.

info

When targeting a HTTPS endpoint, SSL certificate validation can fail if the certificate is not available in the machine or if it is not correctly configured.

To skip SSL certificate validation the `-sk/--skip-ssl` flag should be passed to the script:

```
$ python3 cleanup-alerts-queues.py -c https://localhost:8090/ -sk [Other Options]
```

It is suggested to execute the script without the `-e` flag to understand if all parameters are correctly set. Running in this mode will output the number of events it would try to update.

```
$ python3 cleanup-alerts-queues.py -c http://localhost:8090/ -s "Alert not investigated"
Service account password:
Running in test mode...
Found reason Alert not investigated with id d84ad2d1-xxxx-xxxx-xxxx-xxxxxxxxxxxx
Would try to close 11 events.
Finished running cleanup.
```

Copy

Once validated in test mode the script can be run in execution mode by passing the `-e/--exec-mode` flag:

```
$ python3 cleanup-alerts-queues.py -c http://localhost:8090/ -s "Alert not investigated" -e
Service account password:
Running in change mode...
Found reason Alert not investigated with id d84ad2d1-xxxx-xxxx-xxxx-xxxxxxxxxxxx
Clean queue executed for events: [{'id': '55f3cf56-xxxx-xxxx-xxxx-xxxxxxxxxxxx', 'timestamp': 173
2533728533, 'channelId': 'transfers'}, {'id': '6ec36930-xxxx-xxxx-xxxx-xxxxxxxxxxxx',
'timestamp': 1732533729218, 'channelId': 'transfers'}, {'id': '9735dfbf-xxxx-xxxx-xxxx-
xxxxxxxxxxxx', 'timestamp': 1732533730087, 'channelId': 'transfers'}, {'id': '9ad7ab9e-xxxx-xxxx-
xxxx-xxxxxxxxxxxx', 'timestamp': 1732533730728, 'channelId': 'transfers'}, {'id': '9fdd9fb4-xxxx-
xxxx-xxxx-xxxxxxxxxxxx', 'timestamp': 1732537566308, 'channelId': 'transfers'}, {'id': '4d844905-
xxxx-xxxx-xxxx-xxxxxxxxxxxx', 'timestamp': 1732537672724, 'channelId': 'transfers'}, {'id': '9806
7255-xxxx-xxxx-xxxx-xxxxxxxxxxxx', 'timestamp': 1732537673556, 'channelId': 'transfers'}, {'id':
'2064b6c7-xxxx-xxxx-xxxx-xxxxxxxxxxxx', 'timestamp': 1732537674340, 'channelId': 'transfers'}, {'
id': 'a4df2a90-xxxx-xxxx-xxxx-xxxxxxxxxxxx', 'timestamp': 1732537700312, 'channelId':
'transfers'}, {'id': '756e260b-xxxx-xxxx-xxxx-xxxxxxxxxxxx', 'timestamp': 1732537701173, 'channe
lId': 'transfers'}, {'id': '6729e208-xxxx-xxxx-xxxx-xxxxxxxxxxxx', 'timestamp': 1732537701825, '
channelId': 'transfers'}]
Move to close executed for transactions: [{'id': '55f3cf56-xxxx-xxxx-xxxx-xxxxxxxxxxxx', 'timest
amp': 1732533728533, 'channelId': 'transfers'}, {'id': '6ec36930-xxxx-xxxx-xxxx-xxxxxxxxxxxx', '
timestamp': 1732533729218, 'channelId': 'transfers'}, {'id': '9735dfbf-xxxx-xxxx-xxxx-
xxxxxxxxxxxx', 'timestamp': 1732533730087, 'channelId': 'transfers'}, {'id': '9ad7ab9e-xxxx-xxxx-
xxxx-xxxxxxxxxxxx', 'timestamp': 1732533730728, 'channelId': 'transfers'}, {'id': '9fdd9fb4-xxxx-
xxxx-xxxx-xxxxxxxxxxxx', 'timestamp': 1732537566308, 'channelId': 'transfers'}, {'id': '4d844905-
```

```
xxxx-xxxx-xxxx-xxxxxxxxxxxx', 'timestamp': 1732537672724, 'channelId': 'transfers'}, {'id': '9806
7255-xxxx-xxxx-xxxx-xxxxxxxxxxxx', 'timestamp': 1732537673556, 'channelId': 'transfers'}, {'id':
'2064b6c7-xxxx-xxxx-xxxx-xxxxxxxxxxxx', 'timestamp': 1732537674340, 'channelId': 'transfers'}, {'
id': 'a4df2a90-xxxx-xxxx-xxxx-xxxxxxxxxxxx', 'timestamp': 1732537700312, 'channelId':
'transfers'}, {'id': '756e260b-xxxx-xxxx-xxxx-xxxxxxxxxxxx', 'timestamp': 1732537701173, 'channe
lId': 'transfers'}, {'id': '6729e208-xxxx-xxxx-xxxx-xxxxxxxxxxxx', 'timestamp': 1732537701825, '
channelId': 'transfers'}]
There are no more events to update that matches the condition
Finished running cleanup.
```

Copy

As you can see, when running in execution mode, the script will output the ids of the events it was able to close, which can be used for validation purposes.

Filtering eventsâ□□

There are some filtering options that can be used to better determine the amount of events to close.

Queuesâ□□

To filter events by assigned queue the `-wq, --without_queue` flag can be used. If provided, the script will only search for events that are not assigned to any queue and therefore will not try to clear any queues.

If not provided, the script will search for all events that are assigned or not to queues. Take into consideration the script will remove the queue from events meaning that in case of a re-run the output may not be consistent.

Tenantsâ□□

In multi tenancy setups the options `-ats, --all-tenants` and `-tk, --tenant-keys` can be used to filter events based on all or only the provided tenants. The second option supports one or more tenant ids, where `tenant id` is the Tenant's Unique ID as shown in the Pulse configuration page.

As an example, consider a multi tenant setup where:

- Tenant 1 has the Unique ID `tenant_1`
- Tenant 2 has the Unique ID `tenant_2`
- Tenant 3 has the Unique ID `tenant_3`

And the following cases:

- Clear events from all tenants
- Clear events from tenants 1 and 2
- Clear events from tenant 3

This would require running the script as follows:

```
# Case 1
$ python3 cleanup-alerts-queues.py -c http://localhost:8090/ (...) -ats

# Case 2
$ python3 cleanup-alerts-queues.py -c http://localhost:8090/ (...) -tk "tenant_1" "tenant_2"

# Case 3
$ python3 cleanup-alerts-queues.py -c http://localhost:8090/ (...) -tk "tenant_3"
```

Copy

warning

Please note that these options are mutually exclusive but still required.

Also note that `-ats, --all-tenants` should be used carefully in multi tenancy setups to prevent closing events wrongfully. Consider running the script in test mode prior to execution mode.

Validationâ□□

Validation can also be done in the UI by searching for events that fall in the same criteria. If the script processed every event correctly the search should bear no results.

To validate a subset of events one can check the operation status, close reason and queue in the event details of the events, as illustrated below.

Every operation done by the script is audited and can be seen in the event details page as well.

On this page

Migrate Cassandra Profile Store

This document provides a step-by-step guide for migrating the Cassandra Profile Store, which stores the scheduled metrics (SMJs) values.

Please note this page outlines one of the many approaches to migrate a keyspace/table. This migration is not mandatory but reduces the possibility of metric collisions between channels.

Migration Overviewâ□□

Migrating the Profile Store from a single keyspace/table to multiple keyspaces/tables requires several steps to ensure that the metrics are not unavailable during the migration process. The process includes the following steps:

1. Prepare the new Cassandra keyspace and table by creating all the necessary data structures
2. Backfill the new table with the existing data to avoid null/zero metrics
3. Update the Pulse properties to start using the new keyspace/table
4. Clean up the old keyspace/table

Migration Stepsâ□□

In this section you can find a description of the migration steps.

For this migration process explanation, we will assume that the objective is to migrate the table `profiles` in the keyspace `profiles` to a new table `uk_oub_profiles` in a new keyspace `uk_profiles`.

Prepare the new Cassandra keyspace and/or tableâ□□

The first step is to prepare the new keyspace and/or table, ensuring it has the same definition has the existing one.

This can be done by following the next steps:

1. Describe the existing keyspace/table

```
DESCRIBE profiles;
```

Copy

2. Copy the output from the previous command, e.g.

```
CREATE KEYSPACE profiles WITH replication = {'class': 'NetworkTopologyStrategy', 'dc1': '1
'}  AND durable_writes = true;

CREATE TABLE profiles.profiles (
    key text,
    column1 text,
    value text,
    PRIMARY KEY (key, column1)
) WITH CLUSTERING ORDER BY (column1 ASC)
    AND additional_write_policy = '99p'
    AND bloom_filter_fp_chance = 0.1
    AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
    AND cdc = false
    AND comment = ''
    AND compaction = {'class': 'org.apache.cassandra.db.compaction.LeveledCompactionStrat
egy', 'max_threshold': '32', 'min_threshold': '4'}
```

```
    AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.comp
ress.LZ4Compressor'}
    AND memtable = 'default'
    AND crc_check_chance = 1.0
    AND default_time_to_live = 0
    AND extensions = {}
    AND gc_grace_seconds = 864000
    AND max_index_interval = 2048
    AND memtable_flush_period_in_ms = 0
    AND min_index_interval = 128
    AND read_repair = 'BLOCKING'
    AND speculative_retry = '99p';
```

Copy

3. Rename the keyspace/table and create the new keyspace/table

```
CREATE KEYSPACE uk_profiles WITH replication = {'class': 'NetworkTopologyStrategy', 'dc1':
 '1'}  AND durable_writes = true;

CREATE TABLE uk_profiles.uk_oub_profiles (
    key text,
    column1 text,
    value text,
    PRIMARY KEY (key, column1)
) WITH CLUSTERING ORDER BY (column1 ASC)
    AND additional_write_policy = '99p'
    AND bloom_filter_fp_chance = 0.1
    AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
    AND cdc = false
    AND comment = ''
    AND compaction = {'class': 'org.apache.cassandra.db.compaction.LeveledCompactionStrat
egy', 'max_threshold': '32', 'min_threshold': '4'}
    AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.comp
ress.LZ4Compressor'}
    AND memtable = 'default'
    AND crc_check_chance = 1.0
    AND default_time_to_live = 0
    AND extensions = {}
    AND gc_grace_seconds = 864000
    AND max_index_interval = 2048
    AND memtable_flush_period_in_ms = 0
    AND min_index_interval = 128
    AND read_repair = 'BLOCKING'
    AND speculative_retry = '99p';
```

Copy

warning

Please note the commands above have default configuration values (e.g. replication factor) which might have to be adjusted according to the cluster configuration.

Backfill dataâ□□

To ensure that we have consistent metrics values while migrating the profile store keyspace and/or table, we should backfill it with the current data. Backfilling is an essential step to guarantee that metrics are always available in the current/ new Pulse cluster.

This can be done using the CQL `COPY` command to export the current data and import it into the new table. Example:

```
COPY profiles.profiles TO 'profiles.csv';
COPY uk_profiles.uk_oub_profiles FROM 'profiles.csv';
```

Copy

Please refer to the official Cassandra documentation for more details regarding the CQL `COPY` command:

- COPY TO
- COPY FROM

Update Pulse propertiesâ

To complete the migration, some Pulse properties need to be redirected to the new keyspace/table:

- Keyspace name (Ansible variable: `cassandra_pulse_keyspace`):

```
pulse.app.
{appSlug}.services.rte.workflow.plan.scheduledMetrics.profileStore.cassandra.keyspace.name
```

Copy

- Table name (Ansible variable: `cassandra_table_name`):

```
pulse.app.{appSlug}.services.rte.workflow.plan.scheduledMetrics.profileStore.table
```

Copy

The properties can be updated through Pulse UI or by using Ansible variables (requires a deploy). When updated through the Pulse UI, a publish with restart is required but all changes will be lost once a Blue/Green deployment is performed.

The application variable override should be used to set different keyspaces/tables for different applications. For example, considering a single UK keyspace and different tables per application, the configuration would be:

```
cassandra_pulse_keyspace                 : "uk_profiles"
uk_outbound_payments_cassandra_table_name: "uk_oub_profiles"
uk_cards_cassandra_table_name            : "uk_cds_profiles"
uk_inbound_payments_cassandra_table_name : "uk_inb_profiles"
```

Copy

After these changes are done and all applications are published, the migration process is now complete.

Clean-upâ

Once validated that everything is working as expected, the old keyspace/table can be safely removed:

```
DROP TABLE profiles.profiles;
DROP KEYSPACE profiles;
```

Copy

On this page

Export HAR files

In this page you can find the steps to export a HAR (HTTP Archive) file from different browsers (Chrome, Firefox and Edge). A HAR file contains detailed network requests made by the browser when loading a webpage. It is especially useful for troubleshooting issues like slow page loads, network errors, or failed requests. By generating and sharing the HAR file with the Feedzai team, they can analyze the issue more effectively.

Key Sections:â□□

Export HAR file in Google Chromeâ□□

Please find below the steps to export a HAR file in Chrome:

1. Open Chrome and go to the problematic page
    - Navigate to the page where the issue occurs.
2. Access Developer Tools
    - Click the **three dots (â□®)** in the top right corner.
    - Go to **More Tools > Developer Tools** or press `Ctrl + Shift + I` (`Cmd + Option + I` on Mac).
    - This will open the Developer Tools panel.
3. Select the Network Tab
    - In the Developer Tools panel, click the **Network** tab.
4. Start Recording
    - Ensure the **record button** (a round button in the upper-left corner) is **red**. If itâ□□s **grey**, click it to start recording.
    - Check the **Preserve log** box to ensure all requests are recorded even when the page is refreshed.
5. Reproduce the issue
    - Reload the page and replicate the issue so the network requests can be captured.
6. Export the HAR file
    - After reproducing the issue, click the Export button to download the HAR file. The file will be saved in `.har` format.

Export HAR file in Mozilla Firefoxâ□□

Please find below the steps to export a HAR file in Firefox:

1. Open Firefox and go to the problematic page
    - Navigate to the page with the issue.

2. Open the Network Monitor
    - Press `Ctrl + Shift + E` (`Cmd + Option + E` on Mac) to open the Network Monitor.
3. Reproduce the issue
    - Perform the actions that lead to the issue while the network requests are being logged.
4. Export HAR file
    - Right-click on any entry in the File column and select **Save All As HAR**.
    - Save the HAR file to your computer.

Export HAR file in Microsoft Edgeâ□□

Please find below the steps to export a HAR file in Edge:

1. Open Edge and go to the problematic page
    - Navigate to the page where the issue occurs.
2. Access Developer Tools
    - Press `F12` or **Right-click > Inspect** to open the Developer Tools.
3. Select the Network Tab
    - Click the **Network** tab.
4. Start Recording
    - Ensure the **record button** is **red**. If itâ□□s **grey**, click it to start recording.
    - Check the **Preserve log** box to ensure all requests are recorded even when the page is refreshed.
5. Reproduce the issue
    - Perform the actions that lead to the error while the network requests are being logged.
6. Export HAR file
    - After reproducing the issue, click the Export button to download the HAR file. The file will be saved in `.har` format.

Example: exporting a HAR file for a failed Case Manager search in Chromeâ□□

In this section you can find an example of how to export a HAR file after a failed search in Case Manager.

1. Open Chrome and navigate to the page where the search functionality is located.
2. Open Developer Tools:
    - Click the **three vertical dots** in the top-right corner of Chrome.
    - Go to **More Tools > Developer Tools**.
3. Go to the Network Tab:
    - Click on the **Network** tab in Developer Tools.
    - Ensure the **Preserve log** checkbox is checked. This ensures that the network log is retained even after the page reloads.
    - Make sure the **Record button** (round red circle in the top-left corner) is active (if itâ□□s grey, click it to start recording).

4. Clear existing Logs:
    - Before triggering the search, click the **clear button** (a small grey circle with a line through it) to clear any old network requests from the log.
5. Perform the failing search:
    - In the webpage, perform the search action that fails
    - As the request is made, it will appear in the **Network** tab.
6. Check for error responses:
    - Look for any requests that have failed. These might have:
        - **4xx (Client Errors)**: Such as **404 Not Found** or **400 Bad Request**.
        - **5xx (Server Errors)**: Such as **500 Internal Server** Error or **502 Bad Gateway**.
7. Export the HAR file
    - In the **Network** tab, youâ□□ll see an export button, which looks like a downward arrow icon.
    - Click this button to export the HAR file.
    - Choose a location to save the HAR file on your computer that will be shared.

On this page

Export and Import Pulse applications

Pulse offers application import and export to facilitate interoperability between installations and enable precise application state backups.

Exporting an Applicationâ□□

Applications are exported in a single PAPP file that contains all the application resources like schemas, workflows and so on. In some situations there are security resources that are also exported with the applications; more on that later.

warning

Note that application exports may contain personally-identifiable information (PII) if this data is directly used in any application resources, for example if a rule or a plan map operator checks a specific customer name. Care should be taken to ensure this is not the case before sharing an exported application.

PII is more likely to be found in Pulse lists, and therefore Feedzai always recommends that applications are exported without list items.

Users can easily export an application by selecting the `Export Application` option within the application configuration menu, which initiates the download of the application's PAPP file. Please note, this method of exporting the application will also export the list items stored in the application.

Applications can also be exported using a REST API endpoint. The API exposes a single authenticated endpoint that can be used to export the PAPP file. Authentication details are available in API Authentication.

Calling the endpoint will return a similar PAPP file as if it has downloaded via the UI, but there are additional parameters can be tweaked.

`GET /pulseviews/api/apps/{id}/export?excludeItems={excludeItems}`

Parametersâ□□

Name	Type	Data type	Description
`id`	required	string	The ID of the application to export.
`excludeItems`	optional	boolean	Whether lists should be excluded from the export. Defaults to `false` but it is recommend to set to `true`.

Responsesâ□□

Code	Content-type	Response
`200`	`text/plain`	`200 OK`
`400`	`text/plain`	`400 Bad Request`
`401`	`text/html;charset=iso-8859-1`	`Error 401 Unauthorized`

Example cURLâ□□

```
curl -X GET http://<COMPONENT_URL>/apps/123456789/export?excludeItems=true \
  -H "Cookie: SSOcookie=b7ea8372-2108-46c8-9cae-d16a0c153dfa" \
```

```
  -H "X-Requested-With: XMLHttpRequest" \
  --output test_app.papp
```

Copy

warning

When exporting applications with `excludeItems` set to `false` (the default), the lists are fully loaded into memory, which often results in memory issues. We recommend that application exports are done via API requests with `excludeItems=true`.

Importing an Applicationâ

Importing an application is usually as simple as exporting it. The user only has to click on the `Import Application` button present in the Applications tab and an informative wizard will guide the entire process.

The Import Application wizard can have one to three steps depending on the complexity of the application being imported. If the application has neither security Groups nor Multi-Tenant resources, only one step will be needed which allows the user to upload the application PAPP file.

Applications With Multi-Tenancy Enabledâ

Importing and exporting applications that contain Multi-Tenancy resources do not require any additional configuration steps. However, there are a few considerations worth mentioning regarding the Application import with Multi-Tenancy. Tenants stored in the PAPP file are merged automatically by Pulse using the following algorithm:

1. If the Tenant being imported has the same Unique Id (Key) of a Tenant in the current Pulse Installation, all resources of the Tenant being imported will be added/mapped to the Tenant in the current Pulse Installation.
2. If there is no Tenant with the the same Unique Id in the current Pulse Installation, a new Tenant will be created verifying before hand that there is no naming collision.
    - If there is a name collision, the Tenant Unique Id will be appended at the end of the name, e.g. "Tenant Name (Tenant Unique Id)".
3. All the tenants stored in the PAPP file will be imported to the current Pulse Installation. Even if they are not being explicitly used in the Application being imported. Additional details:
    - Even the Tenants without any explicit resource are used in runtime in case of a Rules of the type "Shared by All Tenants".
    - When Exporting an Application, all the Tenants in the Pulse installation are stored in the PAPP file.

On this page

API Authentication

In this page you can find an outline of the authentication process, necessary parameters, and the required flow to interact with the Pulse APIs, for example, in the context of authenticating a service account before using the Pulse and Case Manager servlets that allow exporting and importing Pulse applications.

Authentication Processâ□□

Pulse and Case Manager APIs require authentication for accessing and performing actions. Specifically, Basic Authentication is used for authenticating service accounts through their credentials (username and password). These credentials need to be encoded in Base64 before being sent in the API requests. The authentication flow is as follows:

- Take note of a Pulse service account's username and password
- Encode the username and password using a Base64 encoder of your choosing
- Send a request to the authentication endpoint with the credentials in the request body
- If the request succeeds, Pulse will return a session cookie that can be used in subsequent API calls, e.g. a request to the logging API

You can find below the authentication diagram:

After receiving a successful response the cookie can be used to make authenticated requests to the Pulse API to, for example, Import and Export Pulse applications using the Pulse API.

API definitionâ□□

In this section you can find the authentication API endpoint definition.

Endpoint URLâ□□

`POST` `/pulseviews/api/sessions` `(authenticates a service account)`

Parametersâ□□

None

Responsesâ□□

Code	Content-type	Response
`200`	`application/json`	`JSON object with user information`
`401`	`text/html;charset=iso-8859-1`	`Error 401 Unauthorized`

Example cURLâ□□

```
curl -X POST http://<PULSE_URL>/pulseviews/api/sessions \
  -H "Content-Type: application/json" \
  -H "X-Requested-With: XMLHttpRequest" \
  -d '{"username": "<base64_username>","password": "base64_password"}' -c -
```

Copy

Example responseâ

```
{"blockedState":"NONE","createdAt":1732196807880,...}
#HttpOnly_localhost FALSE / FALSE 0 SSOcookie cee99ab5-xxxx-xxxx-xxxx-xxx
xxxxxxxxx
```

Copy

On this page

RabbitMQ Segregation for Pulse and Case Manager

This document provides a step-by-step guide for migrating from a single RabbitMQ cluster to two segregated clusters: one dedicated to Pulse and another to Case Manager. The goal is to ensure a transition with no message loss, supporting a zero-downtime deployment.

Migration Overviewâ□□

Migrating RabbitMQ from a single shared cluster to segregated clusters for Pulse and Case Manager requires several steps to ensure zero message loss and minimal downtime. The process includes the following steps:

1. Prepare the new RabbitMQ cluster by creating all the necessary exchanges and queues for Case Manager
2. Enable maintenance mode in Case Manager to ensure no messages are consumed during the process
3. Create a RabbitMQ federation policy to forward messages from the old to the new cluster, preventing data loss and consumption duplication
4. Deploy Case Manager to consume messages from the new cluster
5. Deploy Pulse to publish Case Manager messages to the new cluster
6. Clean up the old Case Manager cluster once migration is complete

The federation step is crucial for a seamless migration, as it ensures that messages published to the old cluster during the transition are not lost and are safely streamed to the new cluster. This is achieved by:

- Setting a one-way policy that replicates messages published to the old RabbitMQ cluster to the new one
- Setting the acknowledge mode to on-confirm, so messages are only consumed a single time

It is expected that during the migration, both clusters will hold messages based on the federation policy above, which should start to be consumed once maintenance mode is disabled.

The diagram below illustrates the expected architecture after the migration:

Migration Stepsâ□□

In this section you can find a description of the migration steps.

Prepare the new RabbitMQ clusterâ□□

The first step is to prepare the new cluster, ensuring it has all the necessary queues/exchanges that Case Manager requires. This can be done in several ways, for instance:

- Creating a temporary Case Manager cluster, configured to use the new cluster, and decommissioning it afterwards
- Creating all the resources manually using the RabbitMQ Management UI
- Using `rabbitmqctl` to list and create all the necessary resources

warning

To avoid message loss it is crucial that all queues/exchanges are created in the new RabbitMQ cluster before enabling the federation policy.

If the first option is used make sure to remove the temporary cluster before creating the federation policy to ensure no messages are consumed.

Enable maintenance mode in Case Managerâ

To ensure that no messages are processed during the migration, maintenance mode should be enabled in Case Manager. By enabling maintenance mode it is guaranteed that messages are only consumed by the new cluster once the migration is concluded.

Set Up Federationâ

Setting up a federation policy to replicate all Case Manager messages to the new cluster guarantees that no messages are lost in the process and that each is only consumed once. The process is described as follows:

- Enable the federation plugin on both clusters:

```
rabbitmq-plugins enable rabbitmq_federation
rabbitmq-plugins enable rabbitmq_federation_management
```

Copy

- On the new cluster, configure a federation upstream policy to connect to the old cluster. The 'ack-mode' is set to 'on-confirm' on the configuration of the federation, ensuring messages are only removed from the old cluster after being safely received and consumed by the new cluster. This establishes a link so the new cluster can pull messages from the old cluster during migration:
  - `<new-vhost>` : the vhost in the new Case Manager RabbitMQ cluster. It can be the same vhost name as in the old cluster
  - `<user>/<pass>` : RabbitMQ credentials with access to the old cluster
  - `<old-cluster>` : hostname or IP of the old cluster
  - `<old-vhost>` : the vhost used by Case Manager on the old cluster

```
rabbitmqctl set_parameter -p <new-vhost> federation-upstream cm-federation '{"uri":
["amqp://<user>:<pass>@<old-cluster>/<old-vhost>"], "ack-mode":"on-confirm"}'
```

Copy

- Apply a federation policy on the new cluster to specify which exchanges should receive messages from the old cluster via federation. Federate only the exchanges used by Case Manager (`CM_UPDATES_CASES` , `CM_UPDATES_EVENTS` , and `EVENT_QUEUE_.*`):

```
rabbitmqctl set_policy -p <new-vhost> --apply-to exchanges cm-federation-policy
'^(CM_UPDATES_CASES|CM_UPDATES_EVENTS|EVENT_QUEUE_.*)$" '{"federation-upstream":"cm-
federation"}'
```

Copy

warning

Ensure that `<new-vhost>` matches the vhost configured via Case Manager Ansible variables for the new cluster. For Pulse (Alert Storage Service), use the vhost set by the corresponding Pulse Ansible variable.

The diagram below illustrates the federation setup between the old and new RabbitMQ clusters. In particular, this shows how the architecture will look when both Pulse and Case Manager are still connected to the old RabbitMQ clusters, but after the new RabbitMQ cluster has been created and the federation link has been set up

Deploy Pulse and Case Managerâ

Following the federation setup, both Pulse and Case Manager should be deployed. The deployment is required so Pulse publishes messages to the new RabbitMQ cluster and Case Manager consumes messages from the new cluster.

To achieve this, the Ansible variables file should be updated for both components:

- Update configuration variables:
  - `rabbitmq_casemanager_brokers` : set to the new RabbitMQ cluster brokers
  - `rabbitmq_casemanager_vhost` : set to the new RabbitMQ cluster vhost

- Deploy Case Manager to the new RabbitMQ cluster. It will start consuming messages from the new cluster and pulling any remaining messages from the old cluster via federation
    - Before disabling maintenance mode on the new Case Manager deployment, stop/decommission the old Case Manager cluster. This ensures only the new cluster is active and prevents message consumption conflicts
- Deploy Pulse which will start publishing messages to the new cluster
- With federation configured using `ack-mode: on-confirm`, the federation link acts as the consumer for the old cluster, safely ensuring messages are only removed from the old cluster after being safely received and consumed by the new cluster

Please refer to Case Manager Configuration Reference and Pulse Configuration Reference for more information.

Cutover and cleanâ□□

warning

Make sure all Case Manager queues in the old RabbitMQ cluster are drained before proceeding with the cleanup.

Once both components are deployed and using the correct RabbitMQ cluster, any leftover configurations/resources can be safely removed:

- Remove federation policies and upstreams from the new cluster:

```
rabbitmqctl clear_policy -p <new-vhost> cm-federation-policy
rabbitmqctl clear_parameter -p <new-vhost> federation-upstream cm-federation
```

Copy

- Clean the old cluster Case Manager exchanges and queues:

```
rabbitmqctl delete_exchange -p <old-vhost> CM_UPDATES_CASES
rabbitmqctl delete_exchange -p <old-vhost> CM_UPDATES_EVENTS
rabbitmqctl delete_queue -p <old-vhost> EVENT_QUEUE_.*
```

Copy

Alerting in Case Manager

In this page you can find a set of alerts that can be added to the monitoring solution in order to improve awareness over Case Manager stability.

info

Note that these alerts are solemnly suggestions that might require new dashboards or alert adjustments based on HSBC specific environments.

Alert	Configuration	Description
New Event Throughput	alert level: Critical for alert: 1m frequency: 2h threshold value: 0.0000003 (messages per second)	Monitors the throughput of event processing in Case Manager. If throughput falls below the threshold, an alert is triggered.
New Event Latency	alert level: High for alert: 10m frequency: 10m threshold value: 60 (seconds)	Monitors the maximum latency for processing "new event" requests, specifically at the 99th percentile.
Queue Occupancy	alert level: High for alert: 10m frequency: 10m threshold value: 0.85 (percentage)	Monitors the average occupancy of the event processing queue. If the average occupancy exceeds 85% over the specified time span, an alert is triggered.
Event Queue Deadletter	alert level: High for alert: 5m frequency: 1h threshold value: 10 (events)	Monitors the total number of event queue deadletter events. If the sum of deadletters exceeds the defined threshold of 10 over the last hour, an alert is triggered.
Event Storage Deadletter	alert level: High for alert: 5m frequency: 1h threshold value: 10 (events)	Monitors the total number of event storage deadletter events. If the sum of deadletters exceeds the defined threshold of 10 over the last hour, an alert is triggered.
Event Automations Throughput	alert level: Moderate for alert: 10m frequency: 10m threshold value: 0 (events)	Monitors the overall throughput count when executing automation rules. If errors exceed the defined threshold of 0 over the last 10 minutes, an alert is triggered.
Event Automations Errors	alert level: Moderate for alert: 10m frequency: 10m threshold value: 0 (events)	Monitors the overall error count when executing automation rules. If errors exceed the defined threshold of 0 over the last 10 minutes, an alert is triggered.
Pulse Requests Errors	alert level: Moderate for alert: 10m frequency: 1m threshold value: 0.01 (percentage)	Monitors the throughput of requests to Pulse instances and tracks the occurrence of errors.
Database Connection Pool	alert level: Low for alert: 10m frequency: 5m threshold value: 0.90 (percentage)	Monitors the usage of the database connection pool, ensuring it does not exceed the defined threshold.

Alert	Configuration	Description
Messaging Healthcheck	alert level: Critical for alert: 5m frequency: 1m threshold value: 1 (event)	Monitors the messaging server healthcheck from Case Manager.
Database Connection	alert level: Critical for alert: 5m frequency: 1m threshold value: 1 (event)	Monitors the database healthcheck for Case Manager.

note

- alert level : The severity of the alert (Low, Moderate, High, Critical).
- for alert : The duration for which the alert condition must persist before an alert is triggered.
- frequency : The interval at which the monitoring system checks the specified metrics or conditions.
- threshold value : A predefined limit or value that, when exceeded, triggers an alert.

Alerting in Pulse

In this page you can find a set of alerts that can be added to the monitoring solution in order to improve awareness over Pulse stability.

info

Note that these alerts are solemnly suggestions that might require new dashboards or alert adjustments based on HSBC specific environments.

Alert	Configuration	Description
Workflow Throughput	alert Level: Critical for alert: 1m frequency: 1m threshold value: 0.00001 (messages per second)	Measures the overall application workflow throughput per message type(normal, recovery, replica).
Workflow Errors	alert Level: High for alert: 10m frequency: 10s threshold value: 0 (events)	Measures the overall application workflow errors, considering all message types(normal, recovery and replica).
Events Dumped to Disk	alert Level: Critical for alert: 1m frequency: 1m threshold value: 0 (events)	Measures the overall application events spilled to disk. An event is spilled to disk when Pulse cannot write it on the event storage.
Auto Recovery Job	alert Level: Moderate for alert: 0s frequency: 5m threshold value: 0 (events)	Measures the overall number of events that are present in the in-memory queue to be written to the dump file in the failed folder, application and workflow.
Profile Fetches Latencies	alert Level: High for alert: 10m frequency: 1m threshold value: 0.4 (seconds)	Measures the overall max latency to fetch the profiles at percentile 99, per application, workflow and plan.
Pulse Application States	alert Level: Critical for alert: 12h frequency: 30s threshold value: 6 (STARTED)	Monitors the state of applications across all Pulse instances. If the application state is not STARTED (state < 6), an alert is triggered.
Pulse JVM GC Time	alert Level: High for alert: 10m frequency: 5m threshold value: 1 (seconds)	Measures maximum JVM GC duration. Long garbage collection pause durations indicate that the heap size is not adjusted for the actual usage.
App JVM GC Time	alert Level: High for alert: 10m frequency: 15m threshold value: 1 (seconds)	Measures maximum JVM GC duration for young and old gen per application and host, indicating heap size adjustment needs.
Pulse JVM Heap Usage	alert Level: Critical for alert: 5m frequency: 1m threshold value: 0.95 (percentage)	Measures JVM heap usage per Pulse instance, indicating high memory utilization.

Alert	Configuration	Description
App JVM Heap Usage	alert Level: Critical for alert: 5m frequency: 1m threshold value: 0.99 (percentage)	Measures JVM heap usage per application and host, indicating high memory utilization.
Pulse Threads Count	alert Level: Moderate for alert: 30m frequency: 10s threshold value: 2500 (threads)	Measures the current number of active threads in the JVM per Pulse instance.
App Threads Count	alert Level: Moderate for alert: 30m frequency: 10s threshold value: 3000 (threads)	Measures the current number of active threads in the JVM per application and host.
Pulse Heap Usage After GC	alert Level: High for alert: 15m frequency: 10s threshold value: 0.90 (percentage)	Measures the JVM heap usage after garbage collection per Pulse instance.
App Heap Usage After GC	alert Level: High for alert: 15m frequency: 10s threshold value: 0.9 (percentage)	Measures the JVM heap usage after garbage collection per application and host.
Pulse JVM File Descriptors Ratio	alert Level: High for alert: 10m frequency: 10s threshold value: 0.85 (percentage)	Measures the JVM file descriptors usage ratio per Pulse instance.
App JVM File Descriptors Ratio	alert Level: High for alert: 10m frequency: 10s threshold value: 0.85 (percentage)	Measures the JVM file descriptors usage ratio per application and host.
Pulse JVM Code Cache Usage	alert Level: Critical for alert: 10m frequency: 10s threshold value: 0.90 (percentage)	Measures the JVM code cache usage per Pulse instance.
App JVM Code Cache Usage	alert Level: Critical for alert: 10m frequency: 10s threshold value: 0.95 (percentage)	Measures the JVM code cache usage per application and host.
Messaging Healthcheck	alert Level: High for alert: 5m frequency: 2m threshold value: 1 (event)	Monitors the Pulse healthcheck for messaging communication with the messaging server.
Zookeeper Healthcheck	alert Level: High for alert: 5m frequency: 2m threshold value: 1 (event)	Monitors the Pulse healthcheck for Zookeeper communication.
Metadata DB Connection Pool	alert Level: Low for alert: 30m frequency: 1m	Monitors the metadata DB connection pool usage.

Alert	Configuration	Description
	threshold value: 0.9 (percentage)	
Metadata DB Connection Pool Per App	alert Level: Low for alert: 30m frequency: 1m threshold value: 0.9 (percentage)	Monitors the metadata connection pools usage per application.

note

- alert Level : The severity of the alert (Low, Moderate, High, Critical).
- for alert : The duration for which the alert condition must persist before an alert is triggered.
- frequency : The interval at which the monitoring system checks the specified metrics or conditions.
- threshold value : A predefined limit or value that, when exceeded, triggers an alert.

On this page

Collect Metrics

Feedzai products and components expose monitoring metrics on a per request fashion. They are Dropwizard metrics formatted in JSON and available on the following HTTP endpoints. The metrics endpoint should be used for consumption purposes, typically by a third-party tool such as Grafana or Prometheus. A companion *"rich metrics"* endpoint is also available for some components, which includes descriptions for the metrics to help users understand their purpose. Refer to the following endpoints per component:

- Pulse:
  - `https://<host:port>/metrics`
  - `https://<host:port>/metrics/rich`
- Case Manager:
  - `https://<host:port>/api/metrics`
  - `https://<host:port>/api/metrics/rich`
- Reference Data Service:
  - `https://<host:management_port>/metrics` (note the use of the management port, typically 8081)
  - `https://<host:management_port>/metrics/rich` (note the use of the management port, typically 8081)

info

Note that these endpoints donâ□□t require authentication.

warning

All metrics are published to a single node. It's the operator's responsibility to use an external monitoring system to aggregate them afterwards.

Next stepsâ□□

Learn how to interpret the available metrics, and how to use the `/metrics/rich` endpoint to understand the meaning of each metric.

On this page

Case Manager Database Table Structure

Find in this page a detailed guide on the important tables used for Case Manager.

AM_EVENT_STORAGEâ□□

- This table contains the referential records to all historical transactions, where the event request is stored in EVENT_DATA in raw json format. This includes all events for a given lifecycle_id, not just a single record per lifecycle_id.

Column Name	Column Type	Description
EVENT_ID	PRIMARY KEY	The event's unique identifier. Randomly generated UUID, generated for each event.
EVENT_VERSION_ID	PRIMARY KEY	It is usually equal to EVENT_ID. New Id gets generated in case the event is an update to the initial transaction.
EVENT_DATA	JSON	The field stores event data

AM_EVENT_DIGITAL_ACTIVITYâ□□

- The records in this table correspond to the Digital Activity events (e.g. Credentials Update, Device Update, New Payee, Alias, Profile Update, etc).
- The AM_EVENT_DIGITAL_ACTIVITY table only holds one record for all events with the same lifecycle_id. Subsequent events for the same lifecycle_id will update the record as opposed to creating a new record.
- Use this extract to find all the unique Digital Activity transactions uploaded using Digital Activity API

Column Name	Column Type	Description
EVENT_ID	PRIMARY KEY	Internal field. The event's unique identifier. Randomly generated UUID, equal to the corresponding FDZ_UUID in the Pulse's event storage.
EVENT_ALERT	BOOLEAN	Boolean for whether it is an alert or not.
EVENT_TENANT	STRING	Internal field which stores the tenant key.
QUEUE_ID	STRING	The ID of the current queue.
USER_ID	STRING	The ID of the current assignee.
ACCESS_GROUP_ID	STRING	The ID of the access group.
channelId	STRING	Channel identifier.
event_type	STRING	An internal field that indicates the endpoint from which the event entered.
fraud_score	STRING	Calculated fraud score.
outcome_decision	STRING	Event decision outcome.
amount	INTEGER	Transaction amount performed by the sender, in the currency specified by the `currency` field.
channel_type	STRING	Specifies the transaction payment channel.
currency_code	STRING	The local currency of the business.
customer_dob	STRING	Customer date of birth (YYYYMMDD).
customer_email	STRING	Customer's email.

Column Name	Column Type	Description
customer_id	STRING	Customer ID generated by the issuing bank.
customer_name	STRING	Customer's name.
customer_phone_number	STRING	Customer phone number.

AM_EVENT_TRANSFERSâ□□

- The records in this table correspond to the Transfer events (e.g. Initiation, Settlement, Info, Feedback, etc).
- The AM_EVENT_TRANSFERS database table only holds one record for all events with the same lifecycle_id. Subsequent events for the same lifecycle_id will update the record as opposed to creating a new record.
- Use this extract to find all the unique transfers uploaded using Transfers API

Column Name	Column Type	Description
EVENT_ID	PRIMARY KEY	Internal field. The event's unique identifier. Randomly generated UUID, equal to the corresponding FDZ_UUID in the Pulse's event storage.
EVENT_ALERT	BOOLEAN	Boolean for whether it is an alert or not.
EVENT_TENANT	STRING	Internal field which stores the tenant key.
QUEUE_ID	STRING	The ID of the current queue.
USER_ID	STRING	The ID of the current assignee.
ACCESS_GROUP_ID	STRING	The ID of the access group.
channelId	STRING	Channel identifier.
event_type	STRING	An internal field that indicates the endpoint from which the event entered.
fraud_score	STRING	Calculated fraud score.
outcome_decision	STRING	Event decision outcome.
merchant_city	STRING	Card acceptor city.
dispute_end_date	STRING	Timestamp corresponding to the moment when the dispute was closed.
dispute_feedback_notes	STRING	Contextual information associated with the dispute communication.
dispute_refunded	STRING	Whether the funds were refunded to the cardholder.
oob_feedback_notes	STRING	Contextual information associated with the out-of-band communication.
feedback_type	STRING	Identifies the different types of feedback.
dispute_notes	STRING	Contextual information left by the analyst at the time of the dispute opening.
dispute_reason	STRING	Why the dispute was opened.
dispute_start_date	STRING	Timestamp that corresponds to the time when the dispute was opened.
dispute_submitted_by	STRING	Who submitted the dispute.

AM_EVENT_STATUSâ□□

- Holds the history on status changes for events - previous state, new state, when the state change happened and who did it.

Column Name	Column Type	Description
EVENT_STATUS_ID	PRIMARY KEY	Internal field, randomly generated identifier for each status change.
OLD_STATUS_ID	STRING	The ID of the old status.
NEW_STATUS_ID	STRING	The ID of the new status.

Column Name	Column Type	Description
TIMESTAMP	INTEGER	Unix timestamp at which the event note was added.
USER_ID	STRING	ID of the user that added the note.

AM_EVENT_EXPLANATIONâ□□

- This table contains the explanations of why the event triggered an alert, that is the Rules that triggered the alert. This contains the rules explanation for events in AM_EVENT table.

Column Name	Column Type	Description
EVENT_ID	STRING	The event's unique identifier. Randomly generated UUID, equal to the corresponding FDZ_UUID in the Pulse's event storage.
EVENT_TIMESTAMP	INTEGER	Corresponds to the FDZ_timestamp field in the Pulse's event storage for this particular EVENT_ID.
EVENT_VERSION_ID	STRING	It is equal to EVENT_ID.
EVENT_VERSION_TIMESTAMP	INTEGER	It is equal to EVENT_TIMESTAMP.
EVENT_EXPLANATION_GENERATOR_ID	STRING	Contains the internal rule ID. The primary key of this table is composed of the event ID and the rule ID.
EVENT_EXPLANATION_GEN_NAME	STRING	Contains the rule name.
EVENT_EXPLANATION_TYPE	STRING	The type of trigger that caused the alert.
EVENT_EXPLANATION_DESCRIPTION	STRING	The explanation provided by the risk engine for triggering the rule.

AM_EVENT_NOTEâ□□

- Notes added by analysts that may include personal information. The key is the EVENT_NOTE_ID column.

Column Name	Column Type	Description
EVENT_NOTE_ID	PRIMARY KEY	Randomly generated identifier for each note manually added by the analysts.
NOTE_MESSAGE	STRING	Note added by the analyst.
USER_ID	STRING	ID of the analyst that added the note.

AM_EVENT_STATUS_REASONâ□□

- This will provide the reason why the status of an event was changed as selected in the UI.
- The table is related to the AM_STATUS_REASON table via the column STATUS_REASON_ID as the foreign key. The possible values for STATUS_REASON_ID is a small set of mostly static values that correspond to a few options in a drop-down menu that a Case Manager user can choose from in the UI.

Column Name	Column Type	Description
EVENT_STATUS_ID	PRIMARY KEY	ID of the status associated with this reason change. Maps to the EVENT_STATUS_ID field in AM_EVENT_STATUS.
STATUS_REASON_ID	FOREIGN KEY	Foreign key to AM_STATUS_REASON table.
TIMESTAMP	PRIMARY KEY	Unix timestamp at which the event note was added.

AM_SLAâ□□

- The table stores the SLA definition or metadata that is created to automatically perform actions on events.

Column Name	Column Type	Description
SLA_ID	PRIMARY KEY	ID of the SLA.
SLA_NAME	PRIMARY KEY	Name of the SLA.
SLA_DESCRIPTION	STRING	Description of the SLA.
SLA_GOAL	STRING	Goal of the SLA.
SLA_CONDITIONS	STRING	Conditions of the SLA.
SLA_ACTIONS	STRING	SLA actions.

AM_EVENT_SLAâ□□

- The table links specific events to SLA's.

Column Name	Column Type	Description
EVENT_ID	PRIMARY KEY	Unique event id.
EVENT_SLA_ID	PRIMARY KEY	Event sla unique id.
SLA_ID	FOREIGN KEY	Foreign key to AM_SLA table.

AM_AUTOMATIONâ□□

- The table stores the Automation rules created to automatically perform actions events and cases based on conditions.

Column Name	Column Type	Description
AUTOMATION_ID	PRIMARY KEY	ID of the service level agreement.
AUTOMATION_POSITION	UNIQUE KEY	Numeric position.
AUTOMATION_NAME	STRING	Name of Automation.
AUTOMATION_DESCRIPTION	STRING	Automation description.
AUTOMATION_CONDITIONS	STRING	Automation conditions.
AUTOMATION_ACTIONS	STRING	Automation actions.

`AM_CASE_*` â□□

note

`AM_CASE_*` - They are not used by HSBC

Dead Letters System

info

Although described in conjunction, Pulse dead letter system runs independently from Case Manager

When Pulse and Case Manager receive events but are unable to persist them, for instance due to a connection error with the database, a recovery system is used to avoid losing important information.

This recovery system consists in storing failed events in Pulse and CM filesystems (dead letters) and a recovery job that parses these files and retries persisting them to the database.

Dead letters are stored in a `retry` folder under `<installation-folder>/var/dead-letters`. When the recovery job triggers, it reads the existing dead letters and tries to persist the events to the database. On success the dead letter is deleted from the filesystem otherwise they are kept in the filesystem until the recovery job next execution (by default 1 hour).

The recovery job maximum retries is by default 3 meaning that there is a 3 hour window to identify and resolve the database issue. If this threshold is reached, Pulse and CM move the dead letters to a failed folder under `<installation-folder>/var/dead-letters`.

Available dead letters metrics for Case Manager are described in Event Queue Deadletter and for Pulse in Events Spilled to Disk.

Monitoring the Infrastructure

info

Note that the following list is simply a reference to the most important metrics, and should not be considered as the single source of monitoring the healthiness of the system.

While monitoring the application is important, it is not possible to have a useful monitoring system if the infrastructure and the operating system are not monitored as well. There are many metrics that could be listed here, but the truth is that some are more important than others, and that list may vary with the system. For example, systems that are prone to spill data to disk should be provisioned with more storage, and disk usage monitoring is an absolute must, as if the disk runs out of space, we may end up with corrupted data and a crashed service.

These are the metrics that should absolutely be monitored for all VMs or containers:

- CPU stats (user, system, iowait & idle percentages)
- Memory usage (used, buffered, cached & free percentages)
- Virtual Memory statistics (dirty page flushes, writeback volume)
- Disk I/O (operations & amount of data transferred per unit time, time to service operations)
- Free disk space on the mount used for data directory
- File descriptors used by `beam.smp` process vs. max system limit
- Network throughput (bytes received, bytes sent) & maximum network throughput
- Network latency (particularly between replicas from the same service)

Case Manager

In this section you will find the key metrics to monitor, a detailed guide to the important tables used and the relevant alerts in Case Manager.

In this guide you will find:

- Which metrics are especially relevant for Case Manager.
- Which tables are important in Case Manager.
- Which alerts are especially relevant for Case Manager.

On this page

Monitoring Cassandra

tip

For a complete reference of the metrics provided by Cassandra, refer to the official documentation. Make sure to refer to the version being used.

Collecting metricsâ□□

Metrics in Cassandra are managed using the Dropwizard Metrics library. The recommendation from the Cassandra maintainers is to query the metrics via JMX.

Feedzai recommends the usage of Jolokia, a JMX-HTTP bridge plugin that makes JMX-based systems more suitable to agent-based solutions. This allows agents like Telegraf to easily poll the metrics from individual nodes and send them to a centralised metric storage.

note

The recommendation from Telegraf for monitoring Cassandra is also to use a Jolokia plugin.

Top Metrics to Monitorâ□□

In this section you will find which metrics are important to monitor in **Cassandra**.

info

Note that the following list is simply a reference to the most important metrics, and should not be considered as the single source of monitoring the healthiness of the system.

Table Operation Latencyâ□□

The metric names are all appended with the specific `Keyspace` and `Table` name: `org.apache.cassandra.metrics.Table.<MetricName>.<Keyspace>.<Table>`

Metric Name	Description
`ReadLatency`	Local read latency for this table.
`WriteLatency`	Local write latency for this table.
`PendingFlushes`	Estimated number of flush tasks pending for this table.
`PendingCompactions`	Estimate of number of pending compactions for this table.
`TotalDiskSpaceUsed`	Total disk space used by SSTables belonging to this table, including obsolete ones waiting to be GCâ□□d.

tip

Table-specific metrics should be monitored for all the relevant tables in the system. In particular, the profiles table maintained for Pulse (i.e. the Profile Store) and the entity table maintained by the Reference Data Service.

Commit Logâ□□

Metrics specific to the `CommitLog` : `org.apache.cassandra.metrics.CommitLog.<MetricName>` .

Metric Name	Description
`CompletedTasks`	Total number of commit log messages written since [re]start.
`PendingTasks`	Number of commit log messages written but yet to be fsyncâ□□d.
`TotalCommitLogSize`	Current size, in bytes, used by all the commit log segments.
`WaitingOnCommit`	The time spent waiting on CL fsync; for Periodic this is only occurs when the sync is lagging its sync interval.

Compactionâ□□

Metrics specific to the `Compaction` : `org.apache.cassandra.metrics.Compaction.<MetricName>` .

Metric Name	Description
`PendingTasks`	Estimated number of compactions remaining to perform.
`CompletedTasks`	Number of completed compactions since server [re]start.
`TotalCompactionsCompleted`	Throughput of completed compactions since server [re]start.

note

The pending compactions per table are also reported under `org.apache.cassandra.metrics.Table.<MetricName>.<Keyspace>.<Table>.PendingCompactions`

JVM metricsâ□□

On this page

Monitoring in GCP

info

Note that this section is simply a reference to the monitoring service from Google Cloud Platform, and is not a guide on how to implement a monitoring solution leveraging that service. For more details about that refer to the official documentation.

Operations Suite (formerly Stackdriver)â□□

When designing a new system, one of the parts you will need to consider is your monitoring solution. That can be a daunting task with all the decisions that need to be taken technology and architecture-wise. However, if you are on Google Cloud Platform, the Operations Suite service provides an integrated option for monitoring and logging for any system running on GCP.

Ops Agentâ□□

There are some metrics that it is impossible for applications to report as they often only have visibility about their process. Likewise, there are metrics that the hypervisor managing the instances can report, such as the memory allocated to a particular instance, but it cannot report which portion of that memory is actually being used. The typical approach to address such observability limitations is to install another process in the same virtual machine, such as an *agent*. This is why Google created the Ops Agent.

The Ops Agent is the primary agent for collecting logs and metrics from individual VMs. This agent uses Fluent Bit and OpenTelemetry Collector for collecting logs and metrics, respectively. In addition to collecting Google Compute Engine metrics, this agent can also be configured for third-party applications, such as ZooKeeper and Cassandra.

tip

If Ops Agent is not compatible with all the systems you want to monitor, a great alternative is Telegraf, one of the best server-based agents for collecting metrics, and can be configured with a Google Cloud Monitoring output plugin.

Pulse

In this section you will find the key metrics to monitor and the relevant alerts in Pulse.

In this guide you will find:

- Which metrics are especially relevant for Pulse.
- Which alerts are especially relevant for Pulse.

On this page

Monitoring RabbitMQ

tip

For a complete reference of the monitoring recommendations for RabbitMQ, refer to the official documentation. Make sure to refer to the version being used.

info

The details in this page may change depending on how you have installing your RabbitMQ cluster. For example, if you are using Kubernetes, you should be using the RabbitMQ Kubernetes Operator, and different recommendations apply.

Collecting metricsâ□□

RabbitMQ exposes several application-specific metrics that are collected and reported by RabbitMQ nodes. Some of these are cluster-wide, and others are node-specific. It is important to consider this when collecting and storing them.

The recommended monitoring approach from RabbitMQ is using the Prometheus plugin. Feedzai has also used the management plugin to serve metrics, and also allowing the cluster to be managed via a UI or an API. When activated, the management plugin provides an HTTP API.

warning

There are some performance concerns with the management plugin, which is why the recommendation is to use the Prometheus plugin. If you just wish to monitor cluster-wide metrics (see below), having the plugin enabled on a single node would suffice. However, for redundancy and node-specific metrics, all nodes in the cluster should have it enabled.

note

Telegraf provides an input plugin that relies on the RabbitMQ Management Plugin.

Top Metrics to Monitorâ□□

In this section you will find which metrics are important to monitor in **RabbitMQ**.

info

Note that the following list is simply a reference to the most important metrics, and should not be considered as the single source of monitoring the healthiness of the system.

Cluster-Wide Metricsâ□□

Cluster-wide metrics provide a high level view of cluster state, including the interaction between cluster nodes. To access these metrics, use the endpoint `GET /api/overview` from the HTTP API

Metric Name	Description
`object_totals.connections`	Total number of connections
`object_totals.channels`	Total number of channels
`object_totals.queues`	Total number of queues

Metric Name	Description
`object_totals.consumers`	Total number of consumers
`queue_totals.messages_ready`	Number of messages ready for delivery
`queue_totals.messages_unacknowledged`	Number of unacknowledged messages
`message_stats.publish_details.rate`	Message delivery rate
`message_stats.deliver_get_details.rate`	Message publish rate

Consider capturing message metrics individually for the most important queues with the `GET /api/queues/{vhost}/{qname}` endpoint.

Node Metricsâ□□

Most of the metrics represent point-in-time absolute values of individual nodes. To access these metrics, use the endpoint `GET /api/nodes/{node}` from the HTTP API (or `GET /api/nodes` if your collector supports an array).

Metric Name	Description
`proc_total`	Erlang process limit
`proc_used`	Erlang processes used
`mem_limit`	Memory usage high watermark
`mem_used`	Total amount of memory used
`queue_totals.messages_ready`	Number of messages ready for delivery
`queue_totals.messages_unacknowledged`	Number of unacknowledged messages

On this page

RabbitMQ Queues

This section presents the messaging topics used in Pulse and Case Manager.

This section describes each topic/queue in detail. Each topic/queue is presented taking in account its general purpose and a list of consumers and producers. Each topic may have one or more queues and each queue one or more consumers.

Pulse Queuesâ□□

This section describes the topic/queue Pulse uses.

Note: topics are identified by an `*` in front on their name. They can have more than one queue associated with them which is identified by the routing key.

Topic/Queue Name	Description	Consumers	Producers	Notes
fdz-sq-digital-activity-non-scoring-*	When a new non-scoring digital event (e.g. async event) is received by Pulse.	Pulse	Pulse	
fdz-sq-transfers-non-scoring-*	When a new non-scoring transfer event (e.g. async event) is received by Pulse.	Pulse	Pulse	
fdz-sq-hsbc-payments-feedback-listenerhsbc-payments-feedback-listener	When a new feedback is received by Pulse.	Pulse	Pulse	
fdz-sq-global-feedbacks-notification-Feedback-Notifier-Migrator	When a feedback change is received by Pulse.	Pulse	Pulse	
fdz-sq-global-feedbacks-queue-appId-<app_id>	When a new Feedback event is sent to the Feedback Storage Service.	Pulse	Pulse	If this queue piles up over time, feedback updates to the Pulse event storage table will not be written (the events in Pulse will be missing the fraud feedback labels), which can have an impact on SMJs depending on the metrics defined, and fraud labels coming from feedbacks will not arrive in Case Manager, leading to missing fraud feedback labels.
fdz-sq-global-fdz-sync-topic-*	Used for workflow replicas to be notified of normal events arriving the original workflow dead letter.	Pulse	Pulse	The number of messages in this topic/queue may increase substantially during an App Publish since the node being published does not process replica messages sent by the other instances. When the App Publish finishes the instance should be able to process the pending messages.
		Pulse	Pulse	

Topic/Queue Name	Description	Consumers	Producers	Notes
fdz-sq-global-fdz-app-*	This is a topic per application, shared by all replicas of that application. It has two main uses: - Communication between the Pulse Server - And the App Engines for publishing / stopping apps.			
fdz-sq-global-fdz-apps-service-app-service-*	This topic is used for two purposes: - Pulse Server leader to communicate application life cycle requests to followers. For example, after creating an app through the web API (on the Pulse Server leader), the app should be bootstrapped in all pulse servers; in order to achieve this, the pulse server leader publishes a Bootstrap App message for the other Pulse servers to know that they must bootstrap the app. - To broadcast changes in model object values. Currently this is used only by the Managed List Service, where changes on list items made on the GUI take effect immediately, with no need to re-publish the app.	Pulse	Pulse	
fdz-sq-global-fdz-dc-heartbeat-*	Receive heartbeat messages sent from other cluster monitor implementations and the self Data Center Heartbeat. Also, listen for Data Center Heartbeat Request messages, requesting each data center to report that it is alive.	Pulse	Pulse	
fdz-sq-global-fdz-metadata-sync-job-manager-*	For App Engines to be notified of request to update a job metadata.	Pulse	Pulse	
fdz-sq-global-fdz-metadata-sync-member-*	For App Engines to be notified of requests to reload the Configuration.	Pulse	Pulse	
fdz-sq-global-fdz-metadata-sync-pulse-config-manager-*	For Pulse Servers to be notified of requests to reload the Configuration.	Pulse	Pulse	
fdz-sq-global-fdz-metadata-sync-pulse-security-*	For Pulse Servers to be notified of requests to reload security metadata.	Pulse	Pulse	
fdz-sq-global-jwk-service-updates-*	When a JWK is created or revoked, the leader node sends an update for other replicas	Pulse	Pulse	
fdz-sq-fdz-cluster-dc1-dc-manager-*	This is used to broadcast to all the cluster members of the local data center that a given data center became primary.	Pulse	Pulse	
fdz-sq-fdz-data-science-jobs-<app_id>*	This is used to keep data science job status updated between Pulse instances	Pulse	Pulse	
fdz-sq-fdz-force-flush-topic-*	Keep workflow clocks synchronized between replicas.	Pulse	Pulse	
fdz-sq-fdz-processes-*	This topic is used for communications between external jobs (e.g. SMJs running	Pulse/ External Job		

Topic/Queue Name	Description	Consumers	Producers	Notes
	on Dataproc) and their launcher, the job manager service.		Pulse/ External Job	
fdz-sq-fdz-pulse-mgmt*	This is used by the pulse-mgmt tool to broadcast requests for disabling and shutting down a data center to all members of the current data center, in fail overs.	Pulse	Pulse	
fdz-sq-fdz-workflow-publish-*	This is used to synchronize the workflow publish state between Pulse instances	Pulse	Pulse	

Case Manager Queuesâ□□

This section describes each topic/queue Case Manager uses.

Note: Since cases are not being used all queues regarding cases in the following table are merged in `CM_UPDATES_CASES_*`.

Topic/Queue Name	Description	Consumers	Producers	Notes
CM_UPDATES_CASES_*	Cases Queues. **Note:** These queues are not in use	Logstash	Case Manager	
CM_UPDATES_EVENTS-EVENT-external-notifications-queue	When a new event (or an event update) arrives in CM.	Logstash	Case Manager	If no consumer is supposed to connect to this queue, it should be pruned occasionally so messages do not pile up.
CM_UPDATES_EVENTS-QUEUE-external-notifications-queue	An event's queue changes.	Logstash	Case Manager	If no consumer is supposed to connect to this queue, it should be pruned occasionally so messages do not pile up.
CM_UPDATES_EVENTS-STATUS-external-notifications-queue	When the state of an event (on any state machine) changes.	Logstash	Case Manager	If no consumer is supposed to connect to this queue, it should be pruned occasionally so messages do not pile up.
CM_UPDATES_EVENTS-USER-external-notifications-queue	The user assigned to an event has changed.	Logstash	Case Manager	If no consumer is supposed to connect to this queue, it should be pruned occasionally so messages do not pile up.
CM_UPDATES_EVENTS-RELATED_TRANSACTIONS-external-notifications-queue	When related transactions arrive to CM.	Logstash	Case Manager	If no consumer is supposed to connect to this queue, it should be pruned occasionally so messages do not pile up.
CM_UPDATES_EVENTS-NOTES-external-notifications-queue	When a new note is added to event	Logstash	Case Manager	If no consumer is supposed to connect to this queue, it should be pruned occasionally so messages do not pile up.
fdz-sq-EVENT_QUEUE_CARDS	When a new Cards event arrives to CM.	Case Manager	Pulse	In case this queue starts to pile up it means CM is not able to process events at a higher rate than they are being produced. This is not critical since the consequence is that events will show in CM later than when they were inserted in the queue. However, if events keep piling up in the queue for longer periods of time, this is a sign that something is wrong in the

Topic/Queue Name	Description	Consumers	Producers	Notes
				system, possibly in Case Manager or the database.
fdz-sq-EVENT_QUEUE_NON-MON	When a new Non-Mon event arrives to CM.	Case Manager	Pulse	In case this queue starts to pile up it means CM is not able to process events at a higher rate than they are being produced. This is not critical since the consequence is that events will show in CM later than when they were inserted in the queue. However, if events keep piling up in the queue for longer periods of time, this is a sign that something is wrong in the system, possibly in Case Manager or the database.
fdz-sq-EVENT_QUEUE_DIGITAL-ACTIVITY	When a new Digital Activity event arrives to CM.	Case Manager	Pulse	In case this queue starts to pile up it means CM is not able to process events at a higher rate than they are being produced. This is not critical since the consequence is that events will show in CM later than when they were inserted in the queue. However, if events keep piling up in the queue for longer periods of time, this is a sign that something is wrong in the system, possibly in Case Manager or the database.
fdz-sq-EVENT_QUEUE_TRANSFERS	When a new Transfers event arrives to CM.	Case Manager	Pulse	In case this queue starts to pile up it means CM is not able to process events at a higher rate than they are being produced. This is not critical since the consequence is that events will show in CM later than when they were inserted in the queue. However, if events keep piling up in the queue for longer periods of time, this is a sign that something is wrong in the system, possibly in Case Manager or the database.
fdz-sq-EVENT_QUEUE_65b71a29808f2220	When a new Inbound Payments event arrives to CM.	Case Manager	Pulse	In case this queue starts to pile up it means CM is not able to process events at a higher rate than they are being produced. This is not critical since the consequence is that events will show in CM later than when they were inserted in the queue. However, if events keep piling up in the queue for longer periods of time, this is a sign that something is wrong in the system, possibly in Case Manager or the database.

On this page

Building a Monitoring Solution

Monitoring has become one of the most critical aspects of operating and maintaining a reliable system. There are numerous technologies that can be leveraged to implement real-time application and infrastructure monitoring according to the industry best practises. Different tools are more appropriate for different architectures, so *you must decide what fits best into your exising system*.

Regardless of the technologies you use or plan to adopt, make sure your monitoring solution covers the following aspects:

- **Real-time feedback**: Regardless of whether you have push or pull-based architecture, knowing the state of your system in near real-time is critical to respond to incidents, or even preempt them.
- **Application state**: The applications are becoming progressively more complex, and being able to observe how the state of the applications change over time is key to pinpoint other issues in the system. For example, in the application system that implements quorum-based consistency (e.g. ZooKeeper), it is important to track the quorum state, as it can have adverse consequences on systems that depend on it.
- **Performance**: With systems growing in size and complexity, the performance of a healthy system should be considered - i.e. how much usable work can be accomplished by the system within a given period. If it varies a lot for a given system component, or degrades over time, that effect can propagate to other components as well, and impact the whole system. For that reason, it is really important that you track the latency of operations that sit in the critical path of your workload.
- **Availability**: This is the most crucial metric to monitor - whether a system or component is accessible and operate the way it is intended to.
- **Resource usage**: On specific architectures and workloads, the performance and availability of your system is closely coupled with the resources that are made available to the system to perform its task. For that reason, it is important to monitor the available capacity of the resources provisioned to your workload.

tip

Feedzai uses the renowned TIG stack

One of the solutions Feedzai has implemented for monitoring is based on the TIG stack. This acronym refers to the following systems:

Telegraf: A server-based agent used for collecting and sending metrics or events to a data sink.

InfluxDB: A scalable time series data platform that is often used for real-time analytics through metrics or events.

Grafana: A data visualization and analytics platform that integrates a myriad of technologies to provide real-time data visualization and alerting.

This stack implements a pull-based architecture, which deploys a telegraph agent on all VMs, containers, etc. It periodically collects operation metrics (i.e. CPU, RAM, disk, etc.) as well as application metrics (i.e. health-checks, internal state, etc.), and propagates them to InfluxDB. Grafana pulls data from InfluxDB both to feed its visualizations and to automatically trigger alerts based on the values or trends of specific metrics.

Next stepsâ□□

Learn how to monitor Feedzai systems and how to integrate them in your preferred monitoring solution.

On this page

Monitoring ZooKeeper

tip

For a complete reference of the monitoring recommendations for ZooKeeper, refer to the official documentation. Make sure to refer to the version being used.

Collecting metricsâ

Since version 3.6.0 ZooKeeper has started to expose several metrics to monitor multiple parts of the system. The recommendation from the maintainers is to use Prometheus with Grafana for visualization, but there is also an alternative to use InfluxDB alongside the Telegraf plugin.

It is also possible to start ZooKeeper with JMX and then use Jolokia, a JMX-HTTP bridge plugin that makes JMX-based systems more suitable to agent-based solutions.

Feedzai also recommends the usage of Exhibitor, a supervisor system for ZooKeeper originally implemented by Netflix that also provides an API-based facility to manage the cluster.

Top Metrics to Monitorâ

In this section you will find which metrics are important to monitor in **ZooKeeper**.

info

Note that the following list is simply a reference to the most important metrics, and should not be considered as the single source of monitoring the healthiness of the system.

Metric Name	Description
`avg_latency`	The average latency of the requests executed against ZooKeeper
`znode_count`	The number of znodes (suggested to be keept below 1000000)
`open_file_descriptor_count`	Number of files opened (suggested to be keept below 300)
`election_time_count`	Used to determine if there was a new leader election recently
`followers`	Number of followers, could be used to computed the quorum based on total number of nodes (leader only)

Get Started With Monitoring

info

Important note

Feedzai systems are preferably deployed in Feedzai Cloud. In such deployments, monitoring is performed internally by Feedzai, and there is no need to provide clients with monitoring instructions. However, for some clients, Feedzai software may be deployed in a client-hosted environment, i.e. on-premise or in a public cloud).

This monitoring guide covers client-hosted deployments and, for that reason, it is not part of our standard client-facing documentation. The guide should only be provided to clients that require instructions on how to monitor their own systems.

The purpose of this guide is to empower users to monitor Feedzai's system performance and investigate statuses autonomously.

In this guide you will learn:

- How to collect metrics.
- What kind of metrics there are.
- Which metrics are especially relevant for Pulse.
- Which metrics and tables are especially relevant for Case Manager.
- Which metrics are especially relevant for Reference Data Service.

tip

Note that this guide only covers some Feedzai services, as an example. Nevertheless, the same principles may apply to other products and components.

Monitoring Third-Party and Open Source Components

info

Important note

While Feedzai can provide recommendations on how to monitor third-party and open source software based on experience of using that software, Feedzai is not accountable for the software or its observability facilities. For client-hosted installations, clients are expected to install and operate such software, and that also extends to monitoring.

This guide only provides a reference to some of the most important metrics to monitor on some of these systems, and does not constitude a definitive monitoring guide.

The purpose of this guide is to complement the recommendations for monitoring Feedzai components with recommendations for third-party dependencies.

This guide covers:

- Monitoring recommendations for ZooKeeper.
- Monitoring recommendations for Cassandra.
- Monitoring recommendations for RabbitMQ.

On this page

Top Metrics to Monitor in Case Manager

In this section you will find which metrics are important to monitor in **Case Manager**.

info

Note that the following list is simply a reference to the most important metrics, and should not be considered as the single source of monitoring the healthiness of the system.

For a complete and comprehensive monitoring guide, please refer to the official Case Manager documentation

Please also refer to Understand Available Metrics for more information.

note

The following Case Manager standard application tags are available in every metric that references `Standard CM application tags`:

- `metricType`
- `appType`

Event Processing Metricsâ□□

New Event Throughputâ□□

Measures the event processing throughput when the event type is "new event", for the selected Case Manager Instance. An event is considered to be "new" if the identifier of that event has never been seen. Otherwise, it is seen as an update.

Metric	Description	Tags	Type
`cm.event.processing.latency`	The event processing throughput for "new events"	• `eventType` • Standard CM application tags	Counter

What to Monitorâ□□

This metric is useful to ensure that the rate at which events are being processed is as expected. If this rate is lower than what Pulse is processing, then it could mean that there is a problem in the system, and there will be backpressure on RabbitMQ.

New Event Latencyâ□□

Measures the event processing latencies per percentile (50th, 95th, 99th, and 99.9th) when the event type is "new event", for the selected Case Manager instance.

Metric	Description	Tags	Type
`cm.event.processing.latency`	The event processing latency for "new events" at various percentiles.	• `eventType` • Standard CM application tags	Timer

What to Monitorâ□□

This metric is useful together with the previous throughput metric. As it reports the latency of the new events processed, if the throughput starts to decrease, we can use this metric as a complement to understand what can be contributing to that. For example, the system might be reaching memory exhaustion (see JVM metrics), or it's taking too long to run all the automations configured to trigger on event arrival.

Event Processing Queue Occupancyâ□□

Measures the percentage of the event processing queue being used, for the selected Case Manager instance.

Metric	Description	Tags	Type
`cm.event.processing.queue.ratio`	The percentage of the event processing queue being used.	• Standard CM application tags	Gauge

What to Monitorâ□□

This metric shows whether events are being queued in Case Manager's internal processing queue or not.

Event Queue Deadletterâ□□

The Event Queue deadletter is the process by which events that could not be processed successfully are written to local storage to be re-attempted later on by an automatic process. The retry outcomes are also measured.

Metric	Description	Tags	Type
`cm.event.queues.dump.storage.dump.entries.failed.counter`	The total number of failed event queue entries dumped to storage.	• Standard CM application tags	Counter
`cm.event.queues.dump.storage.dump.entries.retry.counter`	The total number of retried event queue entries dumped to storage.	• Standard CM application tags	Counter

What to Monitorâ□□

Events are not expected to be spilled to disk, and hence, this metric should report zero. Otherwise, this indicates that some events are failing to be ingested from the upstream queue and processed. This could happen, for example, if there is an automation or custom extension being invoked on event arrival that failed exceptionally.

Event Storage Deadletterâ□□

The Event Storage deadletter is the process by which events were processed but could not be persisted successfully. These events are written to local storage to be re-attempted later on by an automatic process. The retry outcomes are also measured.

Metric	Description	Tags	Type
`cm.event.storage.dump.storage.dump.entries.failed.counter`	The total number of failed event storage entries dumped to storage.	• Standard CM application tags	Counter
`cm.event.storage.dump.storage.dump.entries.retry.counter`	The total number of retried event storage entries dumped to storage.	• Standard CM application tags	Counter

What to Monitorâ□□

Events are not expected to be spilled to disk, and hence, this metric should report zero. Otherwise, this indicates that some events are failing to be persisted to the database. This could happen, for example, if there is a connectivity issue with the database, or if the operation is failing exceptionally.

Automation Rules Metricsâ□□

Event Automations Throughputâ□□

Measures the throughput of the automation rules execution, for a given automation rule identified by the tag `ruleName`.

Metric	Description	Tags	Type
`cm.automation`	The throughput of automation rule execution.	• `ruleName` • Standard CM application tags	Counter

What to Monitorâ□□

This metric is useful to ensure that the rate at which specific automations are being triggered is as expected. If you have a flow that depends on specific rules that trigger on event updates and such automations are not triggering, then it can indicate a failure in some part of the integration.

Event Automations Latencyâ□□

Measures the latency per percentile (50th, 95th, 99th, and 99.9th) of the automation rules execution, for a given automation rule identified by the tag `ruleName`.

Metric	Description	Tags	Type
`cm.automation`	The latency of automation rule execution at various percentiles.	• `ruleName` • Standard CM application tags	Timer

What to Monitorâ□□

This metric is useful together with the previous throughput metric. As it reports the latency of the automations being executed, if the throughput starts to decrease, we can use this metric as a complement to understand what can be contributing to that. For example, specific actions of certain automations could be inflicting some overhead in the system.

Event Automations Errorsâ□□

Measures the errors observed during the execution of automation rules, for individual rules. The rule is identified by the tag `ruleName`.

Metric	Description	Tags	Type
`cm.automation.error`	The rate of errors returned by the automation execution.	• `ruleName` • Standard CM application tags	Meter

What to Monitorâ□□

This metric provides a simple way to understand if the automations execution is failing during its execution. When there are no errors, the value of this counter is zero. If this metric reports a different value, then you must look into the log files for additional error details.

Integration Metricsâ

Pulse Requests Errorsâ

Measures the errors returned by the Pulse Server API, for the endpoint described in the path tag. Case Manager engages with Pulse for several purposes, such as propagating fraud feedbacks, updating list items, validating user roles and permissions, etc. Each operation is done in a different endpoint.

Metric	Description	Tags	Type
`cm.request.pulse.api.error`	The rate of errors returned by the Pulse endpoint.	• `path` • Standard CM application tags	Meter

What to Monitorâ

This metric provides a way to monitor the integration between Case Manager and Pulse, where errors are not expected to occur. If the error count over time is greater than zero, then that means that either there is a network issue preventing Case Manager from contacting Pulse, or Pulse is facing problems. In any case, if there are errors it means that some operations have failed and may need to be retried (e.g., listing entities).

Infrastructure Metricsâ

Database Connection Poolâ

Each instance of Case Manager has a lazy connection pool to a database. One of the most important to monitor is the one used by the Apps Service. This service runs in Pulse Server and is responsible for handling all metadata management. The metrics for the DB pool are available under the `cm.db.connection.pool` prefix.

The metrics of the DB pool are available under the `cm.db.connection.pool` prefix.

Metric	Description	Tags	Type
`cm.db.connection.pool.size`	The current size of the connection pool (if pool is lazy, the value can grow over time).	• Standard CM application tags	Gauge
`cm.db.connection.pool.free`	The number of free database connections.	• Standard CM application tags	Gauge
`cm.db.connection.pool.used`	The number of used database connections.	• Standard CM application tags	Gauge
`cm.db.connection.pool.waiting`	The number of threads waiting for a database connection.	• Standard CM application tags	Gauge
`cm.db.connection.pool.usage`	The current usage of the database connection pool (used/size in percent).	• Standard CM application tags	Gauge

What to Monitorâ

These metrics detect underlying issues with connecting to the DB, or potential problems interacting with it. For example, if connections are not being released because operations are blocked, or if the database is becoming a bottleneck. If the pool usage is always at close to 100%, then there's likely an issue.

JVM Metricsâ□□

Each Pulse Server and Pulse Application instance lives in their own JVM process. Therefore, JVM metrics for each of these processes are available. JVM metrics are available as `letmeknow.jvm`. You'll find many different metrics under this namespace from GC to Memory.

What to Monitorâ□□

Look into at least GC and Memory metrics. These give you an idea of the healthiness of the process.

Other Monitoring Metricsâ□□

info

The following metrics are still being improved and they might change in the near future.

Metric	Description	Tags	Type
`hsbc.afis.responsemeter.httpstatus code`	Operational results (using the count value).	N/A	Counter
`hsbc.afis.retry`	Number of requests retried (using the count value).	N/A	Counter
`hsbc.afis.fallback`	Measures the number of events dumped to disk (using the count value).	N/A	Counter
`hsbc.afis.requestlatencyHistogram`	Requests latency (using the available percentile values).	N/A	Counter

On this page

Top Metrics to Monitor in Pulse

In this section you will find which metrics are important to monitor in **Pulse**.

info

Note that the following list is simply a reference to the most important metrics, and should not be considered as the single source of monitoring the healthiness of the system.

Please refer to Understand Available Metrics for more information.

note

The following Pulse standard application tags are available in every metric that references `Standard Pulse application tags`:

- `metricType`
- `appType`
- `appSlug`

warning

Some metrics will only be available in the metrics endpoint if the value is greater than zero, otherwise it might be expected for them to not appear at all.

Workflowâ□□

Workflow Throughputâ□□

Metric	Description	Tags	Type
`pulse.workflow.global.event.stats`	Measures the global throughput of the selected workflow belonging to the specified appSlug. It displays the throughput per message type (normal, recovery and replica).	• `workflowDesc` • `workflowId` • `messageType` • Standard Pulse application tags	Counter

Because this metric is captured for each type of message, the messageType tag allows you to distinguish between:

- **NORMAL** - Used for news messages that arrive at workflow.
- **RECOVERY** - Used for messages that are received when the system is doing recovery during a publish operation.
- **REPLICA** - Used to propagate the result of a NORMAL message, to all other Pulse instances.

What to monitorâ□□

This metric is useful to ensure that the rate at which events are being processed is as expected. If you are injecting events at 100 events per second into the workflow, this metric should report a similar value. If the rate being reported is consistently below the expected number, this might indicate that events are not being processed fast enough. This can indicate that there is a problem somewhere in the system.

Workflow Latenciesâ

Metric	Description	Tags	Type
`pulse.workflow.global.event.stats`	Measures the global latencies per percentile (50, 75, 95, 99 and 99.9), of the selected workflow belonging to the specified appSlug.	• `workflowDesc` • `workflowId` • `messageType` • Standard Pulse application tags	Timer

Because this metric is captured for each type of message, the `messageType` tag allows us to distinguish between **NORMAL**, **RECOVERY** and **REPLICA** messages.

What to monitorâ

This metric is useful together with the previous throughput metric. As it reports the latency of the events being processed, if the throughput starts to decrease, we can use this metric as a complement to understand what can be contributing to that -- for example, the system might be reaching memory exhaustion (see JVM metrics), or itâs taking to long to fetch historical profiles from storage (see Profile Fetch Latencies metrics).

Workflow Errorsâ

Metric	Description	Tags	Type
`pulse.workflow.eventErrors.count`	Measures the global workflow errors registered on the selected pulse instance, appSlug and appSlug workflow.	• `workflowDesc` • `workflowId` • Standard Pulse application tags	Counter

What to monitorâ

This metric provides a simple way to understand if there are any errors during event processing in a workflow.

When there are no errors, the value of this counter is zero. If this metric reports a different value, then you must look into the log files for additional error details.

Eventsâ

Event Pending Persistence to Event Storageâ

Metric	Description	Tags	Type
`pulse.workflow.eventstorage.eventsQueuedSize`	Measures the number of events that are pending to be persisted to the Event Storage (on DB).	• `workflowDesc` • `workflowId` • Standard Pulse application tags	Counter

What to monitorâ

Events are persisted in batches and at periodic intervals. Both the size of the batch and interval at which the batch persistence is triggered are configurable in the Pulse Administration page.

The Pulse properties are:

- `services.modules.rte.workflowmanager.eventstorage.databasebatch.size`
- `services.modules.rte.workflowmanager.eventstorage.databasebatch.timeout`

As such, the number reported by this metric shouldnât greatly exceed the size configured for the batch (which defaults to 10000).

Custom Alert Storage Service Additional Fieldsâ

Metric	Description	Tags	Type
`hsbc.workflow.ass.missingExpectedFields.count`	Number of additional fields configured to enrich events that are missing from the workflow schema.	N/A	Counter

Workflow Eventsâ

Metric	Description	Tags	Type
`workflow.events`	Measures the events being processed by each workflow belonging to the specified appSlug.	Check detailed description below	Meter

Unlike the Workflow Throughput metrics, this metric only takes normal messages into account.

The following tags are available to further breakdown the events being processed:

- `fdz_channel` : describes the channel of the event.
- `event_type` : describes the event type of the event.
- `decision` : describes the workflow decision outcome, for the event.
- `fraud` : describes the workflow fraud outcome, for the event.
- `Alert` : describes whether the event resulted in an alert.
- `appSlug` : describes the appSlug.
- `tenant` : describes the tenant.
- `workflowId` : describes the workflowId associated with the event.

What to monitorâ

This metric provides a comprehensive breakdown of the events being processed. Its uses include, but are no limited to:

- roughly counting the number of events flowing in the system (possibly by channel or even event type)
- roughly monitoring the decline rate of the system (possibly by channel or even event type)
- roughly monitoring the alert rate of the system (possibly by channel or even event type).

Events Dumped to Diskâ

Metric	Description	Tags	Type
`pulse.dump.storage.dump.entries.retry.counter`	The total number of events that are currently stored in the disk to be retried by the auto recovery job.	• `workflowDesc` • `workflowId` • Standard Pulse application tags	Counter
`pulse.dump.storage.dump.entries.failed.counter`	The total number of events that are currently stored in the disk but that already reached the maximum amounts of retry, therefore are no longer reprocessed.	• `workflowDesc` • `workflowId` • Standard Pulse application tags	Counter

Measures the number of events dumped to disk per host, appSlug, and workflow element. When events reach the end of the workflow they are persisted in the Event Storage (Database). If the Event Storage isn't available (e.g. The Database is offline, there's a network error, or some other unexpected error) the events are dumped to disk and are reprocessed later on by the Event Storage Dump component.

What to monitorâ

Events are not expected to be dumped to disk. Therefore, this metric should report zero. Otherwise, this indicates that some events are failing to be persisted to the DB, which suggests an issue in the system.

Events spilled to Diskâ

Metric	Description	Tags	Type
`pulse.eventStorage.spillToDisk.count`	The number of events that failed persistence to event storage and that were dumped into local disk.	• `workflowDesc` • `workflowId` • Standard Pulse application tags	Counter
`pulse.eventStorage.spillToDisk.pendingQueue`	The number of events that failed persistence to event storage and that are pending in-memory to be dumped into local disk.	• `workflowDesc` • `workflowId` • Standard Pulse application tags	Gauge

Measures the number of events spilled to disk per host, appSlug and workflow element. Events are spilled to disk when, after being processed by the workflow, they fail to be persisted to the Event Storage (on the DB). This might happen because the DB is down, thereâ€™s a network error, a timeout occurred, or some other unexpected error.

What to monitorâ

Events are not expected to be dumped to disk. Therefore, this metric should report zero. Otherwise, this indicates that some events are failing to be persisted to the DB, which suggests an issue in the system.

Auto Recovery Jobâ

Metric	Description	Tags	Type
`pulse.dump.storage.retry.queue.size`	Number of in-memory entries waiting to be written to the retry folder.	• `workflowDesc` • `workflowId` • Standard Pulse application tags	Gauge
`pulse.dump.storage.failed.queue.size`	Number of in-memory entries waiting to be written to the failed folder.	• `workflowDesc` • `workflowId` • Standard Pulse application tags	Gauge
`pulse.dump.storage.recovery.job`	Whether the auto recovery job is running or not.	• `workflowDesc` • `workflowId` • Standard Pulse application tags	Gauge

These sets of metrics capture the current status of the auto recovery job used for persisting Events Dumped to Disk.

These metrics are available under the `pulse.dump` prefix. To capture a specific workflow, use the `workflowDesc` tag. As an example, to capture auto recovery job metrics for the `Payments` workflow, look for `workflowDesc=Payments` .

What to monitorâ

These metrics are important to know if the recovery job may have issues processing the spilled to disk events and for how long it is running. For instance, if the recovery job runs for several hours it may be due to database issues or because the Recovery Job is taking too much time to process events.

Jobsâ

Scheduled Jobsâ

These sets of metrics capture the current status of all scheduled jobs running in the system.

Metric	Description	Tags	Type
`pulse.job.executions.scheduled`	Number of jobs that are scheduled.	• `jobType` • Standard Pulse application tags	Counter
`pulse.job.executions.waiting`	Number of jobs that are waiting.	• `jobType` • Standard Pulse application tags	Counter
`pulse.job.executions.executing`	Number of jobs that are executing.	• `jobType` • Standard Pulse application tags	Counter
`pulse.job.executions.failed`	Number of jobs that failed.	• `jobType` • Standard Pulse application tags	Counter
`pulse.job.executions.crashed`	Number of jobs that crashed.	• `jobType` • Standard Pulse application tags	Counter
`pulse.job.executions.aborted`	Number of jobs that were aborted.	• `jobType` • Standard Pulse application tags	Counter

What to monitorâ

These metrics are especially important to detect jobs that have either failed, crashed or been aborted, as this is a sign that something went wrong -- in which case the log files will provide more detailed information.

Profile Fetches Latenciesâ

Metric	Description	Tags	Type
`pulse.workflow.profileFetches.statistics`	Provides statistics on the latency of fetching these profiles from the external storage.	• `elementId` • `workflowDesc` • `workflowId` • `elementDesc` • Standard Pulse application tags	Timer

What to monitorâ

This depends on the use case. Nevertheless, make sure you understand what are the expected latencies, and look out for any continuous deviation from those values. If the latency starts to increase, there could be an underlying problem with the profile store in Cassandra. Look in the log files for more details.

Pulse Application Statesâ

Pulse Statusâ

As an application server, Pulse is responsible for managing the lifecycle of the applications it serves. This lifecycle varies with the state at which the application is. For example, if an application is healthy and running as expected, it is said to be in state `STARTED`. However, if someone publishes a change, it transitions to state `APPLYING_CHANGES`.

How to monitorâ

There are 10 possible application states, and these are being reported as an integer between 1 and 10. The states and their corresponding value are:

- **STOPPING(1)**: Application is stopping. For each AppEngine, this means that it is shutting down the JVM.
- **STOPPED(2)**: Application has stopped. Therefore, it should not have any JVM process.
- **BOOTSTRAPPING(3)**: Application is bootstrapping, i.e. services are being instantiated. This means that each AppEngine in this state is starting the JVM and creating the services, which haven't started yet.
- **BOOTSTRAPPED(4)**: Application has been bootstrapped. This means that for each AppEngine in this state, the respective JVM is up and running and all services have been created (but not started).
- **STARTING(5)**: Application is starting. This means that each AppEngine is starting its services since they were only instantiated in the previous state.
- **STARTED(6)**: Application has started. This means that, for each AppEngines, the services have started and the definitions that were meant to be published have been loaded. Both the AppEngine and app as a whole transition to this state from `APPLYING_CHANGES`.
- **CRASHED(7)**: Application has crashed. This means that it does not have any AppEngine alive. Each AppEngine never assigns itself this state. This is not possible since its JVMs are off.
- **APPLYING_CHANGES(8)**: Application is applying changes. This includes loading (or unloading) the definitions to be published. The AppEngine may have transitioned to this state from `STARTED` (in case it's re-publishing an already published app) or from `STARTING` (in case it's publishing an app that was previously in the `BOOTSTRAPPED` state). In either case, and if the AppEngine is working properly, it will transition to the `STARTED` state.
- **DEGRADED(9)**: Application is in degraded state, i.e., there may be partitions with no replicas online and/or there are replicas running with distinct versions. This means that some AppEngines lack JVMs, and therefore they can't assign themselves this state. The healthy AppEngines do not transition to this state either.
- **SELF_HEALING(10)** This is the cluster's overall state and it means that the application is being self-healed by a watchdog correction. The individual AppEngines do not transition to this state. This is an overall app state only.

All AppEngines' states are available under the below metrics. To capture a specific AppEngine, use the `appSlug` tag.

Metric	Description	Tags	Type
`pulse.app.state`	The state of the Pulse Application.	• Standard Pulse application tags	Gauge
`pulse.server.state`	The state of the Pulse Server. .	• Standard Pulse application tags	Gauge

Note that the state of each application is shown via the Pulse interface.

What to monitor in Pulse Application Statesâ

It is normal for applications to switch between states as teams promote new changes. However, it is not normal to see the application stop, crash, or degrade. Tracking the state of the application is crucial to identify silent problems that may require manual intervention, such as repairing a cluster that has replicas running with different versions of Risk Strategy.

JVM metricsâ

Each Pulse Server and Pulse Application instance live in their own JVM process. Therefore, JVM metrics for each of these processes are available.

How to measureâ□□

JVM metrics are available as `letmeknow.jvm`. Youâ□□ll find many different metrics under this namespace, from GC to Memory.

What to monitorâ□□

Look into at least GC and Memory metrics. These give you an idea of the healthiness of the process.

Apps Service DB Connection Poolâ□□

There are several DB connections used in Pulse. However, one of the most important to monitor is the one used by the Apps Service. This service runs in Pulse Server and is responsible for handling all metadata management.

How to measureâ□□

The metrics for all DB pools are available under the `pulse.dbPool` prefix. To capture a specific pool, use the `poolName` tag. In this case, for the Apps Service DB connection pool, look for `poolName=appsService`.

There are several metrics that can be captured:

- **Connections usage**: `pulse.dbPool.usage`
- **Connection errors and timeouts**: `pulse.dbPool.connectionErrors`
- **Connection creation throughput and latency**: `pulse.dbPool.connectionCreationTimeStats`

What to monitor in Apps Service DB Connection Poolâ□□

These metrics detect underlying issues with connecting to the DB. For example, if the time to create a new connection takes much more than the usual average, or if the pool usage is always at 100% capacity, then thereâ□□s likely an issue.

Data Transfer Pipeline Serviceâ□□

To support the Data Transfer Pipeline, there is a workflow service that forwards workflow messages and feedback updates to GCP.

How to monitorâ□□

There are three main aspects that can be monitored on the DTP workflow service:

- Batch executor
- Dump storage
- Processing metrics

The metrics have further breakdown using tags for the channel and processing type. Expect to see the following combinations:

- `channel=MESSAGE,processingType=NORMAL` for workflow events being processed for the first time
- `channel=FEEDBACK,processingType=NORMAL` for feedback updates being processed for the first time
- `channel=MESSAGE,processingType=RECOVERY` for workflow events being retried
- `channel=FEEDBACK,processingType=RECOVERY` for feedback updates being retried

Batch Executorâ□□

The following metrics expose the internal batch executors used by the workflow service:

Metric	Description	Tags	Type
`pulse.workflow.dtp.batch.executor.pool.activeThreadsCount`	Number of active threads.	• `workflowId` • `channel` • `processingType`	Counter

Metric	Description	Tags	Type
`pulse.workflow.dtp.batch.executor.pool.maxPoolSize`	The maximum pool size.	• `workflowId` • `channel` • `processingType`	Gauge
`pulse.workflow.dtp.batch.executor.pool.threadsUsageRatio`	The current thread usage ratio.	• `workflowId` • `channel` • `processingType`	Gauge
`pulse.workflow.dtp.batch.executor.pool.taskQueueCount`	The current size of the task queue.	• `workflowId` • `channel` • `processingType`	Gauge
`pulse.workflow.dtp.batch.executor.pool.taskQueueMaximumSize`	The maximum size of the task queue.	• `workflowId` • `channel` • `processingType`	Gauge
`pulse.workflow.dtp.batch.executor.pool.taskQueueUsageRatio`	The current usage ratio of the task queue.	• `workflowId` • `channel` • `processingType`	Gauge
`pulse.workflow.dtp.batch.executor.service.completedFlushes`	The number of completed flushes.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.batch.executor.service.deadletterEvents`	The number of deadletter events.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.batch.executor.service.failedFlushes`	The number of failed flushes.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.batch.executor.service.lastFlush`	The timestamp of the latest flush.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.batch.executor.service.pendingQueueSize`	The size of the pending queue.	• `workflowId` • `channel` • `processingType`	Gauge
`pulse.workflow.dtp.batch.executor.service.pendingQueueUsageRatio`	The usage ratio of the pending queue.	• `workflowId` • `channel`	Gauge

Metric	Description	Tags	Type
		• `processingType`	

Dump Storageâ

When enabled, the Dump Storage will store failed messages and feedback updates in disk so they can be retried at a later time. Note that for these metrics, the processing type will always be `RECOVERY`. The following metrics are available:

Metric	Description	Tags	Type
`pulse.workflow.dtp.dump.storage.dump.entries.retry.counter`	The number of entries in disk that will be retried.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.dump.storage.dump.entries.failed.counter`	The number of entries in disk that have exhausted all retries.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.dump.storage.retry.queue.size`	Number of in-memory entries waiting to be written to the retry folder.	• `workflowId` • `channel` • `processingType`	Gauge
`pulse.workflow.dtp.dump.storage.failed.queue.size`	The number of in-memory entries waiting to be written to the failed folder.	• `workflowId` • `channel` • `processingType`	Gauge
`pulse.workflow.dtp.dump.storage.recovery.action.latency`	The latency of the recovery job.	• `workflowId` • `channel` • `processingType`	Gauge
`pulse.workflow.dtp.dump.storage.recovery.job`	Whether the auto recovery job is running or not.	• `workflowId` • `channel` • `processingType`	Boolean

Processing Metricsâ

These metrics provide a high level view of how many messages and feedback updates are being processed (successfully and otherwise) by the service:

Metric	Description	Tags	Type
`pulse.workflow.dtp.processing.numberOfReceivedMessages`	The number of messages received.	• `workflowId`	Counter

Metric	Description	Tags	Type
		• `channel` • `processingType`	
`pulse.workflow.dtp.processing.numberOfSuccessfullyProcessedMessages`	The number of successfully processed messages.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.processing.numberOfFailedRecoverableMessages`	The number of messages that failed to be processed but were sent for recovery.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.processing.numberOfFailedNonRecoverableMessages`	The number of messages that failed to be processed and are not recoverable.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.processing.messageLatency`	The latency of the processing.	• `workflowId` • `channel` • `processingType`	Timer

What to monitor in Data Transfer Pipeline Serviceâ□□

It's important to keep track of spikes in messages that failed to be processed. If they are recoverable this may suggest connectivity issues with GCP but if they are not recoverable they may indicate data or integration issues.

It's equally important to keep track if failed messages are being successfully retried or piling up in disk.

On this page

Top Metrics to Monitor in Reference Data Service

In this section, you will find which metrics are important to monitor in **Reference Data Service**.

info

Note that the following list is simply a reference to the most important metrics, and should not be considered the single way of monitoring the system's health.

Database Throughputâ□□

The database throughput measures the data operations' throughput for a given entity.

How to measure Database Throughputâ□□

The database throughput metric is available as `reference-data-service.entity-repository` using the `count` value. Since this metric is captured both by each operation and entity, the tags operation and entityType allow you to have a granular view for each one, respectively. For example, you can monitor the throughput of `upsert` operations for the `customer` entity.

The operations available are similar to the CRUD repository:

- **Get**: Fetch an entity's data by the primary key.
- **Delete**: Delete an entity by the primary key.
- **Upsert**: Update the entity's data or create a new entity if it doesn't exist yet.
- **List**: Retrieve a list of entities based on a set of filters.

What to monitor in Database Throughputâ□□

The database throughput metric is useful to ensure that the rate at which records are being updated by / fetched from each entity is as expected. If the number of upsert operations is below the expected, there may be a problem with the processing of reference data events. Likewise, if the number of `get` operations is not consistent with the event throughput, the system may not be enriching transactions with reference data as expected. In both scenarios, this can indicate that there is a problem somewhere in the system.

Database Latencyâ□□

The database latency metric measures the latencies per percentile (50, 95, 99 and 99.9), of the selected database operations for the specified entity.

How to measure Database Latencyâ□□

The database latency is available as `reference-data-service.entity-repository` and uses the available percentile values. Since this metric is captured both by each operation and entity, the tags `operation` and `entityType` allow you to have a granular view for each one, respectively.

The operations available are the same as mentioned above.

What to monitor in Database Latencyâ□□

The database latency metric is useful to monitor the latency of specific operations and can be very useful together with the previous throughput metric. If the throughput decreases, this metric can contribute to explain that, for example, if the latency of operations in the database degrades, there may be an issue with the database.

"Entity Not Found" Errorsâ□□

The "Entity Not Found" Errors metric measures the errors of "entity not found" type - i.e., lookups by entity key that fail because there is no record for such key.

How to measure "Entity Not Found" Errorsâ□□

The "Entity Not Found" Errors metric is available as `reference-data-service.entity-repository.entitynotfound`, using the `count` value.

What to monitor in "Entity Not Found" Errorsâ□□

The "Entity Not Found" Errors metric is useful to understand how many events miss the reference data enrichment and, when tracked over time, can hint at potential integration problems. Reference data updates are often done as a result of a Change Data Capture (CDC) process, or other data pipelines, and such processes are prone to silent failures. This metric helps identify such integration failures. The count over time is expected to be higher than zero, as there may be cases in which events from a given entity are sent before Feedzai gets any records from that same entity. Nevertheless, the the error count should be stable, and an increase is a strong indication of a process failure upstream.

Database Errorsâ□□

The Database Errors metric measures all database errors excluding the ones of type "entity not found", as previously described.

How to measure Database Errorsâ□□

The Database Errors metric is available as `reference-data-service.entity-repository.error`, using the `count` value.

What to monitor in Database Errorsâ□□

This metric shows whether there are operations failing due to issues in the database. For example, if the service receives an upsert request and the database is unresponsive, the operation fails, and this counter is incremented.

JVM metricsâ□□

Each Pulse Server and Pulse Application instance live in their own JVM process. Therefore, JVM metrics for each of these processes are available.

How to measure JVM metricsâ□□

JVM metrics are available as `letmeknow.jvm`. Youâ□□ll find many different metrics under this namespace, from GC to Memory.

What to monitor in JVM metricsâ□□

Look into at least GC and Memory metrics. These give you an idea of the healthiness of the process.

On this page

Understand Available Metrics

Access `https://<host:port>/metrics` on your browser to see which metrics are reported. Note that a large number of metrics is reported.

This section explains the different types of metrics being reported and how they are grouped.

Available types of metricsâ

Learn below about the different types of metrics, what kind of information they report, and how they are represented.

Gaugesâ

Gauges are the simplest metric type. They represent a single value.

Example:

```
"audit.writer.activeThreadsCount,metricType=APPLICATION,appType=PULSE_SERVER": {
  "value": 0
}
```

Copy

Countersâ

Counters are similar to gauges, in that they also report a single value. Conceptually, a counter is a simple incrementing and decrementing 64-bit integer.

Example:

```
"pulse.dbPool.pendingThreads,metricType=APPLICATION,appType=PULSE_SERVER,poolName=appsService": {
  "count": 0
}
```

Copy

Metersâ

A meter measures the rate at which a set of events occur. Meters report the following values:

- **count** - The current number of events.
- **m1_rate** - The median rate at which events occur over a 1-minute window.
- **m5_rate** - The median rate at which events occur over a 5-minute window.
- **m15_rate** - The median rate at which events occur over a 15-minute window.
- **mean_rate** - The median rate at which events occur since they have started being recorded.
- **units** - The units of the rate.

Example:

```
"audit.writer.droppedEntries,metricType=APPLICATION,appType=PULSE_SERVER": {
  count: 0,
  m15_rate: 0,
  m1_rate: 0,
  m5_rate: 0,
  mean_rate: 0,
```

```
  units: "events/second"
}
```

Copy

Timersâ□□

A timer is basically a histogram of the duration of a type of event and a meter of the rate of its occurrence.

info

Note that the elapsed times for events are measured internally in nanoseconds, using Javaâ□□s high-precision System.nanoTime() method. Its precision and accuracy vary depending on the operating system and hardware.

Timers report the following values:

- **count** - The current number of events.
- **max** - The maximum duration since this metric has started being reported.
- **mean** - The maximum duration since this metric has started being reported.
- **min** - The minimum duration since this metric has started being reported.
- **stddev** - The standard deviation of the duration since this metric has started being reported.
- **p50** - The duration in the quantile 50.
- **p75** - The duration in the quantile 75.
- **p95** - The duration in the quantile 95.
- **p98** - The duration in the quantile 98.
- **p99** - The duration in the quantile 99.
- **p999** - The duration in the quantile 99.9.
- **m1_rate** - The median rate at which events occur over a 1-minute window.
- **m5_rate** - The median rate at which events occur over a 5-minute window.
- **m15_rate** - The median rate at which events occur over a 15-minute window.
- **mean_rate** - The median rate at which events occur since they have started being recorded.
- **rate_units** - The units of the rate.
- **duration_units** - The units of the duration.

Example:

```
"pulse.dbPool.connectionCreationTimeStats,metricType=APPLICATION,appType=PULSE_SERVER,poolName=a
ppsService": {
  count: 3,
  max: 0,
  mean: 0,
  min: 0,
  p50: 0,
  p75: 0,
  p95: 0,
  p98: 0,
  p99: 0,
  p999: 0,
  stddev: 0,
  m15_rate: 0.05330846541330047,
  m1_rate: 4.86993058876713e-10,
  m5_rate: 0.003787289909588294,
  mean_rate: 0.0024885250809603644,
  duration_units: "seconds",
  rate_units: "calls/second"
}
```

Copy

Tagsâ□□

Before moving on to how to make use of available metrics, itâ□□s also important to understand some additional information that is included in some metrics, which provides information about their context.

Pulse Server and Pulse Applicationsâ□□

A Feedzai Pulse installation is composed of two main services: Pulse Server and Pulse Application. Each service lives in its own JVM process. A Feedzai Pulse installation has one Pulse Server JVM process per node and one additional JVM process for each application, also in the same node.

When you query the `/metrics` endpoint, the retrieved information corresponds to the relevant Pulse Server JVM process and to the JVM processes of each application. Tags are used to distinguish between metrics that correspond to the Pulse Server and to each of the existing Pulse Applications.

Example 1: Pulse Server

```
pulse.dbPool.pendingThreads,metricType=APPLICATION,appType=PULSE_SERVER,poolName=appsService
```

Copy

Notice the **appType** and **poolName** tags. The first one indicates that this metric corresponds to the Pulse Server instance, while the later is used to identify the name of the DB connection pool whose metric is being reported. In this case, the pool name is required as all DB connection pool metrics have the same `pulse.dbPool.pendingThreads` namespace.

Example 2: Pulse Application

```
pulse.dbPool.pendingThreads,appSlug=transactions,metricType=APPLICATION,creator=SimpleRulesTesti
ngReplayModule,appType=PULSE_APPLICATION,poolName=simpleRulesTesting
```

Copy

This metric also corresponds to `pulse.dbPool.pendingThreads` . However, the **appType** tag indicates *PULSE_APPLICATION*, which means that this metric belongs to a Pulse Application. Additionally, the metric contains an *appSlug=transactions* tag. This **appSlug** tag allows you to distinguish between different Pulse Applications running in the same system.

To recap, when monitoring these metrics, consider the following hierarchy:

1. First per **appType**.

   - *PULSE_SERVER* indicates that a metric corresponds to the Pulse Server instance.
   - *PULSE_APPLICATION* indicates that a metric corresponds to a Pulse Application instance.

1. Then per **appSlug**.

   - Where the value is the name of the Pulse Application. This tag is only available when *appType=PULSE_APPLICATION*.

Metric formatâ□□

Metrics are collected using two endpoints. These endpoints collect the same metrics but output different information each:

`/metrics` provides:

- Version
- Metric type
- Metric namespace
- Value

`/metrics/rich` provides:

- Definition
- Description

- Group namespace
- Thread pool name
- Metric name

See below examples of how this information is displayed.

`/metrics` endpoint informationâ□□

Access `https://<host:port>/metrics` on your browser to display the relevant output. To help explain the format of the reported metrics, consider the example below:

```
{
    "version": "3.1.3",
    "gauges":
    {
            "audit.writer.activeThreadsCount,metricType=APPLICATION,appType=PULSE_SERVER":
            {
                    "value": 0
            }
...
```

Copy

What you see above is the first reported metric. Letâ□□s break down its relevant parts:

- *gauges* represents the type of metric being reported. Metrics are grouped by their type. In this case, itâ□□s a gauge and returns a value.
- *audit.writer.activeThreadsCount* is the namespace of the metric. In this case, you are looking at the *activeThreadsCount* of the *audit.writer*.
- *value* is the actual value of the metric. In this case, itâ□□s the number of active threads.

As this example shows, all metrics are represented by their namespaces. However, this information may not be enough to get the actual meaning of each metric. To get more information, you can use the `/metrics/rich` endpoint.

Letâ□□s see how to do this by taking the previous example that used a metric with the *audit.writer.activeThreadsCount* namespace.

`/metrics/rich` endpoint informationâ□□

In the `/metrics/rich` endpoint, while all the same metrics are available, they are grouped differently. Instead of being grouped by type (e.g. the *gauge* above) and represented by their namespace (e.g. *audit.writer.activeThreadsCount* above), they are provided as follows:

```
{
    "definition":
    {
                "description": "The usage ratio of audit writer executor threads and task queue.",
                "evaluators": [ ],
                "name": "audit.writer",
                "tags":
                {
                        "metricType": "APPLICATION",
                        "appType": "PULSE_SERVER"
                },
                "unit": "n/a"
    },
    "metric":
    {
                "metrics":
                {
                        "activeThreadsCount":
                        {
                                    "value": 0
```

```
                    },
...
```

Copy

This is the exact metric as above but represented differently. Letâ□□s break the relevant parts down as before:

- *definition* includes the definition of the group of metrics that monitor a specific module/action.
- *description* explains the group of metrics. In this case, this group reports metrics regarding the thread pool of the audit writer.
- *evaluators* can be ignored as they will not be populated.
- *name* is the groupâ□□s namespace. This is used as the prefix for the namespace of all metrics within the group.
- *tags* allows you to distinguish metrics that have the same namespace from each other. They are used to identify the name of a thread pool.
- *metrics* represents the actual metrics for this group.
- *unit* can be ignored as they will not be populated.
- *activeThreadsCount* is the metric name.

To recap, metrics are represented by their namespace in `/metrics`, and namespaces are composed by `<metric group prefix>.<metric name>`. Letâ□□s apply this logic to our example, where:

- *audit.writer.activeThreadsCount* is the namespace of the metric.
- *audit.writer* is the prefix of the metric group to which the metric belongs.
- *activeThreadsCount* is the name of the metric within the group.

tip

To locate the `audit.writer.activeThreadsCount` metric in `/metrics`, search its group prefix in `/metrics/rich`, and then search its name under that group *(audit.writer)* and *activeThreadsCount*, respectively.

On this page

Permissions Migration Tool

In this page you can find the documentation on the permissions migration tool, how to configure and run it.

Overviewâ□□

The `fdz-permissions-migrator` tool is designed to export and import Pulse permissions and their associated roles. This process can be used to promote/migrate permissions and their associated roles between environments.

Requirementsâ□□

The tool execution has the following requirements:

- A `bash` shell
- `jq` installed in the machine where the script is being run (tested with version 1.7.1)
- `curl installed in the machine where the script is being run (tested with version 7.68.0)
- The file must have execution permissions (`chmod u+x fdz-permissions-migrator`)
- Connection between the machine where the script is being run and the Pulse load balancer
- A service account credentials

Resourcesâ□□

The tool can be downloaded here: fdz-permissions-migrator.

Configurationâ□□

Running `./fdz-permissions-migrator help` will show the help message for the tool. It lists the available commands and options:

```
â□¯ ./fdz-permissions-migrator help
fdz-permissions-migrator [v1.0.0] - commandline tool to export and import Pulse permissions and
roles between environments

usage: fdz-permissions-migrator <command> <options>

command:
  help                              Show this help message
  export                            Export permissions to stdout, in JSON format
  import                            Import all permissions and all roles from stdin, in JSON
format
  import-role <role>                Import all permissions and the specified role from stdin, in
JSON format

options:
  -u, --username USERNAME           The username of the service account (default: pulse)
  -p, --password PASSWORD           The password of the service account (default: pulse)
  -h, --hostname HOSTNAME           The Pulse Load Balancer URL (default: http://127.0.0.1:8080)
  -k, --insecure                    Sets the --insecure flag in the underlying cURL requests
  -q, --quiet                       Limit the output of skipped operations
  -nc, --no-cleanup                 Preserve folder with temporary files generated during
execution
```

```
Example:
  $ ./fdz-permissions-migrator export --username pulse --hostname http://localhost:8080
```

Copy

Executionâ□□

Running the tool will usually require the user to specify `--hostname`, `--username` and `--password` as the default values are only useful for development environments. Besides that, the user should select one of the available commands: `export`, `import` or `import-role`.

Assume the following example:

- Permissions and roles are to be migrated from `Environment A` to `Environment B`
- `Environment A` is running under URL `http://env_a:8080/` and `Environment B` under `http://env_b:8080/`
- The default service account will be used (i.e. the default username and password)

The first step would be to export the permissions from `Environment A`:

```
$ ./fdz-permissions-migrator export --hostname http://env_a:8080
[
  {
    "name": "security:tenants:read",
    "roles": [
      "admin",
      "Fraud Prevention Agents - L1"
    ]
  },
  {
    "name": "security:tenants:write",
    "roles": [
      "admin"
    ]
  }
  ...
]
```

Copy

Export will always output the result to stdout but it can instead be redirected to a file:

Using HTTPS

This tool uses cURL to make HTTP requests to the remote servers and accepts HTTPS URLs. If there are any problems with certificate validation, the `--insecure` flag can be passed

```
$ ./fdz-permissions-migrator export --hostname http://env_a:8080 > permissions.json
```

Copy

From this point it depends whether all extracted roles should be import or not.

If yes, we would simply import all roles into `Environment B` by running:

```
$ ./fdz-permissions-migrator import --hostname http://env_b:8080 < permissions.json
```

Copy

If instead we want to import a single role, we can use the `import-role` command:

```
$ ./fdz-permissions-migrator import-role "Fraud Prevention Agents - L1" --hostname http://env_b:8080 < permissions.json
```

Copy

This way all permissions will be added to Pulse, but only the "Fraud Prevention Agents - L1" role will have its permissions modified.

Tip: Make it more quiet!

When dealing with production environments, the amount of roles and permissions being imported can be quite big. The tool will output one line for each permission being imported, even the environment already has that permission.

If you want to avoid logging operations that were skipped because the role/permission already existed, add the `--quiet` flag.

Dealing with passwords

Given that during export the script can not output anything so that stdout redirection does not break, the service account password cannot be prompted.

To make the execution more secure, the password can be passed onto the script by prompting it without printing its value:

```
$ read -sp "Password: " PASSWORD
$ ./fdz-permissions-migrator export --password $PASSWORD
```

Copy

Validation can be done in the Pulse UI by going to `Administration -> Security` and checking if the roles have the correct permissions.

On this page

Pulse Recovery Procedure

This document outlines the available options to recover Pulse following a catastrophic event or outage that disrupts all Pulse nodes.
Recovery focuses on restoring the system to a state where Pulse can process and score real-time payments and digital events, ensuring both real-time and scheduled metrics are accurate so that the DS/rule/model performance is as expected.

warning

Please note, the timings in this guide are relative to the time it takes a Pulse node to publish. The figures used were accurate as of April 2024, but may vary over time.

However, while the exact timings may differ, the principles remain the same.

Pulse operates with a three-node setup, where a single node requires approximately **1h 45m** to complete a publish. The following recovery strategies detail how to handle such scenarios effectively.

Recovery Strategiesâ□□

Option 1: Abort Publish and Perform Manual Publishâ□□

Process:â□□

1. Access the Pulse UI and abort the publish manually.
2. Ensure all three nodes are part of the cluster.
3. Initiate a parallel (non-rolling) publish for all nodes simultaneously.

Recovery Timeline:â□□

- **First 1h 50m:** No processing or scoring occurs while the abort and re-publish steps are performed. The timeline includes an additional ~5 minutes required to abort and initiate the manual re-publish.
- **After 1h 50m:** Full service and redundancy are restored simultaneously.

  note

  Note that parallel publishing across all three nodes may take slightly longer if database performance becomes a bottleneck. Pre-production testing is recommended to validate this behavior.

Considerations:â□□

- Events processed by the GCP load-balancer during the outage will likely have incorrect scores due to:
  - **Real-time metrics** being absent or incorrect.
  - **Workflow clock misalignment**, which could negatively affect model scores and alert rates.
- Despite these issues, previous testing has shown that the impact on scores and alert rates is manageable.

Manually publish with recovery skipped, then republishâ□□

Process:â□□

1. Access the Pulse UI and abort the publish manually.
2. Ensure all 3 Pulse server nodes are part of the cluster, and perform a parallel (non-rolling) publish with "skip recovery" selected.

3. After the publish, save the workflow without changes.
4. Perform a rolling publish (with application restart selected, without **skip recovery**)

Recovery Timeline:â□□

- **First 5â□□10 minutes:** Pulse will not be processing or scoring the events.
- **After 1h 45m:**
    - Partial service and full redundancy are restored at the same time.
    - Pulse will score events, but it will consider all real-time metrics as having the value zero.
    - All scheduled metrics will be correctly updated.
    - Model scores/performance and alert rates will be impacted.
- **After another 1h 45m (total 1h 55m from the start):**
- Half the transactions will have totally correct scores, and the other half will continue to have incorrect real-time metrics.

info

This is because after the first node publishes, one of the remaining two will go down. This means there are two nodes scoring at this point - one that just finished publishing, fully correct, and one with correct SMJs but incorrect real-time metrics.

- **After another 1h 45m (total ~3h 40m from the start):**
- Full service recovery is complete with correct real-time and scheduled metrics.

info

Full redundancy has not yet been achieved as 2 of 3 nodes will be scoring, while the 3rd will go down to update itself.

- **After another 1h 45m (total ~5h 25m from the start):** Full redundancy is achieved.

info

All the nodes are up at this and correctly updated.

Considerations:â□□

- This option ensures quicker partial service restoration compared to Option 1, with limited impact on metrics during the recovery process.
- Transactions processed during the initial phase may have incomplete real-time metrics, but overall service availability is restored sooner.
- Data Science (DS) team changes cannot be published during the recovery process.

Recommendationsâ□□

Feedzai recommends Option 2. It is the standard approach followed in the Feedzai cloud. It tends to have the smallest negative impact overall

- The service is mostly restored (with just real-time metrics missing) very quickly
- Followed by full restoration in steps (half the nodes fully recovered after another 1h 45m, and full restoration after another 1h 45m)

On this page

What's Changed

Release `0.1.1` â□□

Dependency Fixesâ□□

Bug Fix

Resolved an issue with missing `spark-dist` artifact and other dependency fixes.

Release `0.1.0` â□□

Transaction Fraud for PSPsâ□□

Feature

This release introduces **Transaction Fraud for PSPs (TFP)** â□□a real-time fraud detection solution that detects fraud scenarios related to card transactions and alternative payments at the merchant level. TFP is designed for acquirers, gateways, and Payment Service Providers (PSPs) that need to detect abnormal payment behavior.

Case Manager Integrationâ□□

Feature

Case-Manager is integrated to support analyst workflows for reviewing and investigating transactions flagged by the fraud detection models, enabling timely action on identified risks.

Default Feedzai Configurationâ□□

Feature

This release includes default Feedzai configuration for both Pulse and Case-Manager, enabling out-of-the-box deployment and operation of the solution.

⚠**Merchant Monitoring Not Available**

Merchant Monitoring (MM) is not supported in this release. The batch processor required by this use case has not yet been implemented and will be delivered in a future release.

On this page

What's Changed

Release `0.2.0` â□□

Merchant Monitoringâ□□

Feature

This release introduces **Merchant Monitoring (MM)** â□□an account monitoring solution for Payment Service Providers (PSPs) to prevent and detect risk at the merchant level. MM is designed for acquirers, gateways, and Payment Service Providers (PSPs) that need to detect abnormal merchant behavior.

Batch Processor Integrationâ□□

Feature

Batch Processor is integrated to support a scheduled behavioral analysis over complete merchants portfolio, allowing detection of abnormal merchant behavior so that PSPs can act before more fraud and damages occur.

- Batch Processor version: 3.5.6

Default Feedzai Configurationâ□□

Feature

This release includes default Feedzai configuration for Batch Processor, enabling out-of-the-box deployment and operation of the solution.

Support RabbitMQ 4.2.X and quorum queuesâ□□

Feature

This release adds support for quorum queues, making the release compatible with the following RabbitMQ and Erlang versions:

- RabbitMQ 4.2.5
- Erlang 27.3.4.10

By default, "QUORUM" is used but this can be changed by updating the following Ansible variables:

- rabbitmq_queue_type
- rabbitmq_pulse_queue_type
- rabbitmq_casemanager_use_quorum_queues

Upgrade Pulseâ□□

Feature

Upgrade Pulse from 25.1.8 to 25.1.9.

Upgrade Case Managerâ□□

Feature

Upgrade Case Manager from 13.29.0 to 13.31.0.

Fix Case Manager deployment pipelineâ□□

Bug Fix

Add maintenance permissions for Case Manager deployment.

Added `cm:resource:maintenance-start-stop:admin` permission to the admin role to prevent 403 errors during CI pipeline execution.

Fix RDS port mismatch in HTTP Input Serviceâ□□

Bug Fix

Fixed a configuration mismatch where the HTTP Input Service was still using the port 8443 to communicate with the Reference Data Service (RDS).

The reference_data_port Ansible variable has been updated to dynamically follow `reference_data_app_port` (defaulting to 8080), ensuring all endpoints (RMPSP / MM) can communicate with RDS.

This change affects the following properties:

- `pulse.app.rmpsp.solutions.services.input.http.forward.connector.payments.reference.data.storage.rds.port`
- `pulse.app.rmpsp.solutions.services.input.http.forward.reference.data.storage.rds.port`
- `pulse.app.mm.solutions.services.input.http.forward.reference.data.storage.rds.port`

Add dummy Pulse Dev licenses in the installer assemblyâ□□

Bug Fix

Added dummy Pulse licenses to the installer assembly artifact. These have been added to unblock the creation of the Pulse base GCP image for RMA at HSBC's end.

info

These dummy licenses are identical to the licenses in HSBC F24's installer assembly, and are already expired. They are **NOT** substitutes for the actual licenses that are required for Pulse startup, and which HSBC expects to set up in their GCP Secret Manager.

Discussions regarding the actual Pulse Dev licenses for HSBC RMA are ongoing between Feedzai Account Management team and HSBC. The real licenses will be issued after these discussions conclude.

Fix encrypt properties task across multiple Ansible playbooksâ□□

Bug Fix

Changed the Ansible task `415.encrypt-properties` to overcome issues with the default task not being able to run on the HSBC GCP environment for Pulse, Case Manager and Reference Data components, similar to what was done for the F24 project.

deploy-new-applications

On this page

Case Manager Setup

In this page you can find a set of suggestions for Case Manager monitoring using Grafana.

Suggestionsâ□□

info

There are VM resource related metrics (CPU, Memory, IO, etc) that can also be added to Grafana. These were left out on purpose Google already has out of the box monitoring support for these resources.

These suggestions can be either new dashboards, changes to existing dashboards or completely new dashboard pages. Please find the metric description in Case Manager Top Metrics.

Cluster pageâ□□

Currently there is only one page where we can filter metrics by each of the instances. Although useful, having a cluster view monitoring page gives a view of how the overall cluster health. Given this, our suggestion is to add a new page with the following metrics:

Row Title	Title	Description	Query Details
Healthchecks - Case Manager	Database	The database health check relies on the shared CM's connection pool to execute the statement SELECT 1. If the connection is successfully retrieved and the statement finishes, the health check succeeds, if any error occurs in any of the steps the check fails. A timeout of 5 seconds was given when retrieving a connection to avoid hanging the health check response. The outcome is a bool, meaning that it was successfully executed when 1, otherwise is 0.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck" = 'Database' , $timeFilter **Group By:** time($__interval), "host"
Healthchecks - Case Manager	Messaging	The Messaging Server Health Check is achieved by trying to initialize a new connection with the messaging server. The outcome is a bool, meaning that it was successfully executed when 1, otherwise is 0.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck" = 'Messaging' , $timeFilter **Group By:** time($__interval), "host"
Healthchecks - Case Manager	Pulse	The Pulse Health Check when triggered tries to connect to pulse and tries to login. The outcome is a bool,	**Query A** **Field:** "result" **From:** "healthchecks"

Row Title	Title	Description	Query Details
		meaning that it was successfully executed when 1, otherwise is 0.	**Where:** "healthcheck" = 'Pulse' , $timeFilter **Group By:** time($__interval), "host"
Healthchecks - Case Manager	Tokenizer	The tokenizer health checks validates if there is a tokenizer instance reachable and if the authorization configured is valid. The outcome is a bool, meaning that it was successfully executed when 1, otherwise is 0.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck" = 'Tokenizer' , $timeFilter **Group By:** time($__interval), "host"
Healthchecks - Case Manager	RabbitMQ Consumers per Event Queue	Measures the total number of consumers per event queue. Having a number of consumers less then the number of nodes represent a problem in the sense that only one/some of the casemanager(s) is/are consuming events, not distributing the load.	**Query A** **Field:** "consumers" **From:** "rabbitmq_queue" **Where:** queue =~ /.*EVENT_QUEUE.*/ , vhost =~ /hsbcasemanager/ , $timeFilter **Group By:** time($__interval), "queue", "vhost"
Case Manager End-to-End Check	End-to-End Check	Measures the execution of a operation set, constituted by 5 operations: - login - get alert list - load first alert list - get all queues - load first queue The operations are executed sequentially, and if one fails the value is immediately returned, meaning that, the set is successfully executed when the outcome is 5. It means also, that, when value is 1 for instance, the login was successfully done, but if fails getting the alert list.	**Query A** **Field:** "operations" **From:** "casemanager_monitoring" **Where:** $timeFilter **Group By:** time($__interval)
Case Manager End-to-End Check	End-to-End Check - Elapsed Time	Measures the elapsed time to execute a set of operations, per case manager instance. This graph is coupled with the "Operation Set Execution", once it shares the same operations The operations are executed sequentially, and if one fails the elapsed time will be the the time when the operation failed minus the start time.	**Query A** **Field:** "elapsed_time" **From:** "casemanager_monitoring" **Where:** $timeFilter **Group By:** time($__interval)
Event Processing - New Event	New Event - Throughput	Measures the overall throughput for the event processing when the event type is: "new event".	**Query A** **Field:** "count" **From:** "cm.event.processing.latency" **Where:** eventType='NEW_EVENT' , $timeFilter

Row Title	Title	Description	Query Details
			Group By: time($__interval), "host" **Query C** **Field:** "uptime" **From:** "system" **Where:** $timeFilter **Group By:** time($interval), "host"
Event Processing - New Event	New Event - Latency per Percentile	Measures the overall cluster event processing max latency, when the event type is "new event". It displays the higher latencies for percentile 50, 90, 99, 99.9 and also the maximum.	**Query A** **Field:** "p50", "p95", "p99", "p999" **From:** "cm.event.processing.latency" **Where:** eventType='NEW_EVENT' , $timeFilter **Group By:** time($__interval), "host" **Query B** **Field:** "max" **From:** "cm.event.processing.latency" **Where:** eventType='NEW_EVENT' , $timeFilter **Group By:** time($__interval)
Event Processing - Status Change	Status Change - Throughput	Measures the overall throughput for the event processing when the event type is: "status change".	**Query A** **Field:** "count" **From:** "cm.event.processing.latency" **Where:** eventType='STATUS_CHANGE' , $timeFilter **Group By:** time($__interval), "host"
Event Processing - Status Change	Status Change - Latency per Percentile	Measures the overall cluster event processing max latency, when the event type is "status change". It displays the higher latencies for percentile 50, 90, 99, 99.9 and also the maximum.	**Query A** **Field:** "p50", "p95", "p99", "p999" **From:** "cm.event.processing.latency" **Where:** eventType='STATUS_CHANGE' , $timeFilter **Group By:** time($__interval), "host" **Query B** **Field:** "max" **From:** "cm.event.processing.latency" **Where:** eventType='STATUS_CHANGE' , $timeFilter **Group By:** time($__interval)
Automation Rules	Automation Rules - Throughput and Errors	Measures the overall throughput and errors when executing automation rules.	**Query A** **Field:** "count" **From:** "cm.automation" **Where:** $timeFilter **Group By:** time($__interval), "host", "ruleName"
Automation Rules	Automation Rules - Latency per Percentile	Measures the overall cluster latency at percentile 99 to execute an automation rule. It displays highest automation rule latency at percentile 99.9 per instance.	**Query A** **Field:** "p999" **From:** "cm.automation" **Where:** $timeFilter **Group By:** time($__interval), "host", "ruleName"
Pulse Requests	Pulse Requests - Throughput and Errors	Measures the overall throughput and the errors in the communication with pulse instances.	**Query A** **Field:** "count" **From:** "cm.request.pulse.api" **Where:** $timeFilter **Group By:** time($__interval), "host", "path"
Pulse Requests	Pulse HTTP Endpoints and Latencies	Measures the latency in the communication with pulse instances.	**Query A** **Field:** "count", "p99" **From:** "cm.request.pulse.api" **Where:** $timeFilter **Group By:** "host", "path"
Event Processing	Event Processing Queue Occupancy	Measures the event processing queue occupancy	**Query A** **Field:** "value"

Row Title	Title	Description	Query Details
		(internal cm queue), per case manager instance.	**From:** "cm.event.processing.queue.ratio" **Where:** $timeFilter **Group By:** time($__interval), "host"
Event Processing	Event Processing Batch audit-batch Queue Occupancy	Measures the audit batch queue occupancy (internal cm queue), per case manager instance.	**Query A** **Field:** "value" **From:** "cm.event.processing.queue.occupancy.audit-batch" **Where:** $timeFilter **Group By:** time($__interval), "host"
Event Processing	Event Processing Batch audit-batch Flush Latency at p99	Measures the audit batch queue latency (internal cm queue), per case manager instance.	**Query A** **Field:** "p99" **From:** "cm.event.processing.batch.flush.latency.audit-batch" **Where:** $timeFilter **Group By:** time($__interval), "host"
Event Processing	Event Processing Batch explanations-batch Queue Occupancy	Measures the explanations batch queue occupancy (internal cm queue), per case manager instance.	**Query A** **Field:** "value" **From:** "cm.event.processing.queue.occupancy.explanations-batch" **Where:** $timeFilter **Group By:** time($__interval), "host"
Event Processing	Event Processing Batch explanations-batch Flush Latency at p99	Measures the explanations batch queue latency (internal cm queue), per case manager instance.	**Query A** **Field:** "p99" **From:** "cm.event.processing.batch.flush.latency.explanations-batch" **Where:** $timeFilter **Group By:** time($__interval), "host"
Event Processing	Event Processing Batch event-storage-batch Queue Occupancy	Measures the event storage batch queue occupancy (internal cm queue), per case manager instance.	**Query A** **Field:** "value" **From:** "cm.event.processing.queue.occupancy.event-storage-batch" **Where:** $timeFilter **Group By:** time($__interval), "host"
Event Processing	Event Processing Batch event-storage-batch Flush Latency at p99	Measures the event storage batch queue latency (internal cm queue), per case manager instance.	**Query A** **Field:** "p99" **From:** "cm.event.processing.batch.flush.latency.event-storage-batch" **Where:** $timeFilter **Group By:** time($__interval), "host"
Event Processing	Event Processing Batch event-common-batch Queue Occupancy	Measures the event common batch queue occupancy (internal cm queue), per case manager instance.	**Query A** **Field:** "value" **From:** "cm.event.processing.queue.occupancy.event-common-batch" **Where:** $timeFilter **Group By:** time($__interval), "host"
Event Processing	Event Processing Batch event-common-batch Flush Latency at p99	Measures the event common batch queue latency (internal cm queue), per case manager instance.	**Query A** **Field:** "p99" **From:** "cm.event.processing.batch.flush.latency.event-common-batch" **Where:** $timeFilter **Group By:** time($__interval), "host"
Event Processing	Event Processing Batch event-queue-automations-batch Queue Occupancy	Measures the event queue automations batch queue occupancy (internal cm	**Query A** **Field:** "value" **From:** "cm.event.processing.queue.occupancy.event-queue-automations-batch"

Row Title	Title	Description	Query Details
		queue), per case manager instance.	**Where:** $timeFilter **Group By:** time($__interval), "host"
Event Processing	Event Processing Batch event-queue-automations-batch Flush Latency at p99	Measures the event queue automations batch queue latency (internal cm queue), per case manager instance.	**Query A** **Field:** "p99" **From:** "cm.event.processing.batch.flush.latency.event-queue-automations-batch" **Where:** $timeFilter **Group By:** time($__interval), "host"
Deadletters	Deadletter Events	Measures the total number of deadletters.	**Query A** **Field:** "count" **From:** "cm.event.processing.deadletter.count" **Where:** $timeFilter **Group By:** time($__interval), "host"
DB Connection Pool	DB Connection Pool usage	Measures the overall db connection pool usage.	**Query A** **Field:** "value" **From:** "cm.db.connection.pool.usage" **Where:** $timeFilter **Group By:** time($__interval), "host"
DB Connection Pool	DB Connection Pool Waiting Threads	Measures the overall db connection pool waiting threads.	**Query A** **Field:** "count" **From:** "cm.db.connection.pool.waiting" **Where:** $timeFilter **Group By:** time($__interval), "host"
Case Manager HTTP Endpoints and Latencies	Case Manager HTTP Endpoints and Latencies	Measures the latency of Case Manager HTTP endpoints.	**Query A** **Field:** "count", "p99" **From:** "cm.request.api" **Where:** $timeFilter **Group By:** "host", "path"
Login Errors	Login Error Percent(10 Minute Window)	Measures the number of logins and its error percentage over a 10 minute window.	**Query A** **Field:** "value" **From:** "cm.request.login" **Where:** $timeFilter **Group By:** time($__interval), "host"
JVM	JVM Heap Usage	Measures the jvm heap usage per component instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.memory.heap.usage" **Where:** $timeFilter **Group By:** time($__interval), "host"
JVM	JVM Heap Usage After GC	Measures the jvm heap usage after gc per instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.memory.heap.after-gc.usage" **Where:** $timeFilter **Group By:** time($__interval), "host"
JVM	JVM Garbage Collector Time	Measures maximum JVM GC duration. Long garbage collection pause durations indicate that the heap size is not adjusted for the actual usage.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.gc.G1-Old-Generation.time" **Where:** $timeFilter **Group By:** "host"
JVM	JVM Active Threads Count	Measures the current number of active threads in the JVM per instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.threads.count" **Where:** $timeFilter **Group By:** time($__interval), "host"
JVM			

Row Title	Title	Description	Query Details
	JVM File Descriptors Ratio	Measures the jvm file descriptors usage ratio per instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.filedescriptor" **Where:** $timeFilter **Group By:** time($__interval), "host"

The `Row Title` column refers to a Grafana feature to group dashboards by type. This is only a suggestion and thus not mandatory.

Node pageâ

These suggestions relate to the existing Case Manager dashboard page. You may find some new dashboards or changes to existing ones.

Our suggestions are to add the dashboards in the following table with a filter by each of the available nodes and applications:

Row Title	Title	Description	Query Details
Case Manager - Healthchecks	Database	The database health check relies on the shared CM's connection pool to execute the statement SELECT 1. If the connection is successfully retrieved and the statement finishes, the health check succeeds, if any error occurs in any of the steps the check fails. A timeout of 5 seconds was given when retrieving a connection to avoid hanging the health check response. The outcome is a bool, meaning that it was successfully executed when 1, otherwise is 0. This healthcheck corresponds to the case manager selected instance.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck" = 'Database' , "host" = '$host' , $timeFilter **Group By:** time($__interval)
Case Manager - Healthchecks	Messaging	The Messaging Server Health Check is achieved by trying to initialize a new connection with the messaging server. The outcome is a bool, meaning that it was successfully executed when 1, otherwise is 0. This healthcheck corresponds to the case manager selected instance.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck" = 'Messaging' , "host" = '$host' , $timeFilter **Group By:** time($__interval)
Case Manager - Healthchecks	Pulse	The Pulse Health Check when triggered tries to connect to pulse and tries to login, if it succeeds the check will also succeed. The outcome is a bool, meaning that it was successfully executed when 1, otherwise is 0. This healthcheck corresponds to the case manager selected instance.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck" = 'Pulse' , "host" = '$host' , $timeFilter **Group By:** time($__interval)
	New Event - Throughput	Measures the event processing throughput when the event type	**Query A** **Field:** "count"

Row Title	Title	Description	Query Details
Event Processing - New Event		is "new event", for the selected case manager instance.	**From:** "cm.event.processing.latency" **Where:** "host" = '$host' , eventType='NEW_EVENT' , $timeFilter **Group By:** time($__interval)
Event Processing - New Event	New Event - Latency per Percentile	Measures the event processing latencies when the event type is "new event", for the selected case manager instance.	**Query A** **Field:** "p50", "p95", "p99", "p999", "max" **From:** "cm.event.processing.latency" **Where:** "host" = '$host' , eventType='NEW_EVENT' , $timeFilter **Group By:** time($__interval)
Event Processing - Status Change	Status Change - Throughput	Measures the event processing throughput when the event type is "status change", for the selected case manager instance.	**Query A** **Field:** "count" **From:** "cm.event.processing.latency" **Where:** "host" = '$host' , eventType='STATUS_CHANGE' , $timeFilter **Group By:** time($__interval)
Event Processing - Status Change	Status Change - Latency per Percentile	Measures the event processing latencies when the event type is "status change", for the selected case manager instance.	**Query A** **Field:** "p50", "p95", "p99", "p999", "max" **From:** "cm.event.processing.latency" **Where:** "host" = '$host' , eventType='STATUS_CHANGE' , $timeFilter **Group By:** time($__interval)
Automation Rules	Automation Rule $automationRule - Throughput and Error Percent	For the selected case manager instance, measures the throughput to execute the selected automation rule and the associated error percentage.	**Query A** **Field:** "count" **From:** "cm.automation" **Where:** "host" = '$host' , "ruleName" = '$automationRule' , $timeFilter **Group By:** time($__interval)
Automation Rules	Automation Rule $automationRule - Latency per Percentile	For the selected case manager instance, measures the latency per percentile (50, 95, 99, 99.9 and 100) to execute the selected automation rule.	**Query A** **Field:** "p50", "p95", "p99", "p999", "max" **From:** "cm.automation" **Where:** "host" = '$host' ruleName = '$automationRule' , $timeFilter **Group By:** time($__interval)
Pulse Requests	Pulse Requests - Throughput and Error Percent	For the selected case manager instance, measures the throughput of the communications with pulse and the error percent inherent to this.	**Query A** **Field:** "count" **From:** "cm.request.pulse.api" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval), "path"
Pulse Requests	Pulse HTTP Endpoints and Latencies	Measures the latency in the communication with pulse instances.	**Query A** **Field:** "count", "p99" **From:** "cm.request.pulse.api" **Where:** "host" = '$host' , $timeFilter **Group By:** "path"
Event Processing	Event Processing Queue Occupancy	Measures the percentage of the event processing queue being used, for the selected case manager instance.	**Query A** **Field:** "value" **From:** "cm.event.processing.queue.ratio" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval)
Event Processing	Events Waiting in Queue	Measures the number of events waiting to be processed, for the selected case manager instance.	**Query A** **Field:** "value" **From:** "cm.event.processing.queue.used.count" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval)

Row Title	Title	Description	Query Details
Deadletters	Deadletter Events	Measures the number of events that were selected case manager deadletter.	**Query A** **Field:** "count" **From:** "cm.event.processing.deadletter.count" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval)
DB Connection Pool	DB Connection Pool Usage	Measures the db connection pool usage, for the selected case manager instance.	**Query A** **Field:** "value" **From:** "cm.db.connection.pool.usage" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval)
DB Connection Pool	DB Connection Pool Waiting Threads	Measures the db connection pool waiting threads, for the selected case manager instance.	**Query A** **Field:** "count" **From:** "cm.db.connection.pool.waiting" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval)
Case Manager Executor Thread Pool and Task Queue	Executor Thread Pool Usage	Measures the thread pool usage, for the selected case manager instance.	**Query A** **Field:** "value" **From:** "cm.executor.pool.threadsUsageRatio" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval)
Case Manager Executor Thread Pool and Task Queue	Tasks on Executor's Queue	Measures the the number of tasks within the executor's queue, on the selected case manager instance.	**Query A** **Field:** "value" **From:** "cm.executor.pool.taskQueueEventsCount" **Where:** "host" = '$host' $timeFilter **Group By:** time($__interval)
Case Manager HTTP Endpoints and Latencies	Case Manager HTTP Endpoints and Latencies	Measures the latency of Case Manager HTTP endpoints.	**Query A** **Field:** "count", "p99" **From:** "cm.request.api" **Where:** "host" = '$host' , $timeFilter **Group By:** "path"
Login Errors	Login Error Percent	Measures the login error percent on the selected case manager instance.	**Query A** **Field:** "value" **From:** "cm.request.login.error.percent" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval)
JVM	JVM Heap	Measures the jvm heap usage of the selected instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.memory.heap.used" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval) **Query B** **Field:** "value" **From:** "letmeknow.jvm.memory.heap.max" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval), "host"
JVM	JVM Heap Used (By Gen)	For the selected instance, considering the heap used, displays the usage of each generation.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.memory.pools.G1-Old-Gen.used" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval)
JVM	JVM Heap Used After GC (By Gen)	For the selected instance, considering the heap used, displays the usage of each generation after gc.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.memory.pools.G1-Old-Gen.used-after-gc"

Row Title	Title	Description	Query Details
			Where: "host" = '$host' , $timeFilter **Group By:** time($__interval)
JVM	JVM Heap Usage After GC	Measures the jvm heap usage after gc on the selected instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.memory.heap.after-gc.usage" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval)
JVM	JVM Garbage Collector Time	Measures the JVM GC duration on the selected instance. Long garbage collection pause durations indicate that the heap size is not adjusted for the actual usage.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.gc.G1-Old-Generation.time" **Where:** "host" = '$host' , $timeFilter **Group By:** N/A
JVM	JVM Active Threads Count	Measures the current number of active threads in the JVM on the selected instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.threads.count" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval)
JVM	JVM File Descriptors Ratio	Measures the jvm file descriptors usage ratio of the selected instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.filedescriptor" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval)

On this page

Grafana

In this page you can find a description of the recommend setup for Grafana.

Overviewâ

For some components (e.g. Pulse and Case Manager) it is recommended to have two dashboards:

- Cluster: holds the cluster view containing data for all component nodes. Ideally this page shouldn't have a filter, and dashboards should aggregated values of all nodes
- Node: holds dashboards with metrics breakdown per node. Note that some of the dashboards in the page can monitor the same metrics as the cluster page

This distinction is important because it may help pinpointing an issue with a specific node while keeping a view of the overall cluster healthiness.

info

The following example assumes the datasource is InfluxDB

To better understand the differences, imagine we want to monitor Case Manager latency when processing new events (`New Event Latency`). We would need the following pages:

- Cluster page:
  - No filters
  - `New Event Latency` dashboard for all percentiles, where:
    - We first select the `mean` value of the various metric values (`p50` , `p95` , etc), grouped by `host` and filtered by `eventType='NEW_EVENT'`
    - We select the `max` value of each metrics values without grouping by hosts
- Node page:
  - A filter by node (name, IP or any other metadata that identifies the node and can be used to filter metric queries)
  - `New Event Latency` dashboard for all percentiles, where:
    - We select the `mean` value of the various metric values (`p50` , `p95` , etc) filtered by `eventType='NEW_EVENT'` and `"host" = '$host'` . Note that the filter `$host` is based on Grafana templating feature

We can see that the overall new event processing reached around 400ms maximum. If we compare this latency value with the node example we can see that the node maximum processing time was around 300ms which means this node was not the one with the highest processing time.

info

This example refers to Case Manager `cm.event.processing.latency` metric (or `cm_event_processing_latency_*` in Prometheus)

Metrics in Prometheusâ

Feedzai products expose various metric fields in a single measurement. For instance `cm.event.processing.latency` will have, among others, `p50` and `p99` .

While in some solutions these are exported (and can be queried) in the same way, in Prometheus each measurement field is exposed as a single metric. In the above example, `p50` for the `cm.event.processing.latency` metric would be `cm_event_processing_latency_p50`.

Comparing to InfluxDB, a single query that aggregates various fields of a measurement would have to be split into `n` queries, where `n` is the amount of fields in the measurement. Considering the example in Expectation, in Prometheus it would look like following:

```
# Query A
cm_event_processing_latency_p50 {
    duration_units = "seconds",
    eventType = "NEW_EVENT",
    metricType = "APPLICATION",
    metric_type = "timer",
    rate_units = "calls/second"
}

# Query B
cm_event_processing_latency_p99 {
    duration_units = "seconds",
    eventType = "NEW_EVENT",
    metricType = "APPLICATION",
    metric_type = "timer",
    rate_units = "calls/second"
}

...
```

Copy

Component metricsâ□□

Please find below the list of Grafana suggestions for each of the available components:

- Pulse Setup
- Case Manager Setup
- Reference data Setup

On this page

Pulse Setup

In this page you can find a set of suggestions for Pulse monitoring using Grafana.

Suggestionsâ

info

These suggestions are based on the shared resources.

info

There are VM resource related metrics (CPU, Memory, IO, etc) that can also be added to Grafana. These were left out on purpose Google already has out of the box monitoring support for these resources.

These suggestions can be either new dashboards, changes to existing dashboards or completely new dashboard pages. Please find the metric description in Pulse Top Metrics.

Cluster pageâ

Currently there is only one page where we can filter metrics by each of the instances. Although useful, having a cluster view monitoring page gives a view of how the overall cluster health. Given this, our suggestion is to add a new page with the following metrics:

Row Title	Title	Description	Query Details
Healthchecks - Pulse	Login	Displays the login healthcheck. This healthcheck is carried out by pulse in order to validate the login operation. The outcome is a bool, meaning that it was successfully executed when 1, otherwise is 0.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck" = 'login' , "source" = 'pulse' , $timeFilter **Group By:** time($__interval), "host"
Healthchecks - Pulse	Messaging	Displays the messaging healthcheck. This healthcheck is carried out by pulse in order to validate the communication with messaging server. The outcome is a bool, meaning that it was successfully executed when 1, otherwise is 0.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck" = 'messaging' , "source" = 'pulse' , $timeFilter **Group By:** time($__interval), "host"
Healthchecks - Pulse	Metadata DB	Displays the metadata db healthcheck. This healthcheck is carried out by pulse in order to validate the communication with metadata db. The outcome is a bool, meaning that it was successfully executed when 1, otherwise is 0.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck" = 'metadata' , "source" = 'pulse' , $timeFilter **Group By:** time($__interval), "host"
Healthchecks - Pulse	Zookeeper	Displays the zookeeper healthcheck. This healthcheck is carried out by pulse in order to validate the communication with zookeeper. The outcome is a bool, meaning that it	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck" = 'zookeeper' , "source" =

Row Title	Title	Description	Query Details
		was successfully executed when 1, otherwise is 0.	'pulse' , $timeFilter **Group By:** time($__interval), "host"
Healthchecks - Pulse	Leadership	This healthcheck indicates the leadership status of each Pulse instance, represented by 1	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck"='leadership' , "source"='pulse' , $timeFilter **Group By:** time($__interval), "host" **Query B** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck"='leadership' , "source"='pulse' , $timeFilter **Group By:** time($__interval), "host"
Healthchecks - Pulse	Jobs Status	This healthcheck verifies that jobs can be submitted and ran in the Dataproc cluster. This healthcheck is based on a dummy job that is submitted every X configured time and reported in its metrics consumed by Telegraf. The outcome is a boolean, meaning that it was successfully executed when 1, otherwise is 0.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck" = 'job-processing-cluster' , $timeFilter **Group By:** time($__interval), "host", "source" **Query B** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck" = 'job-processing-cluster' , $timeFilter **Group By:** time($__interval), "host", "source"
Healthchecks - Apps	Login	Displays the appSlug login healthcheck. This healthcheck is carried out by the appSlug in order to validate the login operation. The outcome is a bool, meaning that it was successfully executed when 1, otherwise is 0.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck"='login' , source != 'pulse' , $timeFilter **Group By:** time($__interval), "host", "source"
Healthchecks - Apps	Leadership	This healthcheck indicates the app's leadership status, represented by 1 and 0 respectively.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck"='leadership' , "source"!='pulse' , $timeFilter **Group By:** time($__interval), "source", "host" **Query B** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck"='leadership' , "source"!='pulse' , $timeFilter **Group By:** time($__interval), "host", "source"
States	Pulse Instances State	Displays the pulse instances states: - 1: INITIAL - 2: BOOTSTRAPPING - 3: RUNNING - 4: STOPPING	**Query A** **Field:** "value" **From:** "pulse.server.state" **Where:** $timeFilter **Group By:** time($__interval), "host"
States	Pulse Apps State	Displays the pulse appSlug states: - 1: INITIAL - 3: BOOTSTRAPPING - 4: BOOTSTRAPPED	**Query A** **Field:** "value" **From:** "pulse.app.state" **Where:** $timeFilter

Row Title	Title	Description	Query Details
		- 5: STARTING - 6: STARTED - 8: APPLYING_CHANGES - 10: SELF_HEALING	**Group By:** time($__interval), "host", "appSlug" **Query B** **Field:** "value" **From:** "pulse.app.state" **Where:** $timeFilter **Group By:** time($__interval), "host", "appSlug"
Leadership Changes	Pulse Leadership Changes	Measures the number of pulse leadership changes.	**Query A** **Field:** "count" **From:** "pulse.server.leadership.count" **Where:** $timeFilter **Group By:** "host"
Leadership Changes	Pulse Apps Leadership Changes	Measures the number of appSlug leadership changes.	**Query A** **Field:** "count" **From:** "pulse.app.leadership.count" **Where:** $timeFilter **Group By:** "host", "appSlug"
Watchdog Activations	Watchdog Activations	Measures the number watchdog activation per appSlug. Watchdog activation is triggered when there is a problem with an appSlug and subsequently, needs to be recovered.	**Query A** **Field:** "count" **From:** "pulse.server.watchdog.trigger" **Where:** $timeFilter **Group By:** "host", "appSlug", "operation"
Connection Pools	Metadata DB Connection Pools Usage	Measures the metadata db connection pool usage. High values implies waiting time to access to metadata db which may degrade the component performance.	**Query A** **Field:** "value" **From:** "pulse.dbPool.usage" **Where:** appSlug = '' , $timeFilter **Group By:** time($__interval), "host", "poolName" **Query B** **Field:** "value" **From:** "pulse.dbPool.usage" **Where:** appSlug = '' , $timeFilter **Group By:** time($__interval), "host", "poolName" **Query C** **Field:** "value" **From:** "pulse.dbPool.maxPoolSize" **Where:** appSlug = '' , $timeFilter **Group By:** time($__interval), "host", "poolName"
Connection Pools	Connection Pools per App Usage	Measures the metadata connection pools usage per appSlug.	**Query A** **Field:** "value" **From:** "pulse.dbPool.usage" **Where:** appSlug != '' , $timeFilter **Group By:** time($__interval), "host", "appSlug", "poolName" **Query B** **Field:** "value" **From:** "pulse.dbPool.maxPoolSize" **Where:** appSlug != '' , $timeFilter **Group By:** time($__interval), "host", "appSlug", "poolName"
Connection Pools	Metadata DB Connection Errors and Timeouts	Measures the number of errors and timeouts occurred when trying to get a metadata db connection.	**Query A** **Field:** "count" **From:** "pulse.dbPool.connectionErrors" **Where:** appSlug = '' , $timeFilter **Group By:** time($__interval), "host", "poolName"

Row Title	Title	Description	Query Details
			Query B **Field:** "count" **From:** "pulse.dbPool.connectionObtainTimeout" **Where:** appSlug = '' , $timeFilter **Group By:** time($__interval), "host", "poolName" **Query C** **Field:** "count" **From:** "pulse.dbPool.connectionErrors" **Where:** appSlug = '' , $timeFilter **Group By:** time($__interval), "host", "poolName" **Query D** **Field:** "count" **From:** "pulse.dbPool.connectionObtainTimeout" **Where:** appSlug = '' , $timeFilter **Group By:** time($__interval), "host", "poolName"
Connection Pools	DB Connection Errors and Timeouts per App	Measures the number of errors and timeouts occurred when the appSlug tries to get a connection .	**Query A** **Field:** "count" **From:** "pulse.dbPool.connectionErrors" **Where:** appSlug != '' , $timeFilter **Group By:** time($__interval), "host", "poolName", "appSlug" **Query B** **Field:** "count" **From:** "pulse.dbPool.connectionObtainTimeout" **Where:** appSlug != '' , $timeFilter **Group By:** time($__interval), "host", "poolName", "appSlug" **Query C** **Field:** "count" **From:** "pulse.dbPool.connectionErrors" **Where:** appSlug != '' , $timeFilter **Group By:** time($__interval), "host", "poolName" **Query D** **Field:** "count" **From:** "pulse.dbPool.connectionObtainTimeout" **Where:** appSlug != '' , $timeFilter **Group By:** time($__interval), "host", "poolName"
Jobs	Jobs Scheduled	Measures the number of jobs scheduled for execution broken-down by job type; i.e., with state SCHEDULED (1).	**Query A** **Field:** "count" **From:** "pulse.job.executions.scheduled" **Where:** jobType = 'Workflow' , $timeFilter **Group By:** time($__interval), *
Jobs	Jobs Waiting	Measures the number of jobs that are waiting for an agent to be free broken-down by job type; i.e., with state WAITING_AGENT (8).	**Query A** **Field:** "count" **From:** "pulse.job.executions.waiting" **Where:** $timeFilter **Group By:** time($__interval), *
Jobs	Jobs Executing	Measures the number of jobs that are currently in state EXECUTING (2) broken-down by job type.	**Query A** **Field:** "count" **From:** "pulse.job.executions.running" **Where:** jobType = 'Workflow' , $timeFilter **Group By:** time($__interval), *
Jobs	Jobs Failed	Measures the number of jobs that failed broken-down by both job type and	**Query A** **Field:** "count" **From:** "pulse.job.executions.failed"

Row Title	Title	Description	Query Details
		state: ERROR (5), CRASHED (6), ABORTED (7).	**Where:** jobType = 'Workflow' , $timeFilter **Group By:** time($__interval), ***Query B*** ***Field:*** *"count"* ***From:*** *"pulse.job.executions.failed"* ***Where:*** *jobType = 'Workflow' , $timeFilter* ***Group By:*** *time($__interval),*
Jobs	Jobs Finished	Measures the number of jobs that completed successfully broken-down by job type; i.e., with state FINIDHED (3).	**Query A** **Field:** "count" **From:** "pulse.job.executions.success" **Where:** jobType = 'Workflow' , $timeFilter **Group By:** time($__interval), * **Query B** **Field:** "count" **From:** "pulse.job.executions.success" **Where:** jobType = 'Workflow' , $timeFilter **Group By:** time($__interval), "host", "appSlug"
Workflow	Global Workflow Throughput per Message Type	Measures the overall appSlug workflow throughput per message type(normal, recovery, replica).	**Query A** **Field:** "count" **From:** "pulse.workflow.global.event.stats" **Where:** $timeFilter **Group By:** time($__interval), "host", "appSlug", "workflowDesc", "messageType"
Workflow	Global Workflow Errors	Measures the overall appSlug workflow errors, considering all message types(normal, recovery and replica).	**Query A** **Field:** "count" **From:** "pulse.workflow.eventErrors.count" **Where:** $timeFilter **Group By:** time($__interval), "host", "appSlug", "workflowDesc"
Workflow	Global Workflow latency per Message Type at Percentile 99	Measures the overall workflow latency at percentile 99 per pulse instance, appSlug, workflow and message type.	**Query A** **Field:** "p99" **From:** "pulse.workflow.global.event.stats" **Where:** "messageType" != 'RECOVERY' , $timeFilter **Group By:** time($__interval), "host", "appSlug", "workflowDesc", "messageType" **Query B** **Field:** "p99" **From:** "pulse.workflow.global.event.stats" **Where:** "messageType" = 'RECOVERY' , $timeFilter **Group By:** time($__interval), "host", "appSlug", "workflowDesc" **Query C** **Field:** "p99" **From:** "pulse.workflow.global.event.stats" **Where:** "messageType" = 'NORMAL' , $timeFilter **Group By:** time($__interval), "host", "appSlug", "workflowDesc"
Workflow	Events Spilled to Disk	Measures the overall appSlug events spilled to disk. An event is spilled to disk when pulse cannot write it on the event storage. This scenario must be avoided, otherwise require human action to move the events from the disk to the event storage.	**Query A** **Field:** "count" **From:** "pulse.eventStorage.spillToDisk.count" **Where:** $timeFilter **Group By:** time($__interval), "host", "appSlug", "workflowDesc"

Row Title	Title	Description	Query Details
Workflow	Global Workflow Throughput by API endpoint	Measures the overral appSlug workflow throughput of normal messages by Endpoint (initiation, info, etc).	**Query B** **Field:** "count" **From:** "workflow.events" **Where:** $timeFilter **Group By:** time($__interval), "workflowDesc", "elementDesc", "host", "appSlug", "event_type"
Event Storage Dumps	Events Storage Dump Failed and Retry Events	Measures the overall number of events that were dumped to disk and are either pending to be retried or reached to the max amount of retries and need to be re-injected manually.	**Query A** **Field:** "count" **From:** "pulse.dump.storage.dump.entries.retry.counter" **Where:** $timeFilter **Group By:** time($__interval), "host", "appSlug", "workflowDesc" **Query B** **Field:** "count" **From:** "pulse.dump.storage.dump.entries.failed.counter" **Where:** $timeFilter **Group By:** time($__interval), "host", "appSlug", "workflowDesc"
Event Storage Dumps	Events Storage Dump Failed and Retry Queue Size	Measures the overall number of events that are present in the in-memory queue to be written to the dump file in the failed folder, appSlug and workflow.	**Query A** **Field:** "value" **From:** "pulse.dump.storage.failed.queue.size" **Where:** $timeFilter **Group By:** time($__interval), "host", "appSlug", "workflowDesc" **Query B** **Field:** "value" **From:** "pulse.dump.storage.retry.queue.size" **Where:** $timeFilter **Group By:** time($__interval), "host", "appSlug", "workflowDesc"
Event Pending Dumps	Event Pending Persistence to Event Storage	Measures the overall number of events that are pending to be persisted to the databased.	**Query A** **Field:** "count" **From:** "pulse.workflow.eventstorage.eventsQueuedSize" **Where:** $timeFilter **Group By:** time($__interval), "host", "appSlug", "workflowDesc"
Profile Fetches	Profile Fetches Latency at Percentile 99 (Excluding Timeouts)	Measures the overall max latency to fetch the profiles at percentile 99, per appSlug, workflow and plan.	**Query A** **Field:** "p99" **From:** "pulse.workflow.profileFetches.statistics" **Where:** $timeFilter **Group By:** time($__interval), "host", "appSlug", "workflowDesc", "elementDesc"
Profile Fetches	Profile Fetches Timeouts	Measures the number of profile fetches that were timed out, per appSlug, workflow and plan.	**Query A** **Field:** "count" **From:** "pulse.workflow.profileFetches.timeouts" **Where:** $timeFilter **Group By:** time($__interval), "host", "appSlug", "workflowDesc", "elementDesc"
JVM	JVM Heap Usage (Pulse Server)	Measures the jvm heap usage per pulse instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.memory.heap.usage" **Where:** appSlug = '' , $timeFilter **Group By:** time($__interval), "host"

Row Title	Title	Description	Query Details
			Query B **Field:** "value" **From:** "letmeknow.jvm.memory.heap.usage" **Where:** appSlug = '' , $timeFilter **Group By:** time($__interval), "host"
JVM	JVM Heap Usage (per App)	Measures the jvm heap usage per application and host.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.memory.heap.usage" **Where:** appSlug != '' , $timeFilter **Group By:** time($__interval), "host", "appSlug" **Query B** **Field:** "value" **From:** "letmeknow.jvm.memory.heap.usage" **Where:** appSlug != '' , $timeFilter **Group By:** time($__interval), "host", "appSlug" **Query C** **Field:** "value" **From:** "letmeknow.jvm.memory.heap.usage" **Where:** appSlug != '' , $timeFilter **Group By:** time($__interval), "host", "appSlug"
JVM	JVM Heap Usage After GC (Pulse Server)	Measures the jvm heap usage after gc per pulse instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.memory.heap.after-gc.usage" **Where:** appSlug = '' , $timeFilter **Group By:** time($__interval), "host"
JVM	JVM Heap Usage After GC (per App)	Measures the jvm heap usage after gc per application and host.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.memory.heap.after-gc.usage" **Where:** appSlug != '' , $timeFilter **Group By:** time($__interval), "host", "appSlug"
JVM	JVM Code Cache Usage (Pulse Server)	Measures the jvm code cache usage per pulse instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.memory.pools.CodeCache.usage" **Where:** appSlug = '' , $timeFilter **Group By:** time($__interval), "host"
JVM	JVM Code Cache Usage (per App)	Measures the jvm code cache usage per application and host.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.memory.pools.CodeCache.usage" **Where:** appSlug != '' , $timeFilter **Group By:** time($__interval), "host", "appSlug"
JVM	JVM Garbage Collector Time (Pulse Server)	Measures maximum JVM GC duration. Long garbage collection pause durations indicate that the heap size is not adjusted for the actual usage.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.gc.G1-Old-Generation.time" **Where:** appSlug = '' , $timeFilter **Group By:** "host" **Query B** **Field:** "value" **From:** "letmeknow.jvm.gc.G1-Young-Generation.time" **Where:** appSlug = '' , $timeFilter **Group By:** "host"
JVM	JVM Garbage Collector Time (per App)	Measures maximum JVM GC duration for young and old gen, per application and host. Long garbage collection	**Query A** **Field:** "value" **From:** "letmeknow.jvm.gc.G1-Old-Generation.time"

Row Title	Title	Description	Query Details
		pause durations indicate that the heap size is not adjusted for the actual usage.	**Where:** appSlug != '' , $timeFilter **Group By:** "host", "appSlug" **Query B** **Field:** "value" **From:** "letmeknow.jvm.gc.G1-Young-Generation.time" **Where:** appSlug != '' , $timeFilter **Group By:** "host", "appSlug"
JVM	JVM Active Threads Count (Pulse Server)	Measures the current number of active threads in the JVM per pulse instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.threads.count" **Where:** appSlug = '' , $timeFilter **Group By:** time($__interval), "host"
JVM	JVM Active Threads Count (per App)	Measures the current number of active threads in the JVM per application and host.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.threads.count" **Where:** appSlug != '' , $timeFilter **Group By:** time($__interval), "host", "appSlug"
JVM	JVM File Descriptors Ratio (Pulse Server)	Measures the jvm file descriptors usage ratio per instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.filedescriptor" **Where:** appSlug = '' , $timeFilter **Group By:** time($__interval), "host"
JVM	JVM File Descriptors Ratio (per App)	Measures the jvm file descriptors usage ratio per application and host.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.filedescriptor" **Where:** appSlug != '' , $timeFilter **Group By:** time($__interval), "host", "appSlug"

Note that in the cluster page some dashboards are separated. For instance, instead of having the `Jobs` dashboards separated by job state, we can have a dashboard that gathers all states. Given that in cluster view we are not filtering by any node, it is recommend to have disassociated dashboards so they do not get cluttered with information.

Also note the `Row Title` column which refers to a Grafana feature to group dashboards by type. This is only a suggestion and thus not mandatory.

Node pageâ□□

These suggestions relate to the existing Pulse dashboard page. You may find some new dashboards or changes to existing ones.

Our suggestions are to add the dashboards in the following table with a filter by each of the available nodes and applications:

Row Title	Title	Description	Query Details
Healthchecks - Pulse	Login	Displays the login healthcheck on the selected instance. This healthcheck is carried out by pulse in order to validate the login operation. The outcome is a bool, meaning that it was successfully executed when 1, otherwise is 0.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "source" = 'pulse' , "healthcheck" = 'login' , "host" = '$host' , $timeFilter **Group By:** time($__interval)

Row Title	Title	Description	Query Details
Healthchecks - Pulse	Messaging	Displays the messaging healthcheck on the selected instance. This healthcheck is carried out by pulse in order to validate the communication with rabbitmq. The outcome is a bool, meaning that it was successfully executed when 1, otherwise is 0.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "source" = 'pulse' , "healthcheck" = 'messaging' , "host" = '$host' , $timeFilter **Group By:** time($__interval)
Healthchecks - Pulse	Metadata DB	Displays the metadata db healthcheck on the selected instance. This healthcheck is carried out by pulse in order to validate the communication with metadata db. The outcome is a bool, meaning that it was successfully executed when 1, otherwise is 0.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "source" = 'pulse' , "healthcheck" = 'metadata' , "host" = '$host' , $timeFilter **Group By:** time($__interval)
Healthchecks - Pulse	Zookeeper	Displays the zookeeper healthcheck on the selected instance. This healthcheck is carried out by pulse in order to validate the communication with zookeeper. The outcome is a bool, meaning that it was successfully executed when 1, otherwise is 0.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "source" = 'pulse' , "healthcheck" = 'zookeeper' , "host" = '$host' , $timeFilter **Group By:** time($__interval)
Pulse State and Leadership Changes	Pulse Leadership Changes	Displays pulse leadership changes, for the selected pulse instance.	**Query A** **Field:** "count" **From:** "pulse.server.leadership.count" **Where:** "host" = '$host' , $timeFilter **Group By:** N/A
Healthchecks - Apps	Login	Displays the login healthcheck for the specified app on the selected instance. This healthcheck is carried out by the appSlug in order to validate the login operation. The outcome is a bool, meaning that it was successfully executed when 1, otherwise is 0.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck"='login' , source =~ /^app.$appSlug.*/ , "host" = '$host' , $timeFilter **Group By:** time($__interval)
Healthchecks - Apps	Leadership	Displays the leadership healthcheck for the specified app on the selected instance. This healthcheck is carried out by the appSlug in order to validate the login operation. The outcome is a bool, meaning that it was successfully executed when 1, otherwise is 0.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck"='leadership' , source =~ /^app.$appSlug.*/ , "host" = '$host' , $timeFilter **Group By:** time($__interval)
Watchdog Activations	Watchdog Activations	Measures the number of watchdog activations triggered for the selected appSlug and the specified pulse instance.	**Query A** **Field:** "count" **From:** "pulse.server.watchdog.trigger" **Where:** "host" = '$host' , "appSlug" = '$appSlug' , $timeFilter **Group By:** "operation"
Jobs	Jobs Scheduled	Measures the number of jobs scheduled for execution broken-down by job type; i.e., with state SCHEDULED (1).	**Query A** **Field:** "count" **From:** "pulse.job.executions.scheduled" **Where:** jobType = 'Workflow' , "host" = '$host' , "appSlug" = '$appSlug' , $timeFilter **Group By:** time($__interval), "host", "appSlug", "jobType"
Jobs	Jobs Waiting	Measures the number of jobs that are waiting for an agent to be free broken-down by job type; i.e., with state WAITING_AGENT (8).	**Query A** **Field:** "count" **From:** "pulse.job.executions.waiting" **Where:** jobType = 'Workflow' , "host" = '$host' ,

Row Title	Title	Description	Query Details
			"appSlug" = '$appSlug' , $timeFilter **Group By:** time($__interval), "host", "appSlug", "jobType"
Jobs	Jobs Executing	Measures the number of jobs that are currently in state EXECUTING (2) broken-down by job type.	**Query A** **Field:** "count" **From:** "pulse.job.executions.running" **Where:** jobType = 'Workflow' , "host" = '$host' , "appSlug" = '$appSlug' , $timeFilter **Group By:** time($__interval), "host", "appSlug", "jobType"
Jobs	Jobs Failed	Measures the number of jobs that failed broken-down by both job type and state: ERROR (5), CRASHED (6), ABORTED (7).	**Query A** **Field:** "count" **From:** "pulse.job.executions.failed" **Where:** jobType = 'Workflow' , "host" = '$host' , "appSlug" = '$appSlug' , $timeFilter **Group By:** time($__interval), "host", "appSlug", "jobType"
Jobs	Jobs Finished	Measures the number of jobs that completed successfully broken-down by job type; i.e., with state FINIDHED (3).	**Query A** **Field:** "count" **From:** "pulse.job.executions.success" **Where:** jobType = 'Workflow' , "host" = '$host' , "appSlug" = '$appSlug' , $timeFilter **Group By:** time($__interval), "host", "appSlug", "jobType"

On this page

Reference Data Setup

In this page you can find a set of suggestions for Reference Data monitoring using Grafana.

Suggestionsâ□□

info

There are VM resource related metrics (CPU, Memory, IO, etc) that can also be added to Grafana. These were left out on purpose Google already has out of the box monitoring support for these resources.

These suggestions can be either new dashboards, changes to existing dashboards or completely new dashboard pages. Please find the metric description in Reference Data Top Metrics.

Cluster pageâ□□

Currently there is only one page where we can filter metrics by each of the instances. Although useful, having a cluster view monitoring page gives a view of how the overall cluster health. Given this, our suggestion is to add a new page with the following metrics:

Row Title	Title	Description	Query Details
Healthchecks - RDS	Reference Data Service	Displays reference data server healthcheck. The outcome is a bool, meaning that it assumes the value one when everything is okay with the service itself, otherwise is 0.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck" = 'reference-data-server' , $timeFilter **Group By:** time($__interval), "host"
Healthchecks - RDS	Deadlocks	Displays deadlocks healthcheck. This healthcheck is carried out by reference data service in order to validate if some of its threads entered in deadlock. The outcome is a bool, meaning that it was successfully executed when 1, otherwise is 0.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck" = 'deadlocks' , $timeFilter **Group By:** time($__interval), "host"
Healthchecks - RDS	Entity Repository	Displays entity repository healthcheck. This healthcheck is carried out by reference data service in order to validate if it is able to reach entity repository, which is an abstraction for a non-sql db. The outcome is a bool, meaning that it was successfully executed when 1, otherwise is 0.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck" = 'entity-repository' , $timeFilter **Group By:** time($__interval), "host"
HTTP Requests	HTTP Throughput	Measures reference-data HTTP throughput.	**Query A** **Field:** "count"

Row Title	Title	Description	Query Details
			From: "io.dropwizard.jetty.MutableServletContextHandler.requests" **Where:** $timeFilter **Group By:** time($interval), "host"
HTTP Requests	HTTP Throughput per Method	Measures reference-data HTTP throughput per method.	**Query A** **Field:** "count" **From:** "io.dropwizard.jetty.MutableServletContextHandler.get-requests" **Where:** $timeFilter **Group By:** time($interval), "host"
HTTP Requests	2XX Requests Rate	Measures reference-data HTTP throughput with return code falls into 2xx.	**Query A** **Field:** "count" **From:** "io.dropwizard.jetty.MutableServletContextHandler.2xx-responses" **Where:** $timeFilter **Group By:** time($interval), "host"
HTTP Requests	4XX Errors	Measures the number of 4xx errors returned by reference-data.	**Query A** **Field:** "count" **From:** "io.dropwizard.jetty.MutableServletContextHandler.4xx-responses" **Where:** $timeFilter **Group By:** time($interval), "host"
HTTP Requests	5XX Errors	Measures the number of 5xx errors returned by reference-data.	**Query A** **Field:** "count" **From:** "io.dropwizard.jetty.MutableServletContextHandler.5xx-responses" **Where:** $timeFilter **Group By:** time($interval), "host"
Global Database Requests	Global Database Throughput	Measures the overall database throughput, considering all operations executed by reference-data. It also shows a break down per operation.	**Query A** **Field:** "count" **From:** "reference-data-service.entity-repository" **Where:** $timeFilter **Group By:** time($interval), "host", "operation"
Global Database Requests	Global Database Latency at Percentile 99	Measures global database latency at percentile 99 considering all operations executed by reference-data.	**Query A** **Field:** "p99" **From:** "reference-data-service.entity-repository" **Where:** $timeFilter **Group By:** time($__interval), "host", "operation"
Global Database Requests	Global Database Errors	Measures the total number of errors registered while performing some operations onto the database.	**Query A** **Field:** "count" **From:** "reference-data-service.entity-repository.error" **Where:** $timeFilter **Group By:** time($__interval), "host"
Global Database Requests	Global Database Errors - Entities Not Found	Measures the total number of errors caused by entities not found, registered while performing some operations onto the database.	**Query A** **Field:** "count" **From:** "reference-data-service.entity-repository.entitynotfound" **Where:** $timeFilter **Group By:** time($__interval), "host"
Database Operations	Database Throughput Per Entity	Measures reference-data throughput per entity.	**Query A** **Field:** "count" **From:** "reference-data-service.entity-repository" **Where:** $timeFilter **Group By:** time($interval), "host", "entityType", "operation"
	Global Database Latency at	Measures database latency at percentile 99 per entity,	**Query A** **Field:** "p99"

Row Title	Title	Description	Query Details
Database Operations	Percentile 99 per Entity	considering all operations executed by reference-data.	**From:** "reference-data-service.entity-repository" **Where:** $timeFilter **Group By:** time($__interval), "host", "entityType", "operation"
JVM	JVM Heap Usage	Measures the jvm heap usage per component instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.memory.heap.usage" **Where:** $timeFilter **Group By:** time($__interval), "host"
JVM	JVM Heap Usage After GC	Measures the jvm heap usage after gc per instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.memory.heap.after-gc.usage" **Where:** $timeFilter **Group By:** time($__interval), "host"
JVM	JVM Garbage Collector Time	Measures maximum JVM GC duration. Long garbage collection pause durations indicate that the heap size is not adjusted for the actual usage.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.gc.G1-Old-Generation.time" **Where:** $timeFilter **Group By:** "host"
JVM	JVM Active Threads Count	Measures the current number of active threads in the JVM per instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.threads.count" **Where:** $timeFilter **Group By:** time($__interval), "host"
JVM	JVM File Descriptors Ratio	Measures the jvm file descriptors usage ratio per instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.filedescriptor" **Where:** $timeFilter **Group By:** time($__interval), "host"

The `Row Title` column refers to a Grafana feature to group dashboards by type. This is only a suggestion and thus not mandatory.

Node pageâ□□

These suggestions relate to the existing Reference Data dashboard page. You may find some new dashboards or changes to existing ones.

Our suggestions are to add the dashboards in the following table with a filter by each of the available nodes and applications:

Row Title	Title	Description	Query Details
Healthchecks - RDS	Reference Data Service	Displays reference data server healthcheck. The outcome is a bool, meaning that it assumes the value one when everything is okay with the service itself, otherwise is 0.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck" = 'reference-data-server' , "host" = '$host' , $timeFilter **Group By:** time($__interval)
Healthchecks - RDS	Deadlocks	Displays deadlocks healthcheck. This healthcheck is carried out by reference data service in order to validate if some of its threads entered in deadlock. The outcome is a bool, meaning that it was	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck" = 'deadlocks' , "host" = '$host' , $timeFilter **Group By:** time($__interval)

Row Title	Title	Description	Query Details
		successfully executed when 1, otherwise is 0.	
Healthchecks - RDS	Entity Repository	Displays entity repository healthcheck. This healthcheck is carried out by reference data service in order to validate if it is able to reach entity repository, which is an abstraction for a non-sql db. The outcome is a bool, meaning that it was successfully executed when 1, otherwise is 0.	**Query A** **Field:** "result" **From:** "healthchecks" **Where:** "healthcheck" = 'entity-repository' , "host" = '$host' , $timeFilter **Group By:** time($__interval)
HTTP Requests	HTTP Throughput	Measures reference-data service HTTP throughput on the selected instance.	**Query A** **Field:** "count" **From:** "io.dropwizard.jetty.MutableServletContextHandler.requests" **Where:** "host" = '$host' , $timeFilter **Group By:** time($interval), "host"
HTTP Requests	HTTP Throughput per Method	Measures reference-data service HTTP throughput per method on the selected instance.	**Query A** **Field:** "count" **From:** "io.dropwizard.jetty.MutableServletContextHandler.get-requests" **Where:** "host" = '$host' , $timeFilter **Group By:** time($interval), "host"
HTTP Requests	2XX Requests Rate	Measures reference-data service HTTP throughput, on the selected instance, with return code falls into 2xx.	**Query A** **Field:** "count" **From:** "io.dropwizard.jetty.MutableServletContextHandler.2xx-responses" **Where:** "host" = '$host' , $timeFilter **Group By:** time($interval), "host"
HTTP Requests	4XX Errors	Measures the number of 4xx errors returned by the selected instance.	**Query A** **Field:** "count" **From:** "io.dropwizard.jetty.MutableServletContextHandler.4xx-responses" **Where:** "host" = '$host' , $timeFilter **Group By:** time($interval), "host"
HTTP Requests	5XX Errors	Measures the number of 5xx errors returned by the selected instance.	**Query A** **Field:** "count" **From:** "io.dropwizard.jetty.MutableServletContextHandler.5xx-responses" **Where:** "host" = '$host' , $timeFilter **Group By:** time($interval), "host"fill(none)
Global Database Requests	Global Database Throughput	Measures the overall database throughput, considering all operations executed by the selected instance. It also shows a break down per operation.	**Query A** **Field:** "count" **From:** "reference-data-service.entity-repository" **Where:** "host" = '$host' , $timeFilter **Group By:** time($interval), "host", "operation"
Global Database Requests	Global Database Errors	Measures the total number of errors registered, on the selected instance, while performing some operations onto the database.	**Query A** **Field:** "count" **From:** "reference-data-service.entity-repository.error" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval), "host"

Row Title	Title	Description	Query Details
Global Database Requests	Global Database Errors - Entities Not Found	Measures the total number of errors caused by entities not found, registered on the selected instance while performing some operations onto the database.	**Query A** **Field:** "count" **From:** "reference-data-service.entity-repository.entitynotfound" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval), "host"fill(none)
Global Database Requests	Global Database Latency per Percentile for Operation	Measures global database latency at percentile 99 considering all operations executed by the selected instance.	**Query A** **Field:** "p50", "p95", "p99", "p999", "max" **From:** "reference-data-service.entity-repository" **Where:** "host" = '$host' , operation='$operation' , $timeFilter **Group By:** time($__interval), "host"
Database Operation	Operation throughput for Entity	Measures the database throughput, considering the operations executed by the selected instance using the selected entity type.	**Query A** **Field:** "count" **From:** "reference-data-service.entity-repository" **Where:** "host" = '$host' , operation='$operation' , entityType = '$entity' , $timeFilter **Group By:** time($__interval), "host", "operation"
Database Operation	Operation for Entity Latency Per Percentile	Measures the latency per percentile to execute the specified operation on the selected entity type using the selected instance.	**Query A** **Field:** "p50", "p95", "p99", "p999", "max" **From:** "reference-data-service.entity-repository" **Where:** "host" = '$host' , operation='$operation' , entityType = '$entity' , $timeFilter **Group By:** time($__interval), "host", "operation"
JVM	JVM Heap	Measures the jvm heap usage of the selected instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.memory.heap.used" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval) **Query B** **Field:** "value" **From:** "letmeknow.jvm.memory.heap.max" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval), "host"
JVM	JVM Heap Used (By Gen)	For the selected instance, considering the heap used, displays the usage of each generation.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.memory.pools.G1-Old-Gen.used" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval)
JVM	JVM Heap Used After GC (By Gen)	For the selected instance, considering the heap used, displays the usage of each generation after gc.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.memory.pools.G1-Old-Gen.used-after-gc" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval)
JVM	JVM Heap Usage After GC	Measures the jvm heap usage after gc on the selected instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.memory.heap.after-gc.usage" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval)
JVM	JVM Garbage Collector Time	Measures the JVM GC duration on the selected instance. Long garbage collection pause durations indicate that the heap size is not adjusted for the actual usage.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.gc.G1-Old-Generation.time" **Where:** "host" = '$host' , $timeFilter **Group By:** N/A

Row Title	Title	Description	Query Details
JVM	JVM Active Threads Count	Measures the current number of active threads in the JVM on the selected instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.threads.count" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval)
JVM	JVM File Descriptors Ratio	Measures the jvm file descriptors usage ratio of the selected instance.	**Query A** **Field:** "value" **From:** "letmeknow.jvm.filedescriptor" **Where:** "host" = '$host' , $timeFilter **Group By:** time($__interval)